In [207]:

```
import array
import cmath
import collections
import copy
import csv
import datetime
import decimal
import functools
import importlib
import json
import logging
import math
import os
import pprint
import random
import re
import statistics
import string
import sys
import time
import timeit

import dateutil
import matplotlib as mpl
import matplotlib.pyplot as plt
import numpy as np
import pandas
import pyaml
import pytz
import requests
```

In [2]:

```
num1 = 10_00_000
print(num1)
```

```
1000000
```

In [3]:

```
print(format(0.1, ".25f"))
```

```
0.1000000000000000055511151
```

In [4]:

```
print(format(0.125, ".25f"))
```

```
0.1250000000000000000000000
```

In [5]:

```
print(0.1 + 0.1 + 0.1 == 0.3)
```

```
False
```

In [6]:

```
print(format(0.3, ".25f"))
```

```
0.2999999999999999888977698
```

In [7]:
```
print(format(0.2, ".25f"))
```

```
0.2000000000000000111022302
```

In [8]:
```
print(abs((0.1 + 0.1 + 0.1) - 0.3) < 0.001)
```

```
True
```

In [9]:
```
print((0.125).as_integer_ratio())
```

```
(1, 8)
```

In [10]:
```
print((10).       (25))
```

```
35
```

In [11]:
```
a = float(10)
print(a)
```

```
10.0
```

In [12]:
```
c = (-4) ** 0.5
print(c)
```

```
(1.2246467991473532e-16+2j)
```

In [13]:
```
print(type(c))
```

```
<class 'complex'>
```

In [14]:
```
print(c.real)
```

```
1.2246467991473532e-16
```

In [15]:
```
print(c.imag)
```

```
2.0
```

In [16]:
```
c = 10 + 2j
print(type(c))
```

```
<class 'complex'>
```

In [17]:
```
print(c)
```

```
(10+2j)
```

In [18]:
```
try:
        print("2" < 2)
except TypeError as ex:
        print(type(ex).       )
```

```
TypeError
```

In [19]:
```
print("2" == 2)
```

```
False
```

In [20]:
```
a = 0
b = 1
c = 500
d = 0
e = 1
f = 500
print(id(a), id(b), id(c), id(d), id(e), id(f))
```

```
11370064 11370096 128573173327504 11370064 11370096 128573173326608
```

In [21]:
```
print(0.1 * 3 - 0.3 < abs(1e-6))
```

```
True
```

In [22]:
```
a = 257
print(a is 257)
```

```
False
```

```
<>:2: SyntaxWarning: "is" with 'int' literal. Did you mean "=="?
<>:2: SyntaxWarning: "is" with 'int' literal. Did you mean "=="?
/tmp/ipykernel_18021/1958631963.py:2: SyntaxWarning: "is" with 'int' literal. Did you mean "=="?
  print(a is 257)
```

In [23]:
```
a = 10
b = 0
print(b != 0 and a / b)
```

```
False
```

In [24]:
```
a = 1e-6
b = 2e-6
tol = 1e-7
print(abs(a - b) <= tol)
```

```
False
```

In [25]:
```
elapsed = 7835
hours = elapsed // 3600
minutes = (elapsed - (hours * 3600)) // 60
seconds = elapsed - ((hours * 3600) + (minutes * 60))
print(hours, minutes, seconds)
```

```
2 10 35
```

In [26]:
```
elapsed = 7835
hours = elapsed // (60 * 60)
minutes = (elapsed % (60 * 60)) // 60
seconds = (elapsed % (60 * 60)) % 60
print(hours, minutes, seconds)
```

```
2 10 35
```

In [27]:
```
a = None
if a == None:
```

```
    a = "N/A"
print(a)
```

```
N/A
```

In [28]:
```
a = None
a = "N/A" if a is None else a
print(a)
```

```
N/A
```

In [29]:
```
score = 700
rating = "Invalid"
if score < 0:
        rating = "Negative"
elif score < 580:
        rating = "Poor"
elif score < 670:
        rating = "Fair"
elif score < 740:
        rating = "Good"
elif score < 800:
        rating = "Very Good"
elif score < 850:
        rating = "Excellent"
print(rating)
```

```
Good
```

In [30]:
```
elapsed = 1000000
magnitude = "Invalid"
if elapsed < 60:
        magnitude = "seconds"
elif elapsed < (60 * 60):
        magnitude = "minutes"
elif elapsed < (60 * 60 * 24):
        magnitude = "hours"
elif elapsed < (60 * 60 * 24 * 7):
        magnitude = "days"
elif elapsed > (60 * 60 * 24 * 7):
        magnitude = "weeks"
print(magnitude)
```

```
weeks
```

In [31]:
```
t = [1, 2], 3, 4
l = list(t)
print(l)
```

```
[[1, 2], 3, 4]
```

In [32]:
```
l[0][1] = 20
print(l)
```

```
[[1, 20], 3, 4]
```

In [33]:
```
print(t)
```

```
([1, 20], 3, 4)
```

In [34]:
```
for i in t:
        print(id(i))
```

```
128573172499456
11370160
11370192
```

In [35]:
```
for i in l:
        print(id(i))
```

```
128573172499456
11370160
11370192
```

In [36]:
```
for i, j in zip(l, t):
        print(i is j)
```

```
True
True
True
```

In [37]:
```
s = "ABCDEFGHIJ"
print(s[3:6:-1])
```

In [38]:
```
print(s[3:5 + 1])
```

```
DEF
```

In [39]:
```
m1 = [[0, 0, 0], [0, 0, 0], [0, 0, 0]]
m2 = m1[0:2]
print(m1[0] is m2[1])
```

```
False
```

In [40]:
```
m1[0] = "Python"
print(m1)
```

```
['Python', [0, 0, 0], [0, 0, 0]]
```

In [41]:
```
print(m2)
```

```
[[0, 0, 0], [0, 0, 0]]
```

In [42]:
```
m1[1][0] = 100
```

In [43]:
```
print(m1)
```

```
['Python', [100, 0, 0], [0, 0, 0]]
```

In [44]:
```
print(m2)
```

```
[[0, 0, 0], [100, 0, 0]]
```

In [45]:
```
print("abcdef"[-1:-4])
```

In [46]:
```
print("abcdef"[1:4:-1])
```

In [47]:
```
print("racecar" == "racecar"[::-1])
```

```
True
```

In [48]:
```
list1 = [0, 1, 2, 3, 4]
list1[0:3] = "abc"
print(list1)
```

```
['a', 'b', 'c', 3, 4]
```

In [49]:
```
list1[0:3] = ["a", "b", "c"]
print(list1)
```

```
['a', 'b', 'c', 3, 4]
```

In [50]:
```
list1[0:3] = ["a", "b"]
print(list1)
```

```
['a', 'b', 3, 4]
```

In [51]:
```
del list1[2]
print(list1)
```

```
['a', 'b', 4]
```

In [52]:
```
del list1[0:2]
print(list1)
```

```
[4]
```

In [53]:
```
list1.extend("ABC")
print(list1)
```

```
[4, 'A', 'B', 'C']
```

In [54]:
```
list1.insert(0, "YO!")
print(list1)
```

```
['YO!', 4, 'A', 'B', 'C']
```

In [55]:
```
list1[:-3:-1] = "cb"
print(list1)
```

```
['YO!', 4, 'A', 'b', 'c']
```

In [56]:
```
l = []
print(timeit.timeit("l.append(1)", globals = globals(), number = 100_000))
```

```
0.002112314999976661
```

In [57]:
```
print(len(l))
```

```
100000
```

In [58]:
```
list2 = []
print(timeit.timeit("list2.insert(0, 1)", globals = globals(), number =
100_000))
```

```
1.5377864980000027
```

In [59]:
```
print(len(list2))
```

```
100000
```

In [60]:
```
print(1.249788 / 0.0018043)
```

```
692.6719503408524
```

In [61]:
```
list2 = []
print(timeit.timeit("list2.insert(len(list2), 1)", globals = globals(), number
= 100_000))
```

```
0.008461738999983481
```

In [62]:
```
print(0.0057634 / 0.0018043)
```

```
3.194258161059691
```

In [63]:
```
l1 = [[0, 1], 2, "3"]
l2 = l1[:]
for i in l1:
        print(id(i))
```

```
128573172569024
11370128
11434576
```

In [64]:
```
for i in l2:
        print(id(i))
```

```
128573172569024
11370128
11434576
```

In [65]:
```
l2[0][0] = "0"
for i in l1:
        print(id(i))
```

```
128573172569024
11370128
11434576
```

In [66]:
```
for i in l2:
        print(id(i))
```

```
128573172569024
11370128
11434576
```

In [67]:
```
for i in l1:
        print(i)
```

```
['0', 1]
2
3
```

In [68]:
```
for i in l2:
        print(i)
```

```
['0', 1]
2
3
```

In [69]:
```
l2[1] = "22"
for i in l1:
        print(id(i))
```

```
128573172569024
11370128
11434576
```

In [70]:
```
for i in l2:
        print(id(i))
```

```
128573172569024
128574645937104
11434576
```

In [71]:
```
for i in l1:
        print(i)
```

```
['0', 1]
2
3
```

In [72]:
```
for i in l2:
        print(i)
```

```
['0', 1]
22
3
```

In [73]:
```
l1 = [[0, 1], 2, "3"]
l2 = l1[:]
l2[0] = ["0", 1]
for i in l1:
        print(id(i))
```

```
128573172599040
11370128
11434576
```

In [74]:
```
for i in l2:
        print(id(i))
```

```
128573172596864
11370128
11434576
```

In [75]:
```
for i in l1:
        print(i)
```

```
[0, 1]
2
3
```

In [76]:
```
for i in l2:
        print(i)
```

```
['0', 1]
2
3
```

In [77]:
```
l1 = [[0, 1], 2, "3"]
l2 = l1[:]
print(id(l1), id(l2))
```

```
128573172599104 128573172598912
```

In [78]:
```
for i, j in zip(l1, l2):
        print(id(i), id(j))
```

```
128573172598848 128573172598848
11370128 11370128
11434576 11434576
```

In [79]:
```
for i, j in zip(l1[0], l2[0]):
        print(id(i), id(j))
```

```
11370064 11370064
11370096 11370096
```

In [80]:
```
l1 = [[0, 1], 2, "3"]
l2 = l1.copy()
print(id(l1), id(l2))
```

```
128573172597248 128573172597312
```

In [81]:
```
for i, j in zip(l1, l2):
        print(id(i), id(j))
```

```
128573172601472 128573172601472
11370128 11370128
11434576 11434576
```

In [82]:
```
for i, j in zip(l1[0], l2[0]):
        print(id(i), id(j))
```

```
11370064 11370064
11370096 11370096
```

In [83]:
```
l1 = [[0, 1], 2, "3"]
l2 = copy.deepcopy(l1)
print(id(l1), id(l2))
```

```
128574051537216 128573172602368
```

In [84]:
```
for i, j in zip(l1, l2):
        print(id(i), id(j))
```

```
128573172600640 128573172437184
11370128 11370128
11434576 11434576
```

In [85]:
```
for i, j in zip(l1[0], l2[0]):
        print(id(i), id(j))
```

```
11370064 11370064
11370096 11370096
```

In [86]:
```
t1 = (0, "1", "23")
t2 = t1[:]
print(t1 is t2)
```

```
True
```

In [87]:
```
a, b, c, d = "abcd"
print(a, b, c, d)
```

```
a b c d
```

In [88]:
```
s = "FfEeDdCcBbAa"
lowerLetters = s[::-2]
upperLetters = s[-2::-2]
print(lowerLetters)
```

```
abcdef
```

In [89]:
```
print(upperLetters)
```

```
ABCDEF
```

In [90]:
```
t1 = 1, 2, 3, 4, 5, 6
t2 = 7, 8, 9, 10
t3 = 11, 12, 13, 14, 15, 16, 17
t = t1 + t2 + t3
l = list(t)
l[::2] = [0] * len(l[::2])
print(tuple(l))
```

```
(0, 2, 0, 4, 0, 6, 0, 8, 0, 10, 0, 12, 0, 14, 0, 16, 0)
```

In [91]:
```
m = [[0, 0, 0], [0, 0, 0], [0, 0, 0]]
m[0][0] = 1
```

```
m[1][1] = 1
m[2][2] = 1
print(m)
```

```
[[1, 0, 0], [0, 1, 0], [0, 0, 1]]
```

In [92]:
```
m = [[0, 0, 0], [0, 0, 0], [0, 0, 0]]
m1 = copy.deepcopy(m)
m1[0][0] = 1
m1[1][1] = 1
m1[2][2] = 1
print(m1)
```

```
[[1, 0, 0], [0, 1, 0], [0, 0, 1]]
```

In [93]:
```
print(m)

data = [(100, "USD", "EUR", 0.83), (100, "USD", "CAD", 1.27), (100, "CAD",
"EUR", 0.65)]
USDAmountInEUR = data[0][0] * data[0][3]
print(str(data[0][0]) + " " + data[0][1] + " = " + str(USDAmountInEUR) + " " +
str(data[0][2]))
```

```
[[0, 0, 0], [0, 0, 0], [0, 0, 0]]
100 USD = 83.0 EUR
```

In [94]:
```
USDAmountInCAD = data[1][0] * data[1][3]
print(str(data[1][0]) + " " + data[1][1] + " = " + str(USDAmountInCAD) + " " +
str(data[1][2]))
```

```
100 USD = 127.0 CAD
```

In [95]:
```
CADAmountInEUR = data[2][0] * data[2][3]
print(str(data[2][0]) + " " + data[2][1] + " = " + str(CADAmountInEUR) + " " +
str(data[2][2]))
```

```
100 CAD = 65.0 EUR
```

In [96]:
```
print(ord("A"))
```

```
65
```

In [97]:
```
print("\N{SNAKE}")
```

```
🐍
```

In [98]:
```
print(ord("\N{SNAKE}"))
```

```
128013
```

In [99]:
```
print(hex(ord("\N{SNAKE}")))
```

```
0x1f40d
```

In [100]:
```
print("\u03b1")
```

α

In [101]:
```
print("\U0001f40d")
```

🐍

In [102]:
```
print(int("1f40d", 16))
```

128013

In [103]:
```
print("\N{SNAKE} is a snake")
```

🐍 is a snake

In [104]:
```
print(int("7ff", 16))
```

2047

In [105]:
```
print(int("ffff", 16))
```

65535

In [106]:
```
print(int("10ffff", 16))
```

1114111

In [107]:
```
print(bin(2047))
```

0b11111111111

In [108]:
```
print(len(bin(2047)))
```

13

In [109]:
```
print(bin(65535))
```

0b1111111111111111

In [110]:
```
print(len(bin(65535)))
```

18

In [111]:
```
print(bin(1114111))
```

0b100001111111111111111

In [112]:
```
print(len(bin(1114111)))
```

23

In [113]:
```
print(bin(128013))
```

0b11111010000001101

In [114]:
```
print("\n{snake}")
```

{snake}

In [115]:
```
a = True
b = True
print(a is b)
```

```
True
```

In [116]:
```
a = False
b = False
print(a is b)
```

```
True
```

In [117]:
```
a = True
print(id(a))
```

```
11120192
```

In [118]:
```
a = False
print(id(a))
```

```
11045056
```

In [119]:
```
b = False
print(id(b))
```

```
11045056
```

In [120]:
```
?
```

```
Docstring:
Return the lowest index in S where substring sub is found, such that sub is contained within
S[start:end].

Optional arguments start and end are interpreted as in slice notation.
Return -1 on failure.
Type:      method_descriptor
```

In [121]:
```
print(f"1 + 1 = {1 + 1}")
```

```
1 + 1 = 2
```

In [122]:
```
bid = 1.5760
ask = 1.5763
print("bid: {}, ask: {}, spread: {}".format(bid, ask, ask - bid))
```

```
bid: 1.576, ask: 1.5763, spread: 0.00029999999999996696
```

In [123]:
```
print("bid: {:.4f}, ask: {:.4f}, spread: {:.4f}".format(bid, ask, ask - bid))
```

```
bid: 1.5760, ask: 1.5763, spread: 0.0003
```

In [124]:
```
print(f"bid: {bid:.4f}, ask: {ask:.4f}, spread: {ask - bid:.4f}")
```

```
bid: 1.5760, ask: 1.5763, spread: 0.0003
```

In [125]:
```
print(f"bid: {bid:.4f}, ask: {ask:.4f}, spread: {(ask - bid):.4f}")
```

```
bid: 1.5760, ask: 1.5763, spread: 0.0003
```

In [126]:
```
string1 = "ß, ß, ♯"
string1 = "Π, ύ, θ, ω, ν"
chars = string1.split(", ")
print(chars)
```

```
['Π', 'ύ', 'θ', 'ω', 'ν']
```

In [127]:
```
list1 = []
for c in chars:
        list1.append(hex(ord(c)))
print(list1)
```

```
['0x3a0', '0x3cd', '0x3b8', '0x3c9', '0x3bd']
```

In [128]:
```
list1 = []
for c in chars:
        list1.append(c.upper())
print(list1)
```

```
['Π', 'Υ', 'Θ', 'Ω', 'Ν']
```

In [129]:
```
list1 = []
for c in chars:
        list1.append(c.lower())
print(list1)
```

```
['π', 'ύ', 'θ', 'ω', 'ν']
```

In [130]:
```
a = 42
print(f"The number {a} is not even") if a % 2 else print(f"The number {a} is
even")
```

```
The number 42 is even
```

In [131]:
```
a = 41
print(f"The number {a} is not even") if a % 2 else print(f"The number {a} is
even")
```

```
The number 41 is not even
```

In [132]:
```
a = 42
print("The number {} is not even".format(a)) if a % 2 else print("The number {}
is even".format(a))
```

```
The number 42 is even
```

In [133]:
```
a = 41
print("The number {} is not even".format(a)) if a % 2 else print("The number {}
is even".format(a))
```

```
The number 41 is not even
```

In [134]:
```
a = 0.3
b = 0.1
```

```
3 print(f"{a} / {b} = {a / b:.4f}")
```

```
0.3 / 0.1 = 3.0000
```

In [135]:
```
1 a = 42
2 qualifier = "is not" if a % 2 else "is"
3 print(f"The number {a} {qualifier} even")
```

```
The number 42 is even
```

In [136]:
```
1 a = 421
2 qualifier = "is not" if a % 2 else "is"
3 print(f"The number {a} {qualifier} even")
```

```
The number 421 is not even
```

In [137]:
```
1 print(range(10))
```

```
range(0, 10)
```

In [138]:
```
1 print(type(range(10)))
```

```
<class 'range'>
```

In [139]:
```
1 for i in range(3):
2         print(i)
3 print(i)
```

```
0
1
2
2
```

In [140]:
```
1 for i in "abc":
2         pass
3 print(i)
```

```
c
```

In [141]:
```
1 raggedArray = [[1, 2], [3, 4, 5, 6], [7, 8, 9], [10]]
2 for r in range(len(raggedArray)):
3         for c in range(len(raggedArray[r])):
4                 print(f"[{r}, {c}] = {raggedArray[r][c]}")
5         print("-" * 5)
```

```
[0, 0] = 1
[0, 1] = 2
-----
[1, 0] = 3
[1, 1] = 4
[1, 2] = 5
[1, 3] = 6
-----
[2, 0] = 7
[2, 1] = 8
[2, 2] = 9
-----
[3, 0] = 10
-----
```

In [142]:

```
identityMatrix = []
size = 10
for r in range(size):
        row = []
        for c in range(size):
                if r == c:
                        row.append(1)
                else:
                        row.append(0)
        identityMatrix.append(row)
identityMatrix
```

Out[142]:

```
[[1, 0, 0, 0, 0, 0, 0, 0, 0, 0],
 [0, 1, 0, 0, 0, 0, 0, 0, 0, 0],
 [0, 0, 1, 0, 0, 0, 0, 0, 0, 0],
 [0, 0, 0, 1, 0, 0, 0, 0, 0, 0],
 [0, 0, 0, 0, 1, 0, 0, 0, 0, 0],
 [0, 0, 0, 0, 0, 1, 0, 0, 0, 0],
 [0, 0, 0, 0, 0, 0, 1, 0, 0, 0],
 [0, 0, 0, 0, 0, 0, 0, 1, 0, 0],
 [0, 0, 0, 0, 0, 0, 0, 0, 1, 0],
 [0, 0, 0, 0, 0, 0, 0, 0, 0, 1]]
```

In [143]:

```
raggedArray = [[1, 2], [3, 4, 5, 6], [7, 8, 9], [10]]
for i, r in enumerate(raggedArray):
        for j, e in enumerate(raggedArray[i]):  # e for element, cuz it's not a
column technically
                print(f"[{i}, {j}] = {raggedArray[i][j]}")
        print("-" * 8)
```

```
[0, 0] = 1
[0, 1] = 2
--------
[1, 0] = 3
[1, 1] = 4
[1, 2] = 5
[1, 3] = 6
--------
[2, 0] = 7
[2, 1] = 8
[2, 2] = 9
--------
[3, 0] = 10
--------
```

In [144]:
```
print(f"{0.1:.4f}")
```

```
0.1000
```

In [145]:
```
print("{:.4f}".format(0.1))
```

```
0.1000
```

In [146]:
```
total, count = (0,) * 2
data = [10.5, 11.2, 9.8, None, 11.5, None]
for e in data:
        if e:
                total += e
                count += 1
average = total / count
print(average)
```

```
10.75
```

In [147]:
```
for i, e in enumerate(data):
        if not e:
                data[i] = average
print(data)
```

```
[10.5, 11.2, 9.8, 10.75, 11.5, 10.75]
```

In [148]:
```
data = [10.5, 11.2, 9.8, None, 11.5, None]
count = sum(1 for e in data if e)
total = sum(e for e in data if e)
average = total / count
print(sum, average)
```

```
<built-in function sum> 10.75
```

In [149]:
```
data = [10.5, 11.2, 9.8, None, 11.5, None]
average = statistics.fmean(e for e in data if e)
data = [average if e is None else e for e in data]
print(data)
```

```
[10.5, 11.2, 9.8, 10.75, 11.5, 10.75]
```

In [150]:
```
data = [10.5, 11.2, 9.8, None, 11.5, None]
df = pandas.DataFrame(data)
df.fillna(df.mean(), inplace = True)
print(df[0].tolist())
```

```
[10.5, 11.2, 9.8, 10.75, 11.5, 10.75]
```

In [151]:
```
print(len(str((1, 2))))
```

```
6
```

In [152]:
```
print(str((1, 2)))
```

```
(1, 2)
```

In [153]:
```
m, n = 4, 5
for i in range(1, m):
        for j in range(1, n):
                print(f"{i} x {j} = {i * j}")
        print("-" * 8)
```

```
1 x 1 = 1
1 x 2 = 2
1 x 3 = 3
1 x 4 = 4
--------
2 x 1 = 2
2 x 2 = 4
2 x 3 = 6
2 x 4 = 8
--------
3 x 1 = 3
3 x 2 = 6
3 x 3 = 9
3 x 4 = 12
--------
```

In [154]:
```
data = (["2021-01-01", 10, 20], ["2021-01-02", 20, 18], ["2021-01-03", -10, 10], ["2021-01-04", 100, 102], ["2021-01-05", 20, 45])
largest = 0
for e in data:
        distance = abs(e[2] - e[1])
        e.append(distance)
        if distance > largest:
                dateOfLargest = e[0]
                largest = distance
print(data)
print(dateOfLargest)
```

```
1
0
```

```
(['2021-01-01', 10, 20, 10], ['2021-01-02', 20, 18, 2], ['2021-01-03', -10, 10, 20], ['2021-01-04',
100, 102, 2], ['2021-01-05', 20, 45, 25])
2021-01-05
```

In [155]:

```
data = [[10, 20], [20, 30], [35, 50], [45, 60]]
for i, r in enumerate(data):
        if i == 0:
                r.append("")
        else:
                if r[0] < data[i - 1][1]:
                        r.append("down")
                elif r[0] > data[i - 1][1]:
                        r.append("up")
                else:
                        r.append("same")
print(data)
```

```
[[10, 20, ''], [20, 30, 'same'], [35, 50, 'up'], [45, 60, 'down']]
```

In [156]:

```
tuple1 = ([0],)
print(type(tuple1))
print(tuple1)
print(id(tuple1))
print(id(tuple1[0]))
tuple1[0][0] += 1
print(tuple1)
print(id(tuple1))
print(id(tuple1[0]))
```

```
<class 'tuple'>
([0],)
128573173295168
128573172611328
([1],)
128573173295168
128573172611328
```

In [157]:

```
print(hash(True))
a = True
print(hash(a))
b = True
print(hash(b))
a = b
print(hash(a))
```

```
1
1
1
1
```

In [158]:

```
print(hash(257))
a = 257
```

```
print(hash(a))
b = 257
print(hash(b))
a = b
print(hash(a))
```

```
257
257
257
257
```

In [159]:

```
print(hash(2570))
a = 2570
print(hash(a))
b = 2570
print(hash(b))
a = b
print(hash(a))
```

```
2570
2570
2570
2570
```

In [160]:

```
print(hash(25))
a = 25
print(hash(a))
b = 25
print(hash(b))
a = b
print(hash(a))
```

```
25
25
25
25
```

In [161]:

```
print(hash(2.5))
a = 2.5
print(hash(a))
b = 2.5
print(hash(b))
print(hex(hash(b)))
a = b
print(len(hex(hash(a))))
print(hash(b))
```

```
1152921504606846978
1152921504606846978
1152921504606846978
0x1000000000000002
18
1152921504606846978
```

In [162]:

```
for i in range(5):
        for j in range(4):
```

```
                pass
print(i, j)
```

```
4 3
```

In [163]:

```


s1 = """
"'And' and 'or' are the basic operations of logic. Together with 'no' (the logical operation
of negation) they are a complete set of basic logical operations – all other logical
operations, no matter how complex, can be obtained by suitable combinations of these."
— John von Neumann, The Computer and the Brain
"""

s2 = """
Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut
labore et dolore magna aliqua. Ut enim ad minim veniam, quis nostrud exercitation ullamco laboris
nisi ut aliquip ex ea commodo consequat. Duis aute irure dolor in reprehenderit in voluptate velit
esse cillum dolore eu fugiat nulla pariatur. Excepteur sint occaecat cupidatat non proident, sunt
in culpa qui officia deserunt mollit anim id est laborum.
"""

charCounts = {}
for c in s1:
        if (ord(c) >= ord("A") and ord(c) <= ord("Z")) or (ord(c) >= ord("a") and ord(c) <= ord("z")):
                charCounts[c] = charCounts.get(c, 0) + 1
print(charCounts)
```

```
{'A': 1, 'n': 20, 'd': 4, 'a': 23, 'o': 31, 'r': 12, 'e': 27, 't': 21, 'h': 11, 'b': 7, 's': 9, 'i': 17, 'c': 10, 'p': 7, 'f': 4, 'l': 12, 'g': 6, 'T': 2, 'w': 2, 'y': 2, 'm': 6, 'x': 1, 'u': 3, 'J': 1, 'v': 1, 'N': 1, 'C': 1, 'B': 1}
```

In [164]:

```
d1 = {"a": 10, "b": 20, "c": 30}
d2 = {"d": 100, "e": 200, "f": 300}
d3 = {"f": 30, "g": 40}
```

```
keysList, valuesList = [], []
for d in d1, d2, d3:
        for k, v in d.items():
                keysList.append(k)
                valuesList.append(v)
print(keysList)
print(valuesList)
```

```
['a', 'b', 'c', 'd', 'e', 'f', 'f', 'g']
[10, 20, 30, 100, 200, 300, 30, 40]
```

In [165]:

```
keysList, valuesList = [], []
keysList = list(d1.keys())
keysList.extend(list(d2.keys()))
keysList.extend(list(d3.keys()))
print(keysList)
```

```
['a', 'b', 'c', 'd', 'e', 'f', 'f', 'g']
```

In [166]:

```
valuesList = list(d1.values())
valuesList.extend(list(d2.values()))
valuesList.extend(list(d3.values()))
print(valuesList)
```

```
[10, 20, 30, 100, 200, 300, 30, 40]
```

In [167]:

```
print(list(d1.keys()) + list(d2.keys()) + list(d3.keys()))
print(list(d1.values()) + list(d2.values()) + list(d3.values()))
```

```
['a', 'b', 'c', 'd', 'e', 'f', 'f', 'g']
[10, 20, 30, 100, 200, 300, 30, 40]
```

In [168]:

```
grades = {"John": [90, 95, 98], "Eric": [86, 84, 92], "Michael": [90, 89, 85]}
exam = {"Eric": 99, "John": 100}
for k, v in grades.items():
        if k in exam:
                v.insert(0, exam[k])
        else:
                v.insert(0, None)
print(grades)
```

```
{'John': [100, 90, 95, 98], 'Eric': [99, 86, 84, 92], 'Michael': [None, 90, 89, 85]}
```

In [169]:

```
for k, v in grades.items():
        score = exam.get(k)
        grades[k] = [score] + grades[k]
print(grades)
```

```
{'John': [100, 100, 90, 95, 98], 'Eric': [99, 99, 86, 84, 92], 'Michael': [None, None, 90, 89, 85]}
```

In [170]:

```
for k, v in grades.items():
        v.insert(0, exam.get(k))
print(grades)
```

```
{'John': [100, 100, 100, 90, 95, 98], 'Eric': [99, 99, 99, 86, 84, 92], 'Michael': [None, None,
None, 90, 89, 85]}
```

In [171]:
```
for k in grades:
        print(k)
```

```
John
Eric
Michael
```

In [172]:
```
l = ["AAPL", "AAPL", "Aapl", "aapl", "MSFT"]
print(list(set(l)))
```

```
['AAPL', 'Aapl', 'MSFT', 'aapl']
```

In [173]:
```
l = [e.casefold() for e in l]
print(list(set(l)))
```

```
['aapl', 'msft']
```

In [174]:
```
se = {e.casefold() for e in l}
print(se)
```

```
{'aapl', 'msft'}
```

In [175]:
```
data = {"d1": {"a": 1, "b": 2, "c": 3}, "d2": {"b": 20, "c": 30, "d": 40},
"d3": {"d": 100, "x": 200}}
uniqueKeys = {k for v in (data["d1"], data["d2"], data["d3"]) for k in v}
print(uniqueKeys)
```

```
{'d', 'x', 'c', 'b', 'a'}
```

In [176]:
```
try:
        s1 = {[0, 1]}
        print(s1)
except TypeError as ex:
        print(type(ex).         , ": ", *ex.args, ".", sep = "")
```

```
TypeError: cannot use 'list' as a set element (unhashable type: 'list').
```

In [177]:
```
m = [0] * 3
print(m)
print(m[0] is m[1])
```

```
[0, 0, 0]
True
```

In [178]:
```
m = [[0] * 3 for _ in range(3)]
print(m)
print(m[0] is m[1])
print(m[0][0] is m[0][1])
```

```
[[0, 0, 0], [0, 0, 0], [0, 0, 0]]
False
True
```

In [179]:
```
m = [[10000 for i in range(3)] for _ in range(4)]
print(m)
```

```
[[10000, 10000, 10000], [10000, 10000, 10000], [10000, 10000, 10000], [10000, 10000, 10000]]
```

In [180]:
```
pprint.pprint(m, width = 1)
```

```
[[10000,
  10000,
  10000],
 [10000,
  10000,
  10000],
 [10000,
  10000,
  10000],
 [10000,
  10000,
  10000]]
```

In [181]:
```
print(m[0] is m[1])
print(m[0][0] is m[0][1])
print(hash(m[0][0]), hash(m[0][1]))
```

```
False
True
10000 10000
```

In [182]:
```
print(id(m[0][0]), id(m[0][1]))
```

```
128573172653872 128573172653872
```

In [183]:
```
m[0][0] = 1001
pprint.pprint(m)
```

```
[[1001, 10000, 10000],
 [10000, 10000, 10000],
 [10000, 10000, 10000],
 [10000, 10000, 10000]]
```

In [184]:
```
print(id(m[0][0]), id(m[0][1]))
print(m[0][0] is m[0][1])
```

```
128573172653712 128573172653872
False
```

In [185]:
```
data = [{"open": 100, "high": 120, "low": 90, "close": 110}, {"open": 110,
"high": 130, "low": 80, "close": 120}, {"open": 120, "high": 140, "low": 70,
"close": 130}, {"open": 130, "high": 150, "low": 60, "close": 140},]
ranges = []
for d in data:
        ranges.append(d["high"] - d["low"])
print(ranges)
```

```
[30, 50, 70, 90]
```

In [186]:

```
data = [{"open": 100, "high": 120, "low": 90, "close": 110}, {"open": 110,
"high": 130, "low": 80, "close": 120}, {"open": 120, "high": 140, "low": 70,
"close": 130}, {"open": 130, "high": 150, "low": 60, "close": 140},]
ranges = [d["high"] - d["low"] for d in data]
print(ranges)
```

```
[30, 50, 70, 90]
```

In [187]:

```
bogusComposites = {i for i in range(1, 101) for j in range(2, 10) if i % j ==
0}
allNumbers = {i for i in range(11, 101)} | {1}
print(allNumbers - bogusComposites)
```

```
{1, 11, 13, 17, 19, 23, 29, 31, 37, 41, 43, 47, 53, 59, 61, 67, 71, 73, 79, 83, 89, 97}
```

In [188]:

```
allNumbers = set(range(1, 101))
print(allNumbers - bogusComposites)
```

```
{1, 11, 13, 17, 19, 23, 29, 31, 37, 41, 43, 47, 53, 59, 61, 67, 71, 73, 79, 83, 89, 97}
```

In [189]:

```
data = [{"symbol": "ABCD", "name": "ABCD Company", "ranking": 2, "risk": 0.2},
{"symbol": "BCDE", "name": "BCDE Company", "ranking": 5, "risk": 0.2},
{"symbol": "CDEF", "name": "CDEF Company", "ranking": 8, "risk": 0.5},
{"symbol": "DEFG", "name": "DEFG Company", "ranking": 7, "risk": 0.8},
{"symbol": "EFGH", "name": "EFGH Company", "ranking": 9, "risk": 0.6},
{"symbol": "FGHI", "name": "FGHI Company", "ranking": 10, "risk": 0.1},
{"symbol": "GHIJ", "name": "GHIJ Company", "ranking": 3, "risk": 0.6},
{"symbol": "HIJK", "name": "HIJK Company", "ranking": 5, "risk": 0.5},
{"symbol": "IJKL", "name": "IJKL Company", "ranking": 5, "risk": 0.7},
{"symbol": "JKLM", "name": "JKLM Company", "ranking": 6, "risk": 0.9},
{"symbol": "KLMN", "name": "KLMN Company", "ranking": 6, "risk": 0.4},
{"symbol": "LMNO", "name": "LMNO Company", "ranking": 8, "risk": 0.4},
{"symbol": "MNOP", "name": "MNOP Company", "ranking": 8, "risk": 0.2},
{"symbol": "NOPQ", "name": "NOPQ Company", "ranking": 1, "risk": 0.5},
{"symbol": "OPQR", "name": "OPQR Company", "ranking": 9, "risk": 0.2},
{"symbol": "PQRS", "name": "PQRS Company", "ranking": 10, "risk": 0.9},
{"symbol": "QRST", "name": "QRST Company", "ranking": 3, "risk": 0.4},
{"symbol": "RSTU", "name": "RSTU Company", "ranking": 7, "risk": 0.3},
{"symbol": "STUV", "name": "STUV Company", "ranking": 8, "risk": 0.1},
{"symbol": "TUVW", "name": "TUVW Company", "ranking": 7, "risk": 0.9}]
filtered = [d for d in data if d["ranking"] > 5 and d["risk"] < 0.6]
print(filtered)
```

```
[{'symbol': 'CDEF', 'name': 'CDEF Company', 'ranking': 8, 'risk': 0.5}, {'symbol': 'FGHI', 'name':
'FGHI Company', 'ranking': 10, 'risk': 0.1}, {'symbol': 'KLMN', 'name': 'KLMN Company', 'ranking':
6, 'risk': 0.4}, {'symbol': 'LMNO', 'name': 'LMNO Company', 'ranking': 8, 'risk': 0.4}, {'symbol':
'MNOP', 'name': 'MNOP Company', 'ranking': 8, 'risk': 0.2}, {'symbol': 'OPQR', 'name': 'OPQR
Company', 'ranking': 9, 'risk': 0.2}, {'symbol': 'RSTU', 'name': 'RSTU Company', 'ranking': 7,
'risk': 0.3}, {'symbol': 'STUV', 'name': 'STUV Company', 'ranking': 8, 'risk': 0.1}]
```

In [190]:

```
for d in filtered:
    d["weight"] = float(f"{d["ranking"] * d["risk"]:.2f}")
print(filtered)
```

```
[{'symbol': 'CDEF', 'name': 'CDEF Company', 'ranking': 8, 'risk': 0.5, 'weight': 4.0}, {'symbol': 'FGHI', 'name': 'FGHI Company', 'ranking': 10, 'risk': 0.1, 'weight': 1.0}, {'symbol': 'KLMN', 'name': 'KLMN Company', 'ranking': 6, 'risk': 0.4, 'weight': 2.4}, {'symbol': 'LMNO', 'name': 'LMNO Company', 'ranking': 8, 'risk': 0.4, 'weight': 3.2}, {'symbol': 'MNOP', 'name': 'MNOP Company', 'ranking': 8, 'risk': 0.2, 'weight': 1.6}, {'symbol': 'OPQR', 'name': 'OPQR Company', 'ranking': 9, 'risk': 0.2, 'weight': 1.8}, {'symbol': 'RSTU', 'name': 'RSTU Company', 'ranking': 7, 'risk': 0.3, 'weight': 2.1}, {'symbol': 'STUV', 'name': 'STUV Company', 'ranking': 8, 'risk': 0.1, 'weight': 0.8}]
```

In [191]:

```
weightedStocks = [dict(symbol = d["symbol"], weight = d["weight"]) for d in filtered]
print(weightedStocks)
```

```
[{'symbol': 'CDEF', 'weight': 4.0}, {'symbol': 'FGHI', 'weight': 1.0}, {'symbol': 'KLMN', 'weight': 2.4}, {'symbol': 'LMNO', 'weight': 3.2}, {'symbol': 'MNOP', 'weight': 1.6}, {'symbol': 'OPQR', 'weight': 1.8}, {'symbol': 'RSTU', 'weight': 2.1}, {'symbol': 'STUV', 'weight': 0.8}]
```

In [192]:

```
weightedStocks = [dict(symbol = d["symbol"], weight = round(d["ranking"] * d["risk"], 2)) for d in data if d["ranking"] >= 5 and d["risk"] < 0.6]
print(weightedStocks)
```

```
[{'symbol': 'BCDE', 'weight': 1.0}, {'symbol': 'CDEF', 'weight': 4.0}, {'symbol': 'FGHI', 'weight': 1.0}, {'symbol': 'HIJK', 'weight': 2.5}, {'symbol': 'KLMN', 'weight': 2.4}, {'symbol': 'LMNO', 'weight': 3.2}, {'symbol': 'MNOP', 'weight': 1.6}, {'symbol': 'OPQR', 'weight': 1.8}, {'symbol': 'RSTU', 'weight': 2.1}, {'symbol': 'STUV', 'weight': 0.8}]
```

In [193]:

```
weightedStocks = [{"symbol": d["symbol"], "weight": round(d["ranking"] * d["risk"], 2)} for d in data if d["ranking"] >= 5 and d["risk"] < 0.6]
print(weightedStocks)
```

```
[{'symbol': 'BCDE', 'weight': 1.0}, {'symbol': 'CDEF', 'weight': 4.0}, {'symbol': 'FGHI', 'weight': 1.0}, {'symbol': 'HIJK', 'weight': 2.5}, {'symbol': 'KLMN', 'weight': 2.4}, {'symbol': 'LMNO', 'weight': 3.2}, {'symbol': 'MNOP', 'weight': 1.6}, {'symbol': 'OPQR', 'weight': 1.8}, {'symbol': 'RSTU', 'weight': 2.1}, {'symbol': 'STUV', 'weight': 0.8}]
```

In [194]:

```
m = [[0] * 2] * 3
print(m)
for r in range(len(m)):
        for c in range(len(m[r])):
                print(id(m[r][c]))
```

```
[[0, 0], [0, 0], [0, 0]]
11370064
11370064
11370064
11370064
11370064
11370064
```

In [195]:

```
m = [[{0}] * 2] * 3
print(m)
for r in range(len(m)):
        for c in range(len(m[r])):
                print(id(m[r][c]))
```

```
[[{0}, {0}], [{0}, {0}], [{0}, {0}]]
128573172485248
128573172485248
128573172485248
128573172485248
128573172485248
128573172485248
```

In [196]:

```
m[0][0] = 1
for r in range(len(m)):
        for c in range(len(m[r])):
                print(id(m[r][c]))
print(m)
```

```
11370096
128573172485248
11370096
128573172485248
11370096
128573172485248
[[1, {0}], [1, {0}], [1, {0}]]
```

In [197]:

```
m = [[{0} for _ in range(2)] for _ in range(3)]
print(m)
for r in range(len(m)):
        for c in range(len(m[r])):
                print(id(m[r][c]))
m[0][0] = 1
print(m)
```

```
[[{0}, {0}], [{0}, {0}], [{0}, {0}]]
128573172485696
128573172482336
128573172482112
128573172486144
128573172487040
128573172485024
[[1, {0}], [{0}, {0}], [{0}, {0}]]
```

In [198]:

```
m = [[{0} for _ in range(2)] for _ in range(3)]
print(m)
for r in range(len(m)):
        for c in range(len(m[r])):
                print(id(m[r][c]))
m[0][0] = {1}
print(m)
```

```
[[{0}, {0}], [{0}, {0}], [{0}, {0}]]
128573172485696
128573172483680
128573172487488
128573172486816
128573172486592
128573172488160
[[{1}, {0}], [{0}, {0}], [{0}, {0}]]
```

In [199]:
```
m = [[0 for _ in range(2)] for _ in range(3)]
for r in range(len(m)):
        for c in range(len(m[r])):
                print(id(m[r][c]))
```

```
11370064
11370064
11370064
11370064
11370064
11370064
```

In [200]:
```
m = [[10000 for _ in range(2)] for _ in range(3)]
for r in range(len(m)):
        for c in range(len(m[r])):
                print(id(m[r][c]))
m[0][0] = 1
print(m)
```

```
128573172653264
128573172653264
128573172653264
128573172653264
128573172653264
128573172653264
[[1, 10000], [10000, 10000], [10000, 10000]]
```

In [201]:
```
m = [[{0} for _ in range(2)] for _ in range(3)]
for r in range(len(m)):
        for c in range(len(m[r])):
                print(id(m[r][c]))
```

```
128573172483680
128573172487488
128573172485248
128573172486144
128573172488160
128573172486368
```

In [202]:
```
# print(input("Text box displayed ?"))
```

In [203]:
```
a = 0
b = complex(real = 3.0, imag = 4)
print(repr(a))
print(repr(b))
print(b)
```

```
0
(3+4j)
(3+4j)
```

In [204]:
```
values = [4, 3, 1, 2]
try:
        minimum = abs(values[0])
        for value in values[1:]:
```

```
                if abs(value) < minimum:
                        minimum = abs(value)
except IndexError:
        minimum = 0
        pass
finally:
        print(f"Minimum is: {minimum}")
```

```
Minimum is: 1
```

In [205]:

```
values = []

try:
        minimum = abs(values[0])
        for value in values[1:]:
                if abs(value) < minimum:
                        minimum = abs(value)
except IndexError:
        minimum = 0
        pass
finally:
        print(f"Minimum is: {minimum}")
```

```
Minimum is: 0
```

In [206]:

```
try:
        raise ValueError("Custom message")
except ValueError as ex:
        print(type(ex).          + ": " + ex.args[0] + ".")
```

```
ValueError: Custom message.
```

In [ ]:

```
l = [4, 3, 2, 1, 0]
itr = iter(l)
try:
        while True:
                print(next(itr))
except StopIteration as ex:
        print(type(ex).          , ": ", *ex.args, ".", sep = "")
```

```
4
3
2
1
0
StopIteration: .
```

In [208]:

```
try:
        raise IndexError
except IndexError as ex:
        print(type(ex).          , ": ", *ex.args, ".", sep = "")
```

```
IndexError: .
```

In [209]:

```
rangeObject = range(10)
rangeIterator = iter(rangeObject)
print(next(rangeIterator))
print(next(rangeIterator))
print(next(rangeIterator))
print(list(rangeIterator))
```

```
0
1
2
[3, 4, 5, 6, 7, 8, 9]
```

In [210]:

```
rangeObj = range(5)
print(list(rangeObj))
print(list(rangeObj))
```

```
[0, 1, 2, 3, 4]
[0, 1, 2, 3, 4]
```

In [211]:

```
enumIterator = enumerate(string.ascii_lowercase)
print(enumIterator)
l = list(enumIterator)
print(l)
l = list(enumIterator)
print(dir(type(iter([0]))))
print(l)
try:
        print(next(enumIterator))
except StopIteration as ex:
        print(type(ex).        , ": ", *ex.args, ".", sep = "")
```

```
<enumerate object at 0x74efc6458680>
[(0, 'a'), (1, 'b'), (2, 'c'), (3, 'd'), (4, 'e'), (5, 'f'), (6, 'g'), (7, 'h'), (8, 'i'), (9, 'j'),
(10, 'k'), (11, 'l'), (12, 'm'), (13, 'n'), (14, 'o'), (15, 'p'), (16, 'q'), (17, 'r'), (18, 's'),
(19, 't'), (20, 'u'), (21, 'v'), (22, 'w'), (23, 'x'), (24, 'y'), (25, 'z')]
['__class__', '__delattr__', '__dir__', '__doc__', '__eq__', '__format__', '__ge__',
'__getattribute__', '__getstate__', '__gt__', '__hash__', '__init__', '__init_subclass__',
'__iter__', '__le__', '__length_hint__', '__lt__', '__ne__', '__new__', '__next__', '__reduce__',
'__reduce_ex__', '__repr__', '__setattr__', '__setstate__', '__sizeof__', '__str__',
'__subclasshook__']
[]
StopIteration: .
```

In [212]:

```
print(type(enumIterator))
print(type(iter(enumIterator)))
print(issubclass(type(iter([0])), type(enumIterator)))
```

```
<class 'enumerate'>
<class 'enumerate'>
False
```

In [213]:

```
print(iter(enumIterator) is enumIterator)
```

```
True
```

In [214]:
```
print(type(iter({0, 1, 2})))
```

```
<class 'set_iterator'>
```

In [215]:
```
print(type(iter({0: "0", 1: "1", "2": 2})))
```

```
<class 'dict_keyiterator'>
```

In [ ]:
```
l = [4, 3, 2, 1, 0]
itr = iter(l)
for i in itr:
        print(i)
```

```
4
3
2
1
0
```

In [217]:
```
try:
        print(next(l))
except TypeError as ex:
        print(type(ex).        , ": ", *ex.args, ".", sep = "")
```

```
TypeError: 'list' object is not an iterator.
```

In [218]:
```
start = time.perf_counter()
squaresGenerator = (i ** 10 for i in range(1_000_000))
end = time.perf_counter()
print(f"Elapsed: {end - start}")
```

```
Elapsed: 0.00016390199999705146
```

In [219]:
```
start = time.perf_counter()
squaresListComp = [i ** 10 for i in range(1_000_000)]
end = time.perf_counter()
print(f"Elapsed: {end - start}")
```

```
Elapsed: 0.23438802800001213
```

In [220]:
```
print((7.16e-5 / 1.824e-1) ** -1)
print(105 / 9)
print(10.5e1 / 9e0)
print(99 / 1)
print(9.9e1 / 1e0)  # 1 order of magnitude difference means a multiple of
between 10 and 100
```

```
2547.486033519553
11.666666666666666
11.666666666666666
99.0
99.0
```

In [221]:
```
print(999_999 ** 10)
```

```
999990000044999880000209999748000209999880000044999990000001
```

In [222]:
```
print(squaresGenerator is iter(squaresGenerator))
```

```
True
```

In [223]:
```
l = [10, "abc", 3.14, True]
for i, e in enumerate(l):
        print(f"{i}: {e}")
```

```
0: 10
1: abc
2: 3.14
3: True
```

In [224]:
```
def f(a, *b, x, **y):
        print("Positional/Named:", a)
        print("Variable positional:", b)
        print("Named only:", x)
        print("Variable named:", y)


f(0, 1, 2, x = 3, y = 4, z = 5)
```

```
Positional/Named: 0
Variable positional: (1, 2)
Named only: 3
Variable named: {'y': 4, 'z': 5}
```

In [225]:
```
# f(10, 2, x = 10, 20)
```

In [226]:
```
f(10, 2, x = 10, y = 20)
```

```
Positional/Named: 10
Variable positional: (2,)
Named only: 10
Variable named: {'y': 20}
```

In [227]:
```
f(10, z = 2, x = 10, y = 20)
```

```
Positional/Named: 10
Variable positional: ()
Named only: 10
Variable named: {'z': 2, 'y': 20}
```

In [228]:
```
f(10, x = 2, y = 10, z = 20)
```

```
Positional/Named: 10
Variable positional: ()
Named only: 2
Variable named: {'y': 10, 'z': 20}
```

In [229]:
```
# f(a = 10, 2, x = 10, y = 20)
```

In [230]:
```
f(x = 10, y = 20, z = 30, a = 10)
```

```
Positional/Named: 10
Variable positional: ()
Named only: 10
Variable named: {'y': 20, 'z': 30}
```

In [231]:
```
try:
        f(10, a = 2, x = 10, y = 20)
except TypeError as ex:
        print(type(ex).        )
```

```
TypeError
```

In [232]:
```
def f1(a, *, x, **y):
        print(f"a: {a}, x: {x}, y: {y}")
```

In [233]:
```
try:
        f1(10, a = 20, x = 2, y = 10)
except TypeError as ex:
        print(type(ex).        )
```

```
TypeError
```

In [234]:
```
f1(a = 1, x = 2)
```

```
a: 1, x: 2, y: {}
```

In [235]:
```
print(max("cba"))
```

```
c
```

In [236]:
```
def f2(a, b):
        print(f"a: {a}; b: {b}")


f2(2, 3)
```

```
a: 2; b: 3
```

In [237]:
```
f2(2, b = 3)
```

```
a: 2; b: 3
```

In [238]:
```
# f2(a = 2, 3)
```

In [239]:
```
f2(b = 1, a = 0)
```

```
a: 0; b: 1
```

In [240]:
```
# def f3(a = 0, b):
#       print(f"a: {a}; b: {b}")
```

In [241]:
```
def f3(a, b = 0):
        print(f"a: {a}; b: {b}")

```

```
4
5 f3(-1, 1)
```

```
a: -1; b: 1
```

In [242]:
```
1 # f3(a = -1, 1)
```

In [243]:
```
1 f3(a = -1, b = 1)
```

```
a: -1; b: 1
```

In [244]:
```
1 def f4(a, b = 1, *, x):
2         print(a, b, x)
3
4
5 f4(0, 2, x = 3)
```

```
0 2 3
```

In [245]:
```
1 def f5(a, b = 1, *, x = 2, y):
2         print(a, b, x, y)
3
4
5 f5(0, 2, x = 3, y = 4)
```

```
0 2 3 4
```

In [246]:
```
1 def f6(**x):
2         print(x)
3
4
5 f6(a = 1, b = 2, c = 3)
```

```
{'a': 1, 'b': 2, 'c': 3}
```

In [247]:
```
1 # def f7(*, **x):
2 #       print(x)
3 #
4 #
5 # f7(a = 1, b = 2)
```

In [248]:
```
1 def f8(a, *, b):
2         print(f"a: {a}; b: {b}")
3
4
5 f8(b = 1, a = 0)
```

```
a: 0; b: 1
```

In [249]:
```
def toJSON(a, *, b, **c):
        print(a, b, c)


toJSON(0, b = 3, c1 = 0, c2 = 1)
```

```
0 3 {'c1': 0, 'c2': 1}
```

In [250]:
```
def avg(a, *b):
        return (sum(b) + a) / (len(b) + 1)


print(avg(0, 1, 2, 3, 4))
print(avg(0))
print(avg(0, 1))
```

```
2.0
0.0
0.5
```

In [251]:
```
def separator(b = 10, *, a = "-"):
        return a * b


print(separator())
print(separator(a = "A"))
print(separator(5))
print(separator(a = "B", b = 5))
print(separator(5, a = "B"))
```

```
----------
AAAAAAAAAA
-----
BBBBB
BBBBB
```

In [252]:
```
print(sep = "|", *("AB", "CD"))
```

```
AB|CD
```

In [253]:
```
# def f(a, *b, c):
#       print(a, b, c)
#
#
# f(c = 2, 0, 1, 2)
```

In [254]:
```
# def f(*a, b = 0):
#       print(a, b)
#
#
# f(b = 1, 0, 2)
```

In [255]:
```
try:
        f(b = 1, *(0, 2))
except TypeError as ex:
        print(type(ex).          )
```

```
TypeError
```

In [256]:
```
# def f(a, *, b):
#       print(a, b)
#
#
# f(b = 1, 0)
```

In [257]:
```
# def f(a, b):
#       print(a, b)
#
#
# f(b = 1, 0)
```

In [258]:
```
# def f(a, b = 2):
#       print(a, b)
#
#
# f(b = 1, 0)
```

In [259]:
```
def f(a, b = 2):
        print(a, b)


f(b = 1, *(0,))
```

```
0 1
```

In [260]:
```
anonymous = lambda iterable: len(set(iterable))
print(anonymous("abcd"))
print(anonymous("abab"))
try:
        print(anonymous())
except TypeError as ex:
        print(type(ex).          )
print(anonymous(""))
print(anonymous([0, 1, 2]))
print(anonymous([0]))
print(anonymous([]))
print(anonymous([1, 1, 2, 2, 3, 4]))
```

```
4
2
TypeError
0
3
1
0
4
```

In [261]:

```
s = "This is the first sentence. This is the scecond sentence. This is not the fourth sentence, it is the third sentence."


def f(s = ""):
        return dict(collections.Counter(s))


print(f(s))
print(f(""))
```

```
{'T': 3, 'h': 9, 'i': 10, 's': 13, ' ': 20, 't': 13, 'e': 17, 'f': 2, 'r': 3, 'n': 10, 'c': 6, '.': 3, 'o': 3, 'd': 2, 'u': 1, ',': 1}
{}
```

In [262]:

```
s = "This is the first sentence. This is the scecond sentence. This is not the fourth sentence, it is the third sentence."


def f(s = ""):
        words = re.split("\\.|,| ", s)
        for word in words:
                if word == "":
                        words.remove(word)
        frequencies = dict.fromkeys(set(words), 0)
        for word in words:
                frequencies[word] += 1
        print(frequencies)


f(s)
```

```
{'is': 4, 'the': 4, 'fourth': 1, 'third': 1, 'it': 1, 'not': 1, 'first': 1, 'scecond': 1,
'sentence': 4, 'This': 3}
```

In [263]:
```
def f(s = ""):
        s = s.replace(".", "")
        s = s.replace(",", "")
        words = s.split()
        return dict(collections.Counter(words))


print(f(s))
```

```
{'This': 3, 'is': 4, 'the': 4, 'first': 1, 'sentence': 4, 'scecond': 1, 'not': 1, 'fourth': 1, 'it':
1, 'third': 1}
```

In [264]:
```
print(round(1.812))
```

```
2
```

In [265]:
```
n, d = (0.3).as_integer_ratio()
print(n, d)
print(1 / 3)
```

```
5404319552844595 18014398509481984
0.3333333333333333
```

In [266]:
```
print(round(0.325, 2))
```

```
0.33
```

In [267]:
```
print(f"{0.325:.40f}")
```

```
0.3250000000000000011102230246251565404236 3
```

In [268]:
```
def toJSON(a, *, b, **c):
        print(a, b, c)


try:
        toJSON(0, 1, 2, b = 3, c1 = 4, c2 = 5)
except TypeError as ex:
        print(type(ex).        )
```

```
TypeError
```

In [269]:
```
def toJSON(a, *, b, **c):
        print(a, b, c)


toJSON(0, b = 3, c1 = 4, c2 = 5)
```

```
0 3 {'c1': 4, 'c2': 5}
```

In [270]:
```
# def toJSON(a, *):
#       print(a)
```

```
#
#
# toJSON(0, 1, 2)
```

In [271]:
```
def toJSON(a, *args):
        print(a, args)


toJSON(0, 1, 2)
```

```
0 (1, 2)
```

In [272]:
```
l = ["AB CD", "EF CC", "GH BC", "IJ AC", "KL CA", "MN JQ", "OP BA"]
print(sorted(l, key = lambda e: (e[3], e[4])))
```

```
['IJ AC', 'OP BA', 'GH BC', 'KL CA', 'EF CC', 'AB CD', 'MN JQ']
```

In [273]:
```
# print(sorted(l, key = lambda e: e[len(e) - i] for i in range(len(e))))
print(sorted(l, key = lambda e: e[len(e) - 1]))
```

```
['KL CA', 'OP BA', 'EF CC', 'GH BC', 'IJ AC', 'AB CD', 'MN JQ']
```

In [274]:
```
a = (1 for _ in range(5))
print(a)
print(lambda _: "ABCD")
print(sorted)
print(range)
```

```
<generator object <genexpr> at 0x74efc64a24d0>
<function <lambda> at 0x74efc644b740>
<built-in function sorted>
<class 'range'>
```

In [275]:
```
def f(*a, b):
        print(a, b)


f(b = 0, *(1, 2))
```

```
(1, 2) 0
```

In [276]:
```
def f(a, *b, c):
        print(a, b, c)


f(0, c = 1, *(2, 3))
```

```
0 (2, 3) 1
```

In [277]:
```
print(statistics.fmean)
```

```
<function fmean at 0x74effaaa6a30>
```

In [278]:
```
print(dict([("a", 97), ("b", 98), ("c", 99)]))
```

```
{'a': 97, 'b': 98, 'c': 99}
```

In [279]:
```
widgets = [f"w{i}" for i in range(1, 21)]
skus = [f"sku{i}" for i in range(1, len(widgets) + 1)]
print(widgets)
print(skus)
```

```
['w1', 'w2', 'w3', 'w4', 'w5', 'w6', 'w7', 'w8', 'w9', 'w10', 'w11', 'w12', 'w13', 'w14', 'w15',
'w16', 'w17', 'w18', 'w19', 'w20']
['sku1', 'sku2', 'sku3', 'sku4', 'sku5', 'sku6', 'sku7', 'sku8', 'sku9', 'sku10', 'sku11', 'sku12',
'sku13', 'sku14', 'sku15', 'sku16', 'sku17', 'sku18', 'sku19', 'sku20']
```

In [280]:
```
print({w: s for w, s in zip(widgets, skus)})
```

```
{'w1': 'sku1', 'w2': 'sku2', 'w3': 'sku3', 'w4': 'sku4', 'w5': 'sku5', 'w6': 'sku6', 'w7': 'sku7',
'w8': 'sku8', 'w9': 'sku9', 'w10': 'sku10', 'w11': 'sku11', 'w12': 'sku12', 'w13': 'sku13', 'w14':
'sku14', 'w15': 'sku15', 'w16': 'sku16', 'w17': 'sku17', 'w18': 'sku18', 'w19': 'sku19', 'w20':
'sku20'}
```

In [281]:
```
print(dict(zip(widgets, skus)))
```

```
{'w1': 'sku1', 'w2': 'sku2', 'w3': 'sku3', 'w4': 'sku4', 'w5': 'sku5', 'w6': 'sku6', 'w7': 'sku7',
'w8': 'sku8', 'w9': 'sku9', 'w10': 'sku10', 'w11': 'sku11', 'w12': 'sku12', 'w13': 'sku13', 'w14':
'sku14', 'w15': 'sku15', 'w16': 'sku16', 'w17': 'sku17', 'w18': 'sku18', 'w19': 'sku19', 'w20':
'sku20'}
```

In [282]:
```
suits = "shdc"  # Spades, Hearts, Diamonds, Clubs
ranks = list("23456789") + ["10", "J", "Q", "K", "A"]
print(suits)
print(ranks)
```

```
shdc
['2', '3', '4', '5', '6', '7', '8', '9', '10', 'J', 'Q', 'K', 'A']
```

In [283]:
```
print([[(rank, suit) for rank in ranks] for suit in suits])
```

```
[[('2', 's'), ('3', 's'), ('4', 's'), ('5', 's'), ('6', 's'), ('7', 's'), ('8', 's'), ('9', 's'),
('10', 's'), ('J', 's'), ('Q', 's'), ('K', 's'), ('A', 's')], [('2', 'h'), ('3', 'h'), ('4', 'h'),
('5', 'h'), ('6', 'h'), ('7', 'h'), ('8', 'h'), ('9', 'h'), ('10', 'h'), ('J', 'h'), ('Q', 'h'),
('K', 'h'), ('A', 'h')], [('2', 'd'), ('3', 'd'), ('4', 'd'), ('5', 'd'), ('6', 'd'), ('7', 'd'),
('8', 'd'), ('9', 'd'), ('10', 'd'), ('J', 'd'), ('Q', 'd'), ('K', 'd'), ('A', 'd')], [('2', 'c'),
('3', 'c'), ('4', 'c'), ('5', 'c'), ('6', 'c'), ('7', 'c'), ('8', 'c'), ('9', 'c'), ('10', 'c'),
('J', 'c'), ('Q', 'c'), ('K', 'c'), ('A', 'c')]]
```

In [284]:
```
pprint.pprint([[(rank, suit) for rank in ranks] for suit in suits])
```

```
[[('2', 's'),
  ('3', 's'),
  ('4', 's'),
  ('5', 's'),
  ('6', 's'),
  ('7', 's'),
  ('8', 's'),
  ('9', 's'),
  ('10', 's'),
  ('J', 's'),
  ('Q', 's'),
  ('K', 's'),
  ('A', 's')],
 [('2', 'h'),
  ('3', 'h'),
  ('4', 'h'),
  ('5', 'h'),
  ('6', 'h'),
  ('7', 'h'),
  ('8', 'h'),
  ('9', 'h'),
  ('10', 'h'),
  ('J', 'h'),
  ('Q', 'h'),
  ('K', 'h'),
  ('A', 'h')],
 [('2', 'd'),
  ('3', 'd'),
  ('4', 'd'),
  ('5', 'd'),
  ('6', 'd'),
  ('7', 'd'),
  ('8', 'd'),
  ('9', 'd'),
  ('10', 'd'),
  ('J', 'd'),
  ('Q', 'd'),
  ('K', 'd'),
  ('A', 'd')],
 [('2', 'c'),
  ('3', 'c'),
  ('4', 'c'),
  ('5', 'c'),
  ('6', 'c'),
  ('7', 'c'),
  ('8', 'c'),
  ('9', 'c'),
  ('10', 'c'),
  ('J', 'c'),
  ('Q', 'c'),
  ('K', 'c'),
  ('A', 'c')]]
```

In [285]:

```
1 print([list(zip(ranks, s * 13)) for s in suits])
```

```
[[('2', 's'), ('3', 's'), ('4', 's'), ('5', 's'), ('6', 's'), ('7', 's'), ('8', 's'), ('9', 's'),
('10', 's'), ('J', 's'), ('Q', 's'), ('K', 's'), ('A', 's')], [('2', 'h'), ('3', 'h'), ('4', 'h'),
('5', 'h'), ('6', 'h'), ('7', 'h'), ('8', 'h'), ('9', 'h'), ('10', 'h'), ('J', 'h'), ('Q', 'h'),
('K', 'h'), ('A', 'h')], [('2', 'd'), ('3', 'd'), ('4', 'd'), ('5', 'd'), ('6', 'd'), ('7', 'd'),
('8', 'd'), ('9', 'd'), ('10', 'd'), ('J', 'd'), ('Q', 'd'), ('K', 'd'), ('A', 'd')], [('2', 'c'),
('3', 'c'), ('4', 'c'), ('5', 'c'), ('6', 'c'), ('7', 'c'), ('8', 'c'), ('9', 'c'), ('10', 'c'),
('J', 'c'), ('Q', 'c'), ('K', 'c'), ('A', 'c')]]
```

In [286]:

```
def f(list1, *, reverse = False):
        if not reverse:
                return sorted(list1)
        else:
                return sorted(list1, reverse = True)


print(f([3, 2, 4, 5, 1, 6, 8, 7]))
print(f([3, 2, 4, 5, 1, 6, 8, 7], reverse = True))
print(f([3, 2, 4, 5, 1, 6, 8, 7])[0])
print(f([3, 2, 4, 5, 1, 6, 8, 7], reverse = True)[0])
```

```
[1, 2, 3, 4, 5, 6, 7, 8]
[8, 7, 6, 5, 4, 3, 2, 1]
1
8
```

In [287]:

```
def f(list1, *, reverse = False):
        return sorted(list1, reverse = reverse)


print(f([3, 2, 4, 5, 1, 6, 8, 7]))
print(f([3, 2, 4, 5, 1, 6, 8, 7], reverse = True))
print(f([3, 2, 4, 5, 1, 6, 8, 7])[0])
print(f([3, 2, 4, 5, 1, 6, 8, 7], reverse = True)[0])
```

```
[1, 2, 3, 4, 5, 6, 7, 8]
[8, 7, 6, 5, 4, 3, 2, 1]
1
8
```

In [288]:

```
print(hex(ord(chr(0xE3).upper())))
```

```
0xc3
```

In [289]:

```
print("abc" in "defabcghi")
```

```
True
```

In [290]:

```
print("abc" in "abbde")
```

```
False
```

In [291]:

```
def solve(a, b, c):
        solution = ((-1 * b) + math.sqrt((b * b) - (4 * a * c))) / (2 * a)
        return round(solution, 2)


```

```
print(solve(3, 4, -5))
print(solve(1, -5, -8))
```

```
0.79
6.27
```

In [292]:
```
def encrypt(s):
        return "".join(chr(ord(c) + 10) for c in s)


print(encrypt("ABCDEFGHIJKLMNOPQRSTUVWXYZ"))
```

```
KLMNOPQRSTUVWXYZ[\]^_`abcd
```

In [293]:
```
def decrypt(s):
        return "".join(chr(ord(c) - 10) for c in s)


print(decrypt("KLMNOP"))
print(decrypt("S}kkm*Xo\x81~yx"))
```

```
ABCDEF
Isaac Newton
```

In [294]:
```
print(10 % 3)
```

```
1
```

In [295]:
```
a = array.array("l", [20])
print(a)
```

```
array('l', [20])
```

In [296]:
```
for i in a:
        print(i)
```

```
20
```

In [297]:
```
print(sys.getsizeof(a))
```

```
88
```

In [298]:
```
try:
        print(sys.getallocatedblocks(a))
except TypeError as e:
        print(type(e).        )
```

```
TypeError
```

In [299]:
```
print(sys.getsizeof(1))
```

```
28
```

In [300]:
```
def f(s):
        print(s)

```

```
4
5  try:
6          f(*"hello")
7  except TypeError as ex:
8          print(type(ex).         , ex.args)
```

```
TypeError ('f() takes 1 positional argument but 5 were given',)
```

In [301]:

```
1  def func(s):
2          print(s)
3
4
5  def apply(func, *str):
6          result = func(*str)
7          return result
8
9
10 apply(func, "john")
```

```
john
```

In [302]:

```
1  try:
2          apply(func, "john", "smith")
3  except TypeError as ex:
4          print(type(ex).         , *ex.args)
```

```
TypeError func() takes 1 positional argument but 2 were given
```

In [303]:

```
1  def encloseGetPower(b):
2          print("hex(id(b)):", hex(id(b)))
3
4          def getPower(e):
5                  print("hex(id(e)):", hex(id(e)))
6
7                  return b ** e
8
9          print("getPower:", getPower)
10
11         return getPower
12
13
14 getBinaryPower = encloseGetPower(2)
15 print(getBinaryPower(31))
16 getTertiaryPower = encloseGetPower(3)
17 print(getTertiaryPower(4))
```

```
hex(id(b)): 0xad7e90
getPower: <function encloseGetPower.<locals>.getPower at 0x74efc644bd70>
hex(id(e)): 0xad8230
2147483648
hex(id(b)): 0xad7eb0
getPower: <function encloseGetPower.<locals>.getPower at 0x74efc644be20>
hex(id(e)): 0xad7ed0
81
```

In [304]:
```
print("getBinaryPower:", getBinaryPower)
print("encloseGetPower:", encloseGetPower)
try:
        print("getPower:", getPower)
except NameError as ex:
        print(type(ex).         , ex.args)
```

```
getBinaryPower: <function encloseGetPower.<locals>.getPower at 0x74efc644bd70>
encloseGetPower: <function encloseGetPower at 0x74efc644b5e0>
NameError ("name 'getPower' is not defined",)
```

In [305]:
```
print("getBinaryPower:", getBinaryPower.           )
print("encloseGetPower:", encloseGetPower.           )
```

```
getBinaryPower: (<cell at 0x74efc24f1510: int object at 0xad7e90>,)
encloseGetPower: None
```

In [306]:
```
def outer(a, b):
        print(hex(id(a)))
        sumTotal = a + b

        def inner():
                prod = a * b
                print(a, b, sumTotal, prod)
                return "You just called a closure!"

        return inner


f = outer(2, 3)
```

```
0xad7e90
```

In [307]:
```
f = outer(2, 3)
```

```
0xad7e90
```

In [308]:
```
print(f)
```

```
<function outer.<locals>.inner at 0x74efc644bed0>
```

In [309]:
```
print(f.           )
```

```
(<cell at 0x74efc641f430: int object at 0xad7e90>, <cell at 0x74effaaf9db0: int object at 0xad7eb0>,
<cell at 0x74effb46b430: int object at 0xad7ef0>)
```

In [310]:
```
f()
```

```
2 3 5 6
```

Out[310]:
```
'You just called a closure!'
```

In [311]:
```
def execute(func):

        def inner(a, b):
```

```
                return func(a, b)

        return inner


def add(a, b):
        return a + b


add_executor = execute(add)
print(add_executor.          )
```

```
(<cell at 0x74efc24f0a30: function object at 0x74efc24f8040>,)
```

In [312]:
```
print(hex(id(add)))
```

```
0x74efc24f8040
```

In [313]:
```
try:
        print(hex(add))
except TypeError as ex:
        print(type(ex).        )
```

```
TypeError
```

In [314]:
```
add_executor(2, 3)
```

Out[314]: 5

In [315]:
```
def f():
        pass


print(f.          )
```

```
None
```

In [316]:
```
def f():

        def g():
                return

        return g


h = f()
print(f.          )
print(h.          )
```

```
None
None
```

In [317]:
```
a = 0
try:
```

```
        print(a.          )
except AttributeError as e:
        print(type(e).          )
```

```
AttributeError
```

In [318]:

```
a = []
try:
        print(a.          )
except AttributeError as e:
        print(type(e).          )
```

```
AttributeError
```

In [319]:

```
def execute(func):

        def inner(*posargs, **kwargs):
                return func(*posargs, **kwargs)

        return inner


def add(a, b, c):
        return a + b + c


def say_hello(name, *, formal = True):
        return f"Pleased to meet you, {name}" if formal else f"Hi, {name}!"


exec_add = execute(add)
exec_greet = execute(say_hello)
```

In [320]:

```
exec_add(1, 2, 3)
```

Out[320]: 6

In [321]:

```
exec_greet("Michael", formal = False)
```

Out[321]: 'Hi, Michael!'

In [322]:

```
exec_greet("Michael", formal = True)
```

Out[322]: 'Pleased to meet you, Michael'

In [323]:

```
print((0.1).as_integer_ratio())
```

```
(3602879701896397, 36028797018963968)
```

In [324]:

```
print((0.125).as_integer_ratio())
```

```
(1, 8)
```

In [325]:
```
print((0.2).as_integer_ratio())
print(f"{0.2:.40f}")
```

```
(3602879701896397, 18014398509481984)
0.2000000000000000011102230246251565404236 3
```

In [326]:
```
print(math.pi.as_integer_ratio())
n, d = math.pi.as_integer_ratio()
print(n / d)
```

```
(884279719003555, 281474976710656)
3.141592653589793
```

In [327]:
```
try:
        a1 = a1 + 1
        print(a1)
except NameError as ex:
        print(type(ex).        , ": ", *ex.args, ".", sep = "")
```

```
NameError: name 'a1' is not defined.
```

In [328]:
```
def getDiagonalMatrix(val, rows, columns):
        diagonalMatrix = [[val if ce == r else 0 for ce in range(4)] for r in
range(5)]
        return diagonalMatrix
```

In [329]:
```
def getDiagonalMatrix(rows, columns, *, val):
        diagonalMatrix = [[val if cl == row else 0 for cl in range(columns)]
for row in range(rows)]
        return diagonalMatrix
```

In [330]:
```
def getFactorial(n):
        factorial = 1
        for i in range(1, n + 1):
                factorial *= i
        return factorial
```

In [331]:
```
result = getFactorial(5)
print(result)
```

```
120
```

In [332]:
```
result = getDiagonalMatrix(3, 4, val = 5)
pprint.pprint(result, width = 14)
```

```
[[5, 0, 0, 0],
 [0, 5, 0, 0],
 [0, 0, 5, 0]]
```

In [333]:
```
print(f"{timeit.timeit("getFactorial(2)", globals = globals()):.4g}")
```

```
0.1758
```

In [334]:

```
n = 0.00000000000000123456789
n = 4.482882e-05
exp = 0
while n < 1:
        exp -= 3
        n *= 1000
prefixes = "", "m", "\N{GREEK SMALL LETTER MU}", "n", "p", "f", "a"
print(f"{n}{prefixes[abs(exp)//3]}")
```

```
44.82882µ
```

In [335]:

```
n = 1000000000
exp = 0
while n >= 1000:
        exp += 3
        n /= 1000
prefixes = "", "k", "M", "G", "T", "P", "E"
print(f"{n}{prefixes[exp//3]}")
```

```
1.0G
```

In [7]:

```
def toEnggNotation(n, scientificNotation = True, nSignificantDigits = 4):
        exp = 0
        oldN = n
        if n > 1:
                while n >= 1000:
                        exp += 3
                        n /= 1000
                prefixes = "", "k", "M", "G", "T", "P", "E"
                return f"{n:.3f}{prefixes[exp//3]}\t{oldN}" if
scientificNotation else f"{n}{prefixes[exp//3]}"
        elif n < 1:
                while n < 1:
                        exp -= 3
                        n *= 1000
                prefixes = "", "m", "\N{GREEK SMALL LETTER MU}", "n", "p", "f",
"a"
                return f"{n:.{nSignificantDigits}f}
{prefixes[abs(exp)//3]}\t{oldN}" if scientificNotation else f"{n}
{prefixes[abs(exp)//3]}"
        else:
                return 1
```

```
6
1
7
```

In [337]:

```
start = time.perf_counter()
result = getFactorial(1_000)
end = time.perf_counter()
print(f"Result: {result}")
print(f"Elapsed: {end - start}")
```

```
Result:
402387260077093773543702433923003985719374864210714632543799910429938512398629020592044208486969404800479988610197196058631666872994808558901323829669944590997424504087073759918823627727187325197795059509952761208749754624970436014182780946464962910563938874378864873371191810458257836478499770124766328898359557354325131853239584630755574091142624174743493475534286465766116677973966688202912073791438537195882498081268678383745597317461360853795345242215865932019280908782973084313928444032812315586110369768013573042161687476096758713483120254785893207671691324484262361314125087802080002616831510273418279777047846358681701643650241536913982812648102130927612448963599287051149649754199093422215668325720808213331861168115536158365469840467089756029009505376164758477284218896796462449451607653534081989013854424879849599533191017233555566021394503997362807501378376153071277619268490343526250001588853514733161170210396817592151090778801939317811419454525722386554146106289218796022383897147608850627686296714667469756291123408243920816015378088989396451826324367161676217916890977991190375403127462228998800519544441428201218736174599264295658174662830295557029902432415318161721046583203678690611726015878352075151628422554026517048330422614397428693306169089796848259012545832716822645806652676995865268227280707578139185817888965220816434834482599326604336766017699961283186078838615027946595513115655203609398818061213855860030143569452722420634463179746059468257310379008402443243846565724501440282188525247093519062092902313649327349756551395872055965422874977401141334696271542284586237738753823048386568897646192738381490014076731044664025989949022222176590433990188601856652648506179970235619389701786004081188972991831102117122984590164192106888438712185564612496079872290851929681937238864261483965738229112312502418664935314397013742853192664987533721894069428143411852015801412334482801505139969429015348307764456909907315243327828826986460278986432113908350621709500259738986355427719674282224875758676575234422020757363056949882508796892816275384886339690995982628095612145099487170124451646126037902930912088908694202851064018215439945715680594187274899809425474217358240106367740459574178516082923013535808184009699637252423056085590370062427124341690900415369010593398383577793941097002775347200000000000
```

```
print(toEnggNotation(myTimer(getFactorial, 1000)))
print(toEnggNotation(myTimer(getDiagonalMatrix, 3, 5, val = 7)))
```

```
402387260077093773543702433923003985719374864210714632543799910429938512398629020592044208486969404800479988610197196058631666872994808558901323829669944590997424504087073759918823627727188732519779505959095276120874975462497043601418278094646496291056393887437886487337119181045825783647849977012476632889835955735432513185323958463075557409114262417474349347553428646576611667797396668820291207379143853719588249808126867838374559737461360853795345242215865932019280908782973084313928444032812315586110369768013573042161687476096758713483120254785893207671691324484262361314125087802080002616831510273418279777047846358681701643650241536913982812648102130927612448963599287051149649754199093422215668325720808213331861168115536158365469840467089756029009505376164758477284218896796462449451607653534081989013854424879849599533191017233555566021394503997362807501378376153071277619268490343526252000158885351473316117021039681759215109077880193931781141945452572238655414610628921879602238389714760885062768629671466746975629112340824392081601537808898939645182632436716167621791689097799119037540312746222899880051954444142820121873617459926429565817466283029555702990243241531816172104658320367869061172601587835207515162842255402651704833042261439742869330616908979684825901254583271682264580665267699586526822728070757813918581788896522081643483448259932660433676601769996128318607883861502794659551311565520360939881806121385586003014356945272242063446317974605946825731037900840244324384656572450144028218852524709351906209290231364932734975655139587205596542287497740114133469627154228458623773875382304838656889764619273838149001407673104466402598994902222217659043399018860185665264850617997023561938970178600408118897299183110211712298459016419210688843871218556461249607987229085192968193723886426148396573822911231250241866493531439701374285319266498753372189406942814341185201580141233448280150513996942901534830776445690990731524332782882698646027898643211390835062170950025973898635542771967428222487575867657523442202075736305694988250879689281627538488633969099598262809561214509948717012445164612603790293091208890869420285106401821543994571568059418727489980942547421735824010636774045957417851608292301353580818400969963725242305608559037006242712434169090041536901059339838357779394109700277534720000000000
```

```
5.2100µ 5.209999983435409e-06
[[-5, 0, 0, 0], [0, -5, 0, 0]]
4.1080µ 4.108000041469495e-06
39916800
3.6170µ 3.6170000043966866e-06
[[-5, 0, 0, 0], [0, -5, 0, 0]]
2.4240µ 2.4240000016106933e-06
39916800
```

In [341]:
```
# print(eval("2 * 3"))
# print(eval("x = 2 and print(x)"))
```

In [342]:
```
# eval("x = 2")
# print(x)
```

In [343]:
```
x = 2
eval("x ** 2 - 1")
```

Out[343]: 3

In [344]:
```
def f(expr, x):
        print(eval(expr))


x = 2
f("x ** 2 - 1", x)
```

```
3
```

In [345]:
```
def f(x):
        return True


try:
        print(list(map(f(False), range(0, 5))))
except TypeError as ex:
        print(type(ex).         + ":", ex.args[0])
```

```
TypeError: 'bool' object is not callable
```

In [346]:
```
def getExtremum(expr, interval, granularity, minimax):

        def f(expr):

                def g(x):
                        return eval(expr)

                return g

        evalExpr = f(expr)
        print("Range:", list(range(interval[0], interval[1] + granularity,
granularity)))
        if minimax == "minimum":
```

```
                return min(map(evalExpr, range(interval[0], interval[1] + granularity, granularity)))
        elif minimax == "maximum":
                return max(map(evalExpr, range(interval[0], interval[1] + granularity, granularity)))
        else:
                return "Invalid minimax"


expr = "x ** 2 - 1"
print(getExtremum(expr, (-10, 10), 2, "minimum"))
print(getExtremum(expr, (-10, 10), 2, "maximum"))
```

```
Range: [-10, -8, -6, -4, -2, 0, 2, 4, 6, 8, 10]
-1
Range: [-10, -8, -6, -4, -2, 0, 2, 4, 6, 8, 10]
99
```

In [347]:

```
expr = "x ** 2 - 1"
f = lambda x: eval(expr)
print(f(3))
```

```
8
```

In [348]:

```
def getExtremum(expr, interval, resolution, minimax):

        def f(expr):

                def g(x):
                        return eval(expr)

                return g

        evalExpr = f(expr)
```

```
        l = [interval[0] + (i * (interval[1] - interval[0]) / (resolution - 1)) for i in range(resolution)]  # Cheated with the calc.
        return min(map(evalExpr, l)) if minimax == "minimum" else max(map(evalExpr, l))


expr = "x ** 2 - 1"
print(getExtremum(expr, (-10, 10), 5, "minimum"))
print(getExtremum(expr, (-10, 10), 5, "maximum"))
print(getExtremum(expr, (-10, 10), 6, "minimum"))
print(getExtremum(expr, (-10, 10), 6, "maximum"))
expr = "math.log10(x)"
print(getExtremum(expr, (1, 100), 20, "minimum"))
print(getExtremum(expr, (1, 100), 20, "maximum"))
expr = "math.sin(x)"
print(getExtremum(expr, (-10, 10), 20, "minimum"))
print(getExtremum(expr, (-10, 10), 20000, "maximum"))
expr = "x"
print(getExtremum(expr, (1, 6), 8, "minimum"))
print(getExtremum(expr, (1, 6), 8, "maximum"))
```

```
-1.0
99.0
3.0
99.0
0.0
2.0
-0.9999667804441291
0.999999996044897
1.0
6.0
```

In [349]:

```
def f(t):
        return t[0] ** t[1]


l = [(1, 1), (1, 2), (2, 1), (2, 2), (2, 3), (3, 2), (3, 3), (3, 4), (4, 3),
(4, 4)]

start = time.perf_counter()
for _ in range(10_000):
        result = []
        for t in l:
                result.append(f(t))
end = time.perf_counter()
forLoopTime = end - start
print(result)

start = time.perf_counter()
for _ in range(10_000):
        result = [f(t) for t in l]
end = time.perf_counter()
listCompTime = end - start
print(result)

start = time.perf_counter()
for _ in range(10_000):
        result = list(map(f, l))
end = time.perf_counter()
mapTime = end - start
print(result)

print(toEnggNotation(forLoopTime), toEnggNotation(listCompTime),
toEnggNotation(mapTime))
```

```
[1, 1, 2, 4, 8, 9, 27, 81, 64, 256]
[1, 1, 2, 4, 8, 9, 27, 81, 64, 256]
[1, 1, 2, 4, 8, 9, 27, 81, 64, 256]
15.1299m        0.015129860000001827 10.1019m   0.010101901999973961 10.2137m   0.010213654999972732
```

In [350]:

```
def power(x, y):
        return x ** y


def dist(pt1, pt2):
        return math.sqrt(sum(coord1 - coord2 for coord1, coord2 in zip(pt1,
pt2)))


def partial(f, operand2):

        def calcPartialFunction(operand1):
                return f(operand1, operand2)

        return calcPartialFunction


squares = partial(power, 2)
print(squares(3))
print(squares(4))
dist_from_origin = partial(dist, (0, 0))
print(dist_from_origin((1, 1)))
gcd_60 = partial(math.gcd, 60)
print(gcd_60(13))
log_2 = partial(math.log, 2)
print(log_2(8))
log_10 = partial(math.log, 10)
print(log_10(100))
```

```
log_16 = partial(math.log, 16)
print(log_16(65536))
```

```
9
16
1.4142135623730951
1
3.0
2.0
4.0
```

In [351]:

```
def norm(x, y):
        return math.sqrt(x ** 2 + y ** 2)


def find_index_min(seq):
        minima = min(seq)
        return seq.index(minima)


def logged(f):

        def g(*args):
                print("Logging...")
                start = time.perf_counter()
                print(f(*args))
```

```
                end = time.perf_counter()
                print(toEnggNotation(end - start))
                print("... Finished")

        return g


loggedNorm = logged(norm)
loggedNorm(3, 4)
loggedFindIndexMin = logged(find_index_min)
loggedFindIndexMin([8, 7, 2, 9, 4])
```

```
Logging...
5.0
16.9020µ        1.6901999970286852e-05
... Finished
Logging...
2
12.1130µ        1.2113000025237852e-05
... Finished
```

In [352]:

```
def encloseGetPower(f):

        def getPower(*args):
                return f(*args)

        print(hex(id(getPower)))
        return getPower


getPower = encloseGetPower(pow)
print(getPower(2, 5))
print(hex(id(getPower)))
print(hex(id(encloseGetPower)))
print(getPower.           )
print(encloseGetPower.           )
```

```
0x74efc24f9430
32
0x74efc24f9430
0x74efc24f94e0
(<cell at 0x74efc5818ac0: builtin_function_or_method object at 0x74f01fbb0630>,)
None
```

In [353]:

```
l = [(1, 2), (-4, -5.5), (10, -10), (-2, 2)]
print(sorted(l, key = lambda t: abs(t[0] - t[1]), reverse = True))
```

```
[(10, -10), (-2, 2), (-4, -5.5), (1, 2)]
```

In [354]:

```
l = ["s", "0", "1"]
card = "".join(reversed(l))
print(card)
card = ""
for k in reversed(l):
```

```
        card += k
print(card)
```

```
10s
10s
```

In [355]:

```
suits = 'shdc'  # Spades, Hearts, Diamonds, Clubs
ranks = list('23456789') + ['10', 'J', 'Q', 'K', 'A']
cards = [[rank + suit for rank in ranks] for suit in suits]


def sorter(suits, reverse = False):
        return sorted(suits, key = lambda card:
ranks.index(card.strip("s").strip("h").strip("d").strip("c")), reverse =
reverse)


sortedCards = []
for suit in cards:
        sortedCards.append(sorter(suit, reverse = True))
print(sortedCards)

sortedCards = []
for suit in cards:
        sortedCards.append(sorter(suit))
print(sortedCards)
```

```
[['As', 'Ks', 'Qs', 'Js', '10s', '9s', '8s', '7s', '6s', '5s', '4s', '3s', '2s'], ['Ah', 'Kh', 'Qh',
'Jh', '10h', '9h', '8h', '7h', '6h', '5h', '4h', '3h', '2h'], ['Ad', 'Kd', 'Qd', 'Jd', '10d', '9d',
'8d', '7d', '6d', '5d', '4d', '3d', '2d'], ['Ac', 'Kc', 'Qc', 'Jc', '10c', '9c', '8c', '7c', '6c',
'5c', '4c', '3c', '2c']]
[['2s', '3s', '4s', '5s', '6s', '7s', '8s', '9s', '10s', 'Js', 'Qs', 'Ks', 'As'], ['2h', '3h', '4h',
'5h', '6h', '7h', '8h', '9h', '10h', 'Jh', 'Qh', 'Kh', 'Ah'], ['2d', '3d', '4d', '5d', '6d', '7d',
'8d', '9d', '10d', 'Jd', 'Qd', 'Kd', 'Ad'], ['2c', '3c', '4c', '5c', '6c', '7c', '8c', '9c', '10c',
'Jc', 'Qc', 'Kc', 'Ac']]
```

In [356]:

```
data = {'S1': (100, 200, 80, 180), 'S2': (10, 20, 8, 18), 'S3': (50, 150, 50,
150)}
```

```
2

def findMinima(data, reverse = False):
3           minimax = min if reverse else max
            return minimax(data, key = lambda d: data[d][1] - data[d][2])
4


5   print(findMinima(data, reverse = True))
    print(findMinima(data))
6

7


8


9
 1
0
```

S2
S1

In [357]:

```
    quotes = [('AACC', 6.05, 6.07, 6.03, 6.05, 65800), ('AAME', 1.7, 1.82, 1.7,
1   1.82, 4300), ('AAON', 24.98, 25.07, 24.9, 24.94, 28200), ('AAPL', 317.99,
    319.57, 316.75, 317.13, 12901800), ('AATI', 3.82, 3.82, 3.74, 3.79, 194600),
2   ('AAWW', 60.89, 61.44, 60.5, 61.19, 272800), ('AAXJ', 65.4, 65.71, 65.28,
    65.56, 390300), ('ABAT', 4.01, 4.01, 3.95, 3.99, 656300), ('ABAX', 25.26,
3   25.49, 25.04, 25.42, 73700), ('ABBC', 11.75, 11.88, 11.48, 11.53, 29700),
    ('ABCB', 9.3, 9.3, 9.06, 9.14, 42600), ('ABCD', 3.25, 3.25, 3.11, 3.22,
4   122800), ('ABCO', 48.75, 50.41, 46.9, 50.37, 66300), ('ABCW', 0.52, 0.61, 0.52,
    0.53, 83000), ('ABFS', 25.98, 26.27, 25.41, 25.5, 384900), ('ABIO', 3.96, 4,
5   3.88, 4, 38500), ('ABMD', 11.94, 12, 11.69, 11.87, 122600), ('ABTL', 0.82,
    0.84, 0.82, 0.83, 28700), ('ABVA', 3.09, 3.25, 3.09, 3.25, 6200), ('ACAD',
6   0.76, 0.76, 0.7, 0.74, 341500), ('ACAS', 7.52, 7.72, 7.52, 7.66, 5199800),
    ('ACAT', 14.44, 14.44, 14.04, 14.2, 51700), ('ACCL', 8.11, 8.21, 7.94, 8.1,
7   456100), ('ACET', 8.01, 8.04, 7.13, 7.73, 575600), ('ACFC', 1.69, 1.7, 1.5,
    1.6, 12300), ('ACFN', 3.82, 4, 3.82, 3.98, 53700), ('ACGL', 89.76, 90.14,
8   89.39, 89.92, 240900), ('ACGY', 22.41, 22.56, 22.25, 22.46, 86800), ('ACHN',
    3.12, 3.2, 3.07, 3.16, 113700), ('ACIW', 26.96, 27.03, 26.63, 26.8, 157000),
9   ('ACLI', 33.65, 33.77, 33.45, 33.63, 28700), ('ACLS', 2.47, 2.63, 2.46, 2.53,
 1  1818800), ('ACMR', 2.69, 2.84, 2.37, 2.71, 158600), ('ACOM', 25.2, 26.6, 24.9,
0   26.56, 265300), ('ACOR', 26.67, 27.07, 26.38, 27.04, 1415000), ('ACPW', 1.84,
    1.89, 1.77, 1.85, 565500), ('ACTG', 27.2, 27.43, 26.86, 27.18, 228800),
    ('ACTI', 3.25, 3.26, 3.25, 3.26, 148500), ('ACTS', 2.08, 2.09, 2.07, 2.07,
    130500), ('ACUR', 2.6, 2.64, 2.51, 2.6, 16000), ('ACWI', 46.53, 46.7, 46.32,
    46.51, 286200), ('ACWX', 44.49, 44.66, 44.36, 44.6, 55500), ('ACXM', 18, 18.07,
    17.81, 18.01, 289800), ('ADAM', 7.34, 7.49, 7.33, 7.44, 81700), ('ADAT', 0.6,
    0.68, 0.59, 0.66, 86400), ('ADBE', 29.43, 29.71, 29.07, 29.14, 7585300),
    ('ADCT', 12.68, 12.69, 12.66, 12.68, 1660500), ('ADEP', 6.14, 6.14, 4.95, 5.61,
    71000), ('ADES', 6.2, 6.22, 6, 6.19, 4800), ('ADGF', 4.31, 4.55, 4.31, 4.54,
    10200)]
```

```
def myFilter(percentage = 10):
        return list(filter(lambda t: abs((t[4] - t[2]) / t[2]) > (percentage / 100), quotes))


print("10:", myFilter(10))
print("5:", myFilter(5))
print("1:", myFilter(1))
```

```
10: [('ABCW', 0.52, 0.61, 0.52, 0.53, 83000)]
5: [('ABCW', 0.52, 0.61, 0.52, 0.53, 83000), ('ACFC', 1.69, 1.7, 1.5, 1.6, 12300), ('ADEP', 6.14, 6.14, 4.95, 5.61, 71000)]
1: [('ABBC', 11.75, 11.88, 11.48, 11.53, 29700), ('ABCB', 9.3, 9.3, 9.06, 9.14, 42600), ('ABCW', 0.52, 0.61, 0.52, 0.53, 83000), ('ABFS', 25.98, 26.27, 25.41, 25.5, 384900), ('ABMD', 11.94, 12, 11.69, 11.87, 122600), ('ABTL', 0.82, 0.84, 0.82, 0.83, 28700), ('ACAD', 0.76, 0.76, 0.7, 0.74, 341500), ('ACAT', 14.44, 14.44, 14.04, 14.2, 51700), ('ACCL', 8.11, 8.21, 7.94, 8.1, 456100), ('ACET', 8.01, 8.04, 7.13, 7.73, 575600), ('ACFC', 1.69, 1.7, 1.5, 1.6, 12300), ('ACHN', 3.12, 3.2, 3.07, 3.16, 113700), ('ACLS', 2.47, 2.63, 2.46, 2.53, 1818800), ('ACMR', 2.69, 2.84, 2.37, 2.71, 158600), ('ACPW', 1.84, 1.89, 1.77, 1.85, 565500), ('ACUR', 2.6, 2.64, 2.51, 2.6, 16000), ('ADAT', 0.6, 0.68, 0.59, 0.66, 86400), ('ADBE', 29.43, 29.71, 29.07, 29.14, 7585300), ('ADEP', 6.14, 6.14, 4.95, 5.61, 71000)]
```

In [358]:
```
l = [5, 6, -4, 8]
print(min(l, key = abs))
```

```
-4
```

In [462]:
```
importlib.reload(logging)
logging.basicConfig(format = "%(asctime)s %(levelname)s %(message)s", datefmt = "%FT%T")
logging.error("This is an error")
```

```
2026-09-18T20:52:36 ERROR This is an error
```

In [360]:
```
logger = logging.getLogger(        )
logger.setLevel(logging.DEBUG)
logger.error("This is an error message")
logger.debug("This is a debug message")
```

```
2026-09-18T17:28:00b ERROR This is an error message
2026-09-18T17:28:00b DEBUG This is a debug message
```

In [361]:
```
def f():
        pass


print(f.        )
print(type(f.        ))
```

```
f
<class 'str'>
```

In [362]:
```
def partial(f):

        def callF(*pargs, **kwargs):
                logger.debug("Calling your function... " + f.        )
```

```
                start = time.perf_counter()
                result = f(*pargs, **kwargs)
                end = time.perf_counter()
                logger.debug("... Function returned.")
                logger.info(f"Elapsed Time: {toEnggNotation(end - start)}")
                # print(f"{toEnggNotation(end - start)}")
                return result

        return callF


def add(a, b):
        return a + b


def log3(a):
        return math.log(a, 3)


@partial
def log4(a):
        return math.log(a, 4)


myAdder = partial(add)
logger.info("myAdder: " + str(myAdder(2, 3)))
log3 = partial(log3)
logger.info("log3: " + str(log3(9)))
logger.info("log4: " + str(log4(16)))
```

```
2026-09-18T17:28:00b DEBUG Calling your function... add
2026-09-18T17:28:00b DEBUG ... Function returned.
2026-09-18T17:28:00b INFO Elapsed Time: 1.5930µ 1.5930000358821417e-06
2026-09-18T17:28:00b INFO myAdder: 5
2026-09-18T17:28:00b DEBUG Calling your function... log3
2026-09-18T17:28:00b DEBUG ... Function returned.
2026-09-18T17:28:00b INFO Elapsed Time: 20.6290µ        2.0628999948257842e-05
2026-09-18T17:28:00b INFO log3: 2.0
2026-09-18T17:28:00b DEBUG Calling your function... log4
2026-09-18T17:28:00b DEBUG ... Function returned.
2026-09-18T17:28:00b INFO Elapsed Time: 4.6590µ 4.659000012452452e-06
2026-09-18T17:28:00b INFO log4: 2.0
```

In [363]:

```
def cache(f):
        print("Initializing cache...")
        cacheDict = {}

        def callF(*pargs):
                if pargs in cacheDict:
                        print("Cache Hit")
                        return cacheDict[pargs]
                else:
                        print("Cache Miss")
                        result = f(*pargs)
```

```
                        cacheDict[pargs] = result
                        return result

        return callF


@cache
def add(a, b):
        return a + b


@cache
def mul(a, b):
        return a * b
```

```
Initializing cache...
Initializing cache...
```

In [364]:

```
print(add.           )
print(mul.           )
```

```
(<cell at 0x74efc252cdf0: dict object at 0x74efc253fb00>, <cell at 0x74f01f194070: function object
at 0x74efc2537b60>)
(<cell at 0x74efc252c310: dict object at 0x74efc253fb40>, <cell at 0x74efc252f220: function object
at 0x74efc2537d70>)
```

In [365]:

```
print(add(2, 3))
print(mul(2, 3))
print(add(2, 3))
print(mul(2, 3))
```

```
Cache Miss
5
Cache Miss
6
Cache Hit
5
Cache Hit
6
```

In [366]:

```
print(type(functools.lru_cache))
```

```
<class 'function'>
```

In [367]:

```
@functools.lru_cache(maxsize = 2)
def graham(a, b, c):
        print("Cache Miss")
        return a ** b ** c


print(graham(2, 2, 4))
```

```
Cache Miss
65536
```

In [368]:

```
print(graham(2, 2, 4))
print(graham(2, 2, 4))
print(graham(2, 2, 5))
print(graham(2, 2, 6))
print(graham(2, 2, 4))
```

```
65536
65536
Cache Miss
4294967296
Cache Miss
18446744073709551616
Cache Miss
65536
```

In [369]:

```
@functools.lru_cache(maxsize = 100)
def fib(n):
        print(f"Calling fib{n}...")
        if n <= 1:
                return n
        return fib(n - 1) + fib(n - 2)


print(fib(2000))
```

```
Calling fib2000...
Calling fib1999...
Calling fib1998...
Calling fib1997...
Calling fib1996...
Calling fib1995...
Calling fib1994...
Calling fib1993...
Calling fib1992...
Calling fib1991...
Calling fib1990...
Calling fib1989...
Calling fib1988...
Calling fib1987...
Calling fib1986...
Calling fib1985...
Calling fib1984...
Calling fib1983...
Calling fib1982...
Calling fib1981...
Calling fib1980...
Calling fib1979...
Calling fib1978...
Calling fib1977...
Calling fib1976...
Calling fib1975...
Calling fib1974...
Calling fib1973...
Calling fib1972...
Calling fib1971...
Calling fib1970...
Calling fib1969...
Calling fib1968...
Calling fib1967...
Calling fib1966...
Calling fib1965...
Calling fib1964...
Calling fib1963...
Calling fib1962...
Calling fib1961...
Calling fib1960...
Calling fib1959...
Calling fib1958...
Calling fib1957...
Calling fib1956...
Calling fib1955...
Calling fib1954...
Calling fib1953...
Calling fib1952...
Calling fib1951...
Calling fib1950...
Calling fib1949...
Calling fib1948...
Calling fib1947...
Calling fib1946...
Calling fib1945...
Calling fib1944...
Calling fib1943...
Calling fib1942...
Calling fib1941...
Calling fib1940...
```

```
Calling fib1939...
Calling fib1938...
Calling fib1937...
Calling fib1936...
Calling fib1935...
Calling fib1934...
Calling fib1933...
Calling fib1932...
Calling fib1931...
Calling fib1930...
Calling fib1929...
Calling fib1928...
Calling fib1927...
Calling fib1926...
Calling fib1925...
Calling fib1924...
Calling fib1923...
Calling fib1922...
Calling fib1921...
Calling fib1920...
Calling fib1919...
Calling fib1918...
Calling fib1917...
Calling fib1916...
Calling fib1915...
Calling fib1914...
Calling fib1913...
Calling fib1912...
Calling fib1911...
Calling fib1910...
Calling fib1909...
Calling fib1908...
Calling fib1907...
Calling fib1906...
Calling fib1905...
Calling fib1904...
Calling fib1903...
Calling fib1902...
Calling fib1901...
Calling fib1900...
Calling fib1899...
Calling fib1898...
Calling fib1897...
Calling fib1896...
Calling fib1895...
Calling fib1894...
Calling fib1893...
Calling fib1892...
Calling fib1891...
Calling fib1890...
Calling fib1889...
Calling fib1888...
Calling fib1887...
Calling fib1886...
Calling fib1885...
Calling fib1884...
Calling fib1883...
Calling fib1882...
Calling fib1881...
Calling fib1880...
Calling fib1879...
```

```
Calling fib1878...
Calling fib1877...
Calling fib1876...
Calling fib1875...
Calling fib1874...
Calling fib1873...
Calling fib1872...
Calling fib1871...
Calling fib1870...
Calling fib1869...
Calling fib1868...
Calling fib1867...
Calling fib1866...
Calling fib1865...
Calling fib1864...
Calling fib1863...
Calling fib1862...
Calling fib1861...
Calling fib1860...
Calling fib1859...
Calling fib1858...
Calling fib1857...
Calling fib1856...
Calling fib1855...
Calling fib1854...
Calling fib1853...
Calling fib1852...
Calling fib1851...
Calling fib1850...
Calling fib1849...
Calling fib1848...
Calling fib1847...
Calling fib1846...
Calling fib1845...
Calling fib1844...
Calling fib1843...
Calling fib1842...
Calling fib1841...
Calling fib1840...
Calling fib1839...
Calling fib1838...
Calling fib1837...
Calling fib1836...
Calling fib1835...
Calling fib1834...
Calling fib1833...
Calling fib1832...
Calling fib1831...
Calling fib1830...
Calling fib1829...
Calling fib1828...
Calling fib1827...
Calling fib1826...
Calling fib1825...
Calling fib1824...
Calling fib1823...
Calling fib1822...
Calling fib1821...
Calling fib1820...
Calling fib1819...
Calling fib1818...
```

```
Calling fib1817...
Calling fib1816...
Calling fib1815...
Calling fib1814...
Calling fib1813...
Calling fib1812...
Calling fib1811...
Calling fib1810...
Calling fib1809...
Calling fib1808...
Calling fib1807...
Calling fib1806...
Calling fib1805...
Calling fib1804...
Calling fib1803...
Calling fib1802...
Calling fib1801...
Calling fib1800...
Calling fib1799...
Calling fib1798...
Calling fib1797...
Calling fib1796...
Calling fib1795...
Calling fib1794...
Calling fib1793...
Calling fib1792...
Calling fib1791...
Calling fib1790...
Calling fib1789...
Calling fib1788...
Calling fib1787...
Calling fib1786...
Calling fib1785...
Calling fib1784...
Calling fib1783...
Calling fib1782...
Calling fib1781...
Calling fib1780...
Calling fib1779...
Calling fib1778...
Calling fib1777...
Calling fib1776...
Calling fib1775...
Calling fib1774...
Calling fib1773...
Calling fib1772...
Calling fib1771...
Calling fib1770...
Calling fib1769...
Calling fib1768...
Calling fib1767...
Calling fib1766...
Calling fib1765...
Calling fib1764...
Calling fib1763...
Calling fib1762...
Calling fib1761...
Calling fib1760...
Calling fib1759...
Calling fib1758...
Calling fib1757...
```

```
Calling fib1756...
Calling fib1755...
Calling fib1754...
Calling fib1753...
Calling fib1752...
Calling fib1751...
Calling fib1750...
Calling fib1749...
Calling fib1748...
Calling fib1747...
Calling fib1746...
Calling fib1745...
Calling fib1744...
Calling fib1743...
Calling fib1742...
Calling fib1741...
Calling fib1740...
Calling fib1739...
Calling fib1738...
Calling fib1737...
Calling fib1736...
Calling fib1735...
Calling fib1734...
Calling fib1733...
Calling fib1732...
Calling fib1731...
Calling fib1730...
Calling fib1729...
Calling fib1728...
Calling fib1727...
Calling fib1726...
Calling fib1725...
Calling fib1724...
Calling fib1723...
Calling fib1722...
Calling fib1721...
Calling fib1720...
Calling fib1719...
Calling fib1718...
Calling fib1717...
Calling fib1716...
Calling fib1715...
Calling fib1714...
Calling fib1713...
Calling fib1712...
Calling fib1711...
Calling fib1710...
Calling fib1709...
Calling fib1708...
Calling fib1707...
Calling fib1706...
Calling fib1705...
Calling fib1704...
Calling fib1703...
Calling fib1702...
Calling fib1701...
Calling fib1700...
Calling fib1699...
Calling fib1698...
Calling fib1697...
Calling fib1696...
```

```
Calling fib1695...
Calling fib1694...
Calling fib1693...
Calling fib1692...
Calling fib1691...
Calling fib1690...
Calling fib1689...
Calling fib1688...
Calling fib1687...
Calling fib1686...
Calling fib1685...
Calling fib1684...
Calling fib1683...
Calling fib1682...
Calling fib1681...
Calling fib1680...
Calling fib1679...
Calling fib1678...
Calling fib1677...
Calling fib1676...
Calling fib1675...
Calling fib1674...
Calling fib1673...
Calling fib1672...
Calling fib1671...
Calling fib1670...
Calling fib1669...
Calling fib1668...
Calling fib1667...
Calling fib1666...
Calling fib1665...
Calling fib1664...
Calling fib1663...
Calling fib1662...
Calling fib1661...
Calling fib1660...
Calling fib1659...
Calling fib1658...
Calling fib1657...
Calling fib1656...
Calling fib1655...
Calling fib1654...
Calling fib1653...
Calling fib1652...
Calling fib1651...
Calling fib1650...
Calling fib1649...
Calling fib1648...
Calling fib1647...
Calling fib1646...
Calling fib1645...
Calling fib1644...
Calling fib1643...
Calling fib1642...
Calling fib1641...
Calling fib1640...
Calling fib1639...
Calling fib1638...
Calling fib1637...
Calling fib1636...
Calling fib1635...
```

```
Calling fib1634...
Calling fib1633...
Calling fib1632...
Calling fib1631...
Calling fib1630...
Calling fib1629...
Calling fib1628...
Calling fib1627...
Calling fib1626...
Calling fib1625...
Calling fib1624...
Calling fib1623...
Calling fib1622...
Calling fib1621...
Calling fib1620...
Calling fib1619...
Calling fib1618...
Calling fib1617...
Calling fib1616...
Calling fib1615...
Calling fib1614...
Calling fib1613...
Calling fib1612...
Calling fib1611...
Calling fib1610...
Calling fib1609...
Calling fib1608...
Calling fib1607...
Calling fib1606...
Calling fib1605...
Calling fib1604...
Calling fib1603...
Calling fib1602...
Calling fib1601...
Calling fib1600...
Calling fib1599...
Calling fib1598...
Calling fib1597...
Calling fib1596...
Calling fib1595...
Calling fib1594...
Calling fib1593...
Calling fib1592...
Calling fib1591...
Calling fib1590...
Calling fib1589...
Calling fib1588...
Calling fib1587...
Calling fib1586...
Calling fib1585...
Calling fib1584...
Calling fib1583...
Calling fib1582...
Calling fib1581...
Calling fib1580...
Calling fib1579...
Calling fib1578...
Calling fib1577...
Calling fib1576...
Calling fib1575...
Calling fib1574...
```

```
Calling fib1573...
Calling fib1572...
Calling fib1571...
Calling fib1570...
Calling fib1569...
Calling fib1568...
Calling fib1567...
Calling fib1566...
Calling fib1565...
Calling fib1564...
Calling fib1563...
Calling fib1562...
Calling fib1561...
Calling fib1560...
Calling fib1559...
Calling fib1558...
Calling fib1557...
Calling fib1556...
Calling fib1555...
Calling fib1554...
Calling fib1553...
Calling fib1552...
Calling fib1551...
Calling fib1550...
Calling fib1549...
Calling fib1548...
Calling fib1547...
Calling fib1546...
Calling fib1545...
Calling fib1544...
Calling fib1543...
Calling fib1542...
Calling fib1541...
Calling fib1540...
Calling fib1539...
Calling fib1538...
Calling fib1537...
Calling fib1536...
Calling fib1535...
Calling fib1534...
Calling fib1533...
Calling fib1532...
Calling fib1531...
Calling fib1530...
Calling fib1529...
Calling fib1528...
Calling fib1527...
Calling fib1526...
Calling fib1525...
Calling fib1524...
Calling fib1523...
Calling fib1522...
Calling fib1521...
Calling fib1520...
Calling fib1519...
Calling fib1518...
Calling fib1517...
Calling fib1516...
Calling fib1515...
Calling fib1514...
Calling fib1513...
```

```
Calling fib1512...
Calling fib1511...
Calling fib1510...
Calling fib1509...
Calling fib1508...
Calling fib1507...
Calling fib1506...
Calling fib1505...
Calling fib1504...
Calling fib1503...
Calling fib1502...
Calling fib1501...
Calling fib1500...
Calling fib1499...
Calling fib1498...
Calling fib1497...
Calling fib1496...
Calling fib1495...
Calling fib1494...
Calling fib1493...
Calling fib1492...
Calling fib1491...
Calling fib1490...
Calling fib1489...
Calling fib1488...
Calling fib1487...
Calling fib1486...
Calling fib1485...
Calling fib1484...
Calling fib1483...
Calling fib1482...
Calling fib1481...
Calling fib1480...
Calling fib1479...
Calling fib1478...
Calling fib1477...
Calling fib1476...
Calling fib1475...
Calling fib1474...
Calling fib1473...
Calling fib1472...
Calling fib1471...
Calling fib1470...
Calling fib1469...
Calling fib1468...
Calling fib1467...
Calling fib1466...
Calling fib1465...
Calling fib1464...
Calling fib1463...
Calling fib1462...
Calling fib1461...
Calling fib1460...
Calling fib1459...
Calling fib1458...
Calling fib1457...
Calling fib1456...
Calling fib1455...
Calling fib1454...
Calling fib1453...
Calling fib1452...
```

```
Calling fib1451...
Calling fib1450...
Calling fib1449...
Calling fib1448...
Calling fib1447...
Calling fib1446...
Calling fib1445...
Calling fib1444...
Calling fib1443...
Calling fib1442...
Calling fib1441...
Calling fib1440...
Calling fib1439...
Calling fib1438...
Calling fib1437...
Calling fib1436...
Calling fib1435...
Calling fib1434...
Calling fib1433...
Calling fib1432...
Calling fib1431...
Calling fib1430...
Calling fib1429...
Calling fib1428...
Calling fib1427...
Calling fib1426...
Calling fib1425...
Calling fib1424...
Calling fib1423...
Calling fib1422...
Calling fib1421...
Calling fib1420...
Calling fib1419...
Calling fib1418...
Calling fib1417...
Calling fib1416...
Calling fib1415...
Calling fib1414...
Calling fib1413...
Calling fib1412...
Calling fib1411...
Calling fib1410...
Calling fib1409...
Calling fib1408...
Calling fib1407...
Calling fib1406...
Calling fib1405...
Calling fib1404...
Calling fib1403...
Calling fib1402...
Calling fib1401...
Calling fib1400...
Calling fib1399...
Calling fib1398...
Calling fib1397...
Calling fib1396...
Calling fib1395...
Calling fib1394...
Calling fib1393...
Calling fib1392...
Calling fib1391...
```

```
Calling fib1390...
Calling fib1389...
Calling fib1388...
Calling fib1387...
Calling fib1386...
Calling fib1385...
Calling fib1384...
Calling fib1383...
Calling fib1382...
Calling fib1381...
Calling fib1380...
Calling fib1379...
Calling fib1378...
Calling fib1377...
Calling fib1376...
Calling fib1375...
Calling fib1374...
Calling fib1373...
Calling fib1372...
Calling fib1371...
Calling fib1370...
Calling fib1369...
Calling fib1368...
Calling fib1367...
Calling fib1366...
Calling fib1365...
Calling fib1364...
Calling fib1363...
Calling fib1362...
Calling fib1361...
Calling fib1360...
Calling fib1359...
Calling fib1358...
Calling fib1357...
Calling fib1356...
Calling fib1355...
Calling fib1354...
Calling fib1353...
Calling fib1352...
Calling fib1351...
Calling fib1350...
Calling fib1349...
Calling fib1348...
Calling fib1347...
Calling fib1346...
Calling fib1345...
Calling fib1344...
Calling fib1343...
Calling fib1342...
Calling fib1341...
Calling fib1340...
Calling fib1339...
Calling fib1338...
Calling fib1337...
Calling fib1336...
Calling fib1335...
Calling fib1334...
Calling fib1333...
Calling fib1332...
Calling fib1331...
Calling fib1330...
```

```
Calling fib1329...
Calling fib1328...
Calling fib1327...
Calling fib1326...
Calling fib1325...
Calling fib1324...
Calling fib1323...
Calling fib1322...
Calling fib1321...
Calling fib1320...
Calling fib1319...
Calling fib1318...
Calling fib1317...
Calling fib1316...
Calling fib1315...
Calling fib1314...
Calling fib1313...
Calling fib1312...
Calling fib1311...
Calling fib1310...
Calling fib1309...
Calling fib1308...
Calling fib1307...
Calling fib1306...
Calling fib1305...
Calling fib1304...
Calling fib1303...
Calling fib1302...
Calling fib1301...
Calling fib1300...
Calling fib1299...
Calling fib1298...
Calling fib1297...
Calling fib1296...
Calling fib1295...
Calling fib1294...
Calling fib1293...
Calling fib1292...
Calling fib1291...
Calling fib1290...
Calling fib1289...
Calling fib1288...
Calling fib1287...
Calling fib1286...
Calling fib1285...
Calling fib1284...
Calling fib1283...
Calling fib1282...
Calling fib1281...
Calling fib1280...
Calling fib1279...
Calling fib1278...
Calling fib1277...
Calling fib1276...
Calling fib1275...
Calling fib1274...
Calling fib1273...
Calling fib1272...
Calling fib1271...
Calling fib1270...
Calling fib1269...
```

```
Calling fib1268...
Calling fib1267...
Calling fib1266...
Calling fib1265...
Calling fib1264...
Calling fib1263...
Calling fib1262...
Calling fib1261...
Calling fib1260...
Calling fib1259...
Calling fib1258...
Calling fib1257...
Calling fib1256...
Calling fib1255...
Calling fib1254...
Calling fib1253...
Calling fib1252...
Calling fib1251...
Calling fib1250...
Calling fib1249...
Calling fib1248...
Calling fib1247...
Calling fib1246...
Calling fib1245...
Calling fib1244...
Calling fib1243...
Calling fib1242...
Calling fib1241...
Calling fib1240...
Calling fib1239...
Calling fib1238...
Calling fib1237...
Calling fib1236...
Calling fib1235...
Calling fib1234...
Calling fib1233...
Calling fib1232...
Calling fib1231...
Calling fib1230...
Calling fib1229...
Calling fib1228...
Calling fib1227...
Calling fib1226...
Calling fib1225...
Calling fib1224...
Calling fib1223...
Calling fib1222...
Calling fib1221...
Calling fib1220...
Calling fib1219...
Calling fib1218...
Calling fib1217...
Calling fib1216...
Calling fib1215...
Calling fib1214...
Calling fib1213...
Calling fib1212...
Calling fib1211...
Calling fib1210...
Calling fib1209...
Calling fib1208...
```

```
Calling fib1207...
Calling fib1206...
Calling fib1205...
Calling fib1204...
Calling fib1203...
Calling fib1202...
Calling fib1201...
Calling fib1200...
Calling fib1199...
Calling fib1198...
Calling fib1197...
Calling fib1196...
Calling fib1195...
Calling fib1194...
Calling fib1193...
Calling fib1192...
Calling fib1191...
Calling fib1190...
Calling fib1189...
Calling fib1188...
Calling fib1187...
Calling fib1186...
Calling fib1185...
Calling fib1184...
Calling fib1183...
Calling fib1182...
Calling fib1181...
Calling fib1180...
Calling fib1179...
Calling fib1178...
Calling fib1177...
Calling fib1176...
Calling fib1175...
Calling fib1174...
Calling fib1173...
Calling fib1172...
Calling fib1171...
Calling fib1170...
Calling fib1169...
Calling fib1168...
Calling fib1167...
Calling fib1166...
Calling fib1165...
Calling fib1164...
Calling fib1163...
Calling fib1162...
Calling fib1161...
Calling fib1160...
Calling fib1159...
Calling fib1158...
Calling fib1157...
Calling fib1156...
Calling fib1155...
Calling fib1154...
Calling fib1153...
Calling fib1152...
Calling fib1151...
Calling fib1150...
Calling fib1149...
Calling fib1148...
Calling fib1147...
```

```
Calling fib1146...
Calling fib1145...
Calling fib1144...
Calling fib1143...
Calling fib1142...
Calling fib1141...
Calling fib1140...
Calling fib1139...
Calling fib1138...
Calling fib1137...
Calling fib1136...
Calling fib1135...
Calling fib1134...
Calling fib1133...
Calling fib1132...
Calling fib1131...
Calling fib1130...
Calling fib1129...
Calling fib1128...
Calling fib1127...
Calling fib1126...
Calling fib1125...
Calling fib1124...
Calling fib1123...
Calling fib1122...
Calling fib1121...
Calling fib1120...
Calling fib1119...
Calling fib1118...
Calling fib1117...
Calling fib1116...
Calling fib1115...
Calling fib1114...
Calling fib1113...
Calling fib1112...
Calling fib1111...
Calling fib1110...
Calling fib1109...
Calling fib1108...
Calling fib1107...
Calling fib1106...
Calling fib1105...
Calling fib1104...
Calling fib1103...
Calling fib1102...
Calling fib1101...
Calling fib1100...
Calling fib1099...
Calling fib1098...
Calling fib1097...
Calling fib1096...
Calling fib1095...
Calling fib1094...
Calling fib1093...
Calling fib1092...
Calling fib1091...
Calling fib1090...
Calling fib1089...
Calling fib1088...
Calling fib1087...
Calling fib1086...
```

```
Calling fib1085...
Calling fib1084...
Calling fib1083...
Calling fib1082...
Calling fib1081...
Calling fib1080...
Calling fib1079...
Calling fib1078...
Calling fib1077...
Calling fib1076...
Calling fib1075...
Calling fib1074...
Calling fib1073...
Calling fib1072...
Calling fib1071...
Calling fib1070...
Calling fib1069...
Calling fib1068...
Calling fib1067...
Calling fib1066...
Calling fib1065...
Calling fib1064...
Calling fib1063...
Calling fib1062...
Calling fib1061...
Calling fib1060...
Calling fib1059...
Calling fib1058...
Calling fib1057...
Calling fib1056...
Calling fib1055...
Calling fib1054...
Calling fib1053...
Calling fib1052...
Calling fib1051...
Calling fib1050...
Calling fib1049...
Calling fib1048...
Calling fib1047...
Calling fib1046...
Calling fib1045...
Calling fib1044...
Calling fib1043...
Calling fib1042...
Calling fib1041...
Calling fib1040...
Calling fib1039...
Calling fib1038...
Calling fib1037...
Calling fib1036...
Calling fib1035...
Calling fib1034...
Calling fib1033...
Calling fib1032...
Calling fib1031...
Calling fib1030...
Calling fib1029...
Calling fib1028...
Calling fib1027...
Calling fib1026...
Calling fib1025...
```

```
Calling fib1024...
Calling fib1023...
Calling fib1022...
Calling fib1021...
Calling fib1020...
Calling fib1019...
Calling fib1018...
Calling fib1017...
Calling fib1016...
Calling fib1015...
Calling fib1014...
Calling fib1013...
Calling fib1012...
Calling fib1011...
Calling fib1010...
Calling fib1009...
Calling fib1008...
Calling fib1007...
Calling fib1006...
Calling fib1005...
Calling fib1004...
Calling fib1003...
Calling fib1002...
Calling fib1001...
Calling fib1000...
Calling fib999...
Calling fib998...
Calling fib997...
Calling fib996...
Calling fib995...
Calling fib994...
Calling fib993...
Calling fib992...
Calling fib991...
Calling fib990...
Calling fib989...
Calling fib988...
Calling fib987...
Calling fib986...
Calling fib985...
Calling fib984...
Calling fib983...
Calling fib982...
Calling fib981...
Calling fib980...
Calling fib979...
Calling fib978...
Calling fib977...
Calling fib976...
Calling fib975...
Calling fib974...
Calling fib973...
Calling fib972...
Calling fib971...
Calling fib970...
Calling fib969...
Calling fib968...
Calling fib967...
Calling fib966...
Calling fib965...
Calling fib964...
```

```
Calling fib963...
Calling fib962...
Calling fib961...
Calling fib960...
Calling fib959...
Calling fib958...
Calling fib957...
Calling fib956...
Calling fib955...
Calling fib954...
Calling fib953...
Calling fib952...
Calling fib951...
Calling fib950...
Calling fib949...
Calling fib948...
Calling fib947...
Calling fib946...
Calling fib945...
Calling fib944...
Calling fib943...
Calling fib942...
Calling fib941...
Calling fib940...
Calling fib939...
Calling fib938...
Calling fib937...
Calling fib936...
Calling fib935...
Calling fib934...
Calling fib933...
Calling fib932...
Calling fib931...
Calling fib930...
Calling fib929...
Calling fib928...
Calling fib927...
Calling fib926...
Calling fib925...
Calling fib924...
Calling fib923...
Calling fib922...
Calling fib921...
Calling fib920...
Calling fib919...
Calling fib918...
Calling fib917...
Calling fib916...
Calling fib915...
Calling fib914...
Calling fib913...
Calling fib912...
Calling fib911...
Calling fib910...
Calling fib909...
Calling fib908...
Calling fib907...
Calling fib906...
Calling fib905...
Calling fib904...
Calling fib903...
```

```
Calling fib902...
Calling fib901...
Calling fib900...
Calling fib899...
Calling fib898...
Calling fib897...
Calling fib896...
Calling fib895...
Calling fib894...
Calling fib893...
Calling fib892...
Calling fib891...
Calling fib890...
Calling fib889...
Calling fib888...
Calling fib887...
Calling fib886...
Calling fib885...
Calling fib884...
Calling fib883...
Calling fib882...
Calling fib881...
Calling fib880...
Calling fib879...
Calling fib878...
Calling fib877...
Calling fib876...
Calling fib875...
Calling fib874...
Calling fib873...
Calling fib872...
Calling fib871...
Calling fib870...
Calling fib869...
Calling fib868...
Calling fib867...
Calling fib866...
Calling fib865...
Calling fib864...
Calling fib863...
Calling fib862...
Calling fib861...
Calling fib860...
Calling fib859...
Calling fib858...
Calling fib857...
Calling fib856...
Calling fib855...
Calling fib854...
Calling fib853...
Calling fib852...
Calling fib851...
Calling fib850...
Calling fib849...
Calling fib848...
Calling fib847...
Calling fib846...
Calling fib845...
Calling fib844...
Calling fib843...
Calling fib842...
```

```
Calling fib841...
Calling fib840...
Calling fib839...
Calling fib838...
Calling fib837...
Calling fib836...
Calling fib835...
Calling fib834...
Calling fib833...
Calling fib832...
Calling fib831...
Calling fib830...
Calling fib829...
Calling fib828...
Calling fib827...
Calling fib826...
Calling fib825...
Calling fib824...
Calling fib823...
Calling fib822...
Calling fib821...
Calling fib820...
Calling fib819...
Calling fib818...
Calling fib817...
Calling fib816...
Calling fib815...
Calling fib814...
Calling fib813...
Calling fib812...
Calling fib811...
Calling fib810...
Calling fib809...
Calling fib808...
Calling fib807...
Calling fib806...
Calling fib805...
Calling fib804...
Calling fib803...
Calling fib802...
Calling fib801...
Calling fib800...
Calling fib799...
Calling fib798...
Calling fib797...
Calling fib796...
Calling fib795...
Calling fib794...
Calling fib793...
Calling fib792...
Calling fib791...
Calling fib790...
Calling fib789...
Calling fib788...
Calling fib787...
Calling fib786...
Calling fib785...
Calling fib784...
Calling fib783...
Calling fib782...
Calling fib781...
```

```
Calling fib780...
Calling fib779...
Calling fib778...
Calling fib777...
Calling fib776...
Calling fib775...
Calling fib774...
Calling fib773...
Calling fib772...
Calling fib771...
Calling fib770...
Calling fib769...
Calling fib768...
Calling fib767...
Calling fib766...
Calling fib765...
Calling fib764...
Calling fib763...
Calling fib762...
Calling fib761...
Calling fib760...
Calling fib759...
Calling fib758...
Calling fib757...
Calling fib756...
Calling fib755...
Calling fib754...
Calling fib753...
Calling fib752...
Calling fib751...
Calling fib750...
Calling fib749...
Calling fib748...
Calling fib747...
Calling fib746...
Calling fib745...
Calling fib744...
Calling fib743...
Calling fib742...
Calling fib741...
Calling fib740...
Calling fib739...
Calling fib738...
Calling fib737...
Calling fib736...
Calling fib735...
Calling fib734...
Calling fib733...
Calling fib732...
Calling fib731...
Calling fib730...
Calling fib729...
Calling fib728...
Calling fib727...
Calling fib726...
Calling fib725...
Calling fib724...
Calling fib723...
Calling fib722...
Calling fib721...
Calling fib720...
```

```
Calling fib719...
Calling fib718...
Calling fib717...
Calling fib716...
Calling fib715...
Calling fib714...
Calling fib713...
Calling fib712...
Calling fib711...
Calling fib710...
Calling fib709...
Calling fib708...
Calling fib707...
Calling fib706...
Calling fib705...
Calling fib704...
Calling fib703...
Calling fib702...
Calling fib701...
Calling fib700...
Calling fib699...
Calling fib698...
Calling fib697...
Calling fib696...
Calling fib695...
Calling fib694...
Calling fib693...
Calling fib692...
Calling fib691...
Calling fib690...
Calling fib689...
Calling fib688...
Calling fib687...
Calling fib686...
Calling fib685...
Calling fib684...
Calling fib683...
Calling fib682...
Calling fib681...
Calling fib680...
Calling fib679...
Calling fib678...
Calling fib677...
Calling fib676...
Calling fib675...
Calling fib674...
Calling fib673...
Calling fib672...
Calling fib671...
Calling fib670...
Calling fib669...
Calling fib668...
Calling fib667...
Calling fib666...
Calling fib665...
Calling fib664...
Calling fib663...
Calling fib662...
Calling fib661...
Calling fib660...
Calling fib659...
```

```
Calling fib658...
Calling fib657...
Calling fib656...
Calling fib655...
Calling fib654...
Calling fib653...
Calling fib652...
Calling fib651...
Calling fib650...
Calling fib649...
Calling fib648...
Calling fib647...
Calling fib646...
Calling fib645...
Calling fib644...
Calling fib643...
Calling fib642...
Calling fib641...
Calling fib640...
Calling fib639...
Calling fib638...
Calling fib637...
Calling fib636...
Calling fib635...
Calling fib634...
Calling fib633...
Calling fib632...
Calling fib631...
Calling fib630...
Calling fib629...
Calling fib628...
Calling fib627...
Calling fib626...
Calling fib625...
Calling fib624...
Calling fib623...
Calling fib622...
Calling fib621...
Calling fib620...
Calling fib619...
Calling fib618...
Calling fib617...
Calling fib616...
Calling fib615...
Calling fib614...
Calling fib613...
Calling fib612...
Calling fib611...
Calling fib610...
Calling fib609...
Calling fib608...
Calling fib607...
Calling fib606...
Calling fib605...
Calling fib604...
Calling fib603...
Calling fib602...
Calling fib601...
Calling fib600...
Calling fib599...
Calling fib598...
```

```
Calling fib597...
Calling fib596...
Calling fib595...
Calling fib594...
Calling fib593...
Calling fib592...
Calling fib591...
Calling fib590...
Calling fib589...
Calling fib588...
Calling fib587...
Calling fib586...
Calling fib585...
Calling fib584...
Calling fib583...
Calling fib582...
Calling fib581...
Calling fib580...
Calling fib579...
Calling fib578...
Calling fib577...
Calling fib576...
Calling fib575...
Calling fib574...
Calling fib573...
Calling fib572...
Calling fib571...
Calling fib570...
Calling fib569...
Calling fib568...
Calling fib567...
Calling fib566...
Calling fib565...
Calling fib564...
Calling fib563...
Calling fib562...
Calling fib561...
Calling fib560...
Calling fib559...
Calling fib558...
Calling fib557...
Calling fib556...
Calling fib555...
Calling fib554...
Calling fib553...
Calling fib552...
Calling fib551...
Calling fib550...
Calling fib549...
Calling fib548...
Calling fib547...
Calling fib546...
Calling fib545...
Calling fib544...
Calling fib543...
Calling fib542...
Calling fib541...
Calling fib540...
Calling fib539...
Calling fib538...
Calling fib537...
```

```
Calling fib536...
Calling fib535...
Calling fib534...
Calling fib533...
Calling fib532...
Calling fib531...
Calling fib530...
Calling fib529...
Calling fib528...
Calling fib527...
Calling fib526...
Calling fib525...
Calling fib524...
Calling fib523...
Calling fib522...
Calling fib521...
Calling fib520...
Calling fib519...
Calling fib518...
Calling fib517...
Calling fib516...
Calling fib515...
Calling fib514...
Calling fib513...
Calling fib512...
Calling fib511...
Calling fib510...
Calling fib509...
Calling fib508...
Calling fib507...
Calling fib506...
Calling fib505...
Calling fib504...
Calling fib503...
Calling fib502...
Calling fib501...
Calling fib500...
Calling fib499...
Calling fib498...
Calling fib497...
Calling fib496...
Calling fib495...
Calling fib494...
Calling fib493...
Calling fib492...
Calling fib491...
Calling fib490...
Calling fib489...
Calling fib488...
Calling fib487...
Calling fib486...
Calling fib485...
Calling fib484...
Calling fib483...
Calling fib482...
Calling fib481...
Calling fib480...
Calling fib479...
Calling fib478...
Calling fib477...
Calling fib476...
```

```
Calling fib475...
Calling fib474...
Calling fib473...
Calling fib472...
Calling fib471...
Calling fib470...
Calling fib469...
Calling fib468...
Calling fib467...
Calling fib466...
Calling fib465...
Calling fib464...
Calling fib463...
Calling fib462...
Calling fib461...
Calling fib460...
Calling fib459...
Calling fib458...
Calling fib457...
Calling fib456...
Calling fib455...
Calling fib454...
Calling fib453...
Calling fib452...
Calling fib451...
Calling fib450...
Calling fib449...
Calling fib448...
Calling fib447...
Calling fib446...
Calling fib445...
Calling fib444...
Calling fib443...
Calling fib442...
Calling fib441...
Calling fib440...
Calling fib439...
Calling fib438...
Calling fib437...
Calling fib436...
Calling fib435...
Calling fib434...
Calling fib433...
Calling fib432...
Calling fib431...
Calling fib430...
Calling fib429...
Calling fib428...
Calling fib427...
Calling fib426...
Calling fib425...
Calling fib424...
Calling fib423...
Calling fib422...
Calling fib421...
Calling fib420...
Calling fib419...
Calling fib418...
Calling fib417...
Calling fib416...
Calling fib415...
```

```
Calling fib414...
Calling fib413...
Calling fib412...
Calling fib411...
Calling fib410...
Calling fib409...
Calling fib408...
Calling fib407...
Calling fib406...
Calling fib405...
Calling fib404...
Calling fib403...
Calling fib402...
Calling fib401...
Calling fib400...
Calling fib399...
Calling fib398...
Calling fib397...
Calling fib396...
Calling fib395...
Calling fib394...
Calling fib393...
Calling fib392...
Calling fib391...
Calling fib390...
Calling fib389...
Calling fib388...
Calling fib387...
Calling fib386...
Calling fib385...
Calling fib384...
Calling fib383...
Calling fib382...
Calling fib381...
Calling fib380...
Calling fib379...
Calling fib378...
Calling fib377...
Calling fib376...
Calling fib375...
Calling fib374...
Calling fib373...
Calling fib372...
Calling fib371...
Calling fib370...
Calling fib369...
Calling fib368...
Calling fib367...
Calling fib366...
Calling fib365...
Calling fib364...
Calling fib363...
Calling fib362...
Calling fib361...
Calling fib360...
Calling fib359...
Calling fib358...
Calling fib357...
Calling fib356...
Calling fib355...
Calling fib354...
```

```
Calling fib353...
Calling fib352...
Calling fib351...
Calling fib350...
Calling fib349...
Calling fib348...
Calling fib347...
Calling fib346...
Calling fib345...
Calling fib344...
Calling fib343...
Calling fib342...
Calling fib341...
Calling fib340...
Calling fib339...
Calling fib338...
Calling fib337...
Calling fib336...
Calling fib335...
Calling fib334...
Calling fib333...
Calling fib332...
Calling fib331...
Calling fib330...
Calling fib329...
Calling fib328...
Calling fib327...
Calling fib326...
Calling fib325...
Calling fib324...
Calling fib323...
Calling fib322...
Calling fib321...
Calling fib320...
Calling fib319...
Calling fib318...
Calling fib317...
Calling fib316...
Calling fib315...
Calling fib314...
Calling fib313...
Calling fib312...
Calling fib311...
Calling fib310...
Calling fib309...
Calling fib308...
Calling fib307...
Calling fib306...
Calling fib305...
Calling fib304...
Calling fib303...
Calling fib302...
Calling fib301...
Calling fib300...
Calling fib299...
Calling fib298...
Calling fib297...
Calling fib296...
Calling fib295...
Calling fib294...
Calling fib293...
```

```
Calling fib292...
Calling fib291...
Calling fib290...
Calling fib289...
Calling fib288...
Calling fib287...
Calling fib286...
Calling fib285...
Calling fib284...
Calling fib283...
Calling fib282...
Calling fib281...
Calling fib280...
Calling fib279...
Calling fib278...
Calling fib277...
Calling fib276...
Calling fib275...
Calling fib274...
Calling fib273...
Calling fib272...
Calling fib271...
Calling fib270...
Calling fib269...
Calling fib268...
Calling fib267...
Calling fib266...
Calling fib265...
Calling fib264...
Calling fib263...
Calling fib262...
Calling fib261...
Calling fib260...
Calling fib259...
Calling fib258...
Calling fib257...
Calling fib256...
Calling fib255...
Calling fib254...
Calling fib253...
Calling fib252...
Calling fib251...
Calling fib250...
Calling fib249...
Calling fib248...
Calling fib247...
Calling fib246...
Calling fib245...
Calling fib244...
Calling fib243...
Calling fib242...
Calling fib241...
Calling fib240...
Calling fib239...
Calling fib238...
Calling fib237...
Calling fib236...
Calling fib235...
Calling fib234...
Calling fib233...
Calling fib232...
```

```
Calling fib231...
Calling fib230...
Calling fib229...
Calling fib228...
Calling fib227...
Calling fib226...
Calling fib225...
Calling fib224...
Calling fib223...
Calling fib222...
Calling fib221...
Calling fib220...
Calling fib219...
Calling fib218...
Calling fib217...
Calling fib216...
Calling fib215...
Calling fib214...
Calling fib213...
Calling fib212...
Calling fib211...
Calling fib210...
Calling fib209...
Calling fib208...
Calling fib207...
Calling fib206...
Calling fib205...
Calling fib204...
Calling fib203...
Calling fib202...
Calling fib201...
Calling fib200...
Calling fib199...
Calling fib198...
Calling fib197...
Calling fib196...
Calling fib195...
Calling fib194...
Calling fib193...
Calling fib192...
Calling fib191...
Calling fib190...
Calling fib189...
Calling fib188...
Calling fib187...
Calling fib186...
Calling fib185...
Calling fib184...
Calling fib183...
Calling fib182...
Calling fib181...
Calling fib180...
Calling fib179...
Calling fib178...
Calling fib177...
Calling fib176...
Calling fib175...
Calling fib174...
Calling fib173...
Calling fib172...
Calling fib171...
```

```
Calling fib170...
Calling fib169...
Calling fib168...
Calling fib167...
Calling fib166...
Calling fib165...
Calling fib164...
Calling fib163...
Calling fib162...
Calling fib161...
Calling fib160...
Calling fib159...
Calling fib158...
Calling fib157...
Calling fib156...
Calling fib155...
Calling fib154...
Calling fib153...
Calling fib152...
Calling fib151...
Calling fib150...
Calling fib149...
Calling fib148...
Calling fib147...
Calling fib146...
Calling fib145...
Calling fib144...
Calling fib143...
Calling fib142...
Calling fib141...
Calling fib140...
Calling fib139...
Calling fib138...
Calling fib137...
Calling fib136...
Calling fib135...
Calling fib134...
Calling fib133...
Calling fib132...
Calling fib131...
Calling fib130...
Calling fib129...
Calling fib128...
Calling fib127...
Calling fib126...
Calling fib125...
Calling fib124...
Calling fib123...
Calling fib122...
Calling fib121...
Calling fib120...
Calling fib119...
Calling fib118...
Calling fib117...
Calling fib116...
Calling fib115...
Calling fib114...
Calling fib113...
Calling fib112...
Calling fib111...
Calling fib110...
```

```
Calling fib109...
Calling fib108...
Calling fib107...
Calling fib106...
Calling fib105...
Calling fib104...
Calling fib103...
Calling fib102...
Calling fib101...
Calling fib100...
Calling fib99...
Calling fib98...
Calling fib97...
Calling fib96...
Calling fib95...
Calling fib94...
Calling fib93...
Calling fib92...
Calling fib91...
Calling fib90...
Calling fib89...
Calling fib88...
Calling fib87...
Calling fib86...
Calling fib85...
Calling fib84...
Calling fib83...
Calling fib82...
Calling fib81...
Calling fib80...
Calling fib79...
Calling fib78...
Calling fib77...
Calling fib76...
Calling fib75...
Calling fib74...
Calling fib73...
Calling fib72...
Calling fib71...
Calling fib70...
Calling fib69...
Calling fib68...
Calling fib67...
Calling fib66...
Calling fib65...
Calling fib64...
Calling fib63...
Calling fib62...
Calling fib61...
Calling fib60...
Calling fib59...
Calling fib58...
Calling fib57...
Calling fib56...
Calling fib55...
Calling fib54...
Calling fib53...
Calling fib52...
Calling fib51...
Calling fib50...
Calling fib49...
```

```
Calling fib48...
Calling fib47...
Calling fib46...
Calling fib45...
Calling fib44...
Calling fib43...
Calling fib42...
Calling fib41...
Calling fib40...
Calling fib39...
Calling fib38...
Calling fib37...
Calling fib36...
Calling fib35...
Calling fib34...
Calling fib33...
Calling fib32...
Calling fib31...
Calling fib30...
Calling fib29...
Calling fib28...
Calling fib27...
Calling fib26...
Calling fib25...
Calling fib24...
Calling fib23...
Calling fib22...
Calling fib21...
Calling fib20...
Calling fib19...
Calling fib18...
Calling fib17...
Calling fib16...
Calling fib15...
Calling fib14...
Calling fib13...
Calling fib12...
Calling fib11...
Calling fib10...
Calling fib9...
Calling fib8...
Calling fib7...
Calling fib6...
Calling fib5...
Calling fib4...
Calling fib3...
Calling fib2...
Calling fib1...
Calling fib0...
422469633339230487870672560234148278257985284025068109801028013731430858437013070722412359963914151108844608753890960360764019471164359602927198331259873732625355580260699158591522949245390499872225679531698287448247299226390183371677806060701161549788671987985831146887087626459736908672288402365442229524334796448013951534956297208765265606952980649984197744872015561280266540455417171788193032402520431208251681712 5
```

In [370]:

```
1  def myTimer(f):
2
3          def callF(*args):
4                  start = time.perf_counter()
5                  result = f(*args)
```

```
                end = time.perf_counter()
                print(toEnggNotation(end - start))
                return result

        return callF


# timedPower = myTimer(math.pow)
# print(timedPower(5, 188))
#
#
@myTimer
def myPower(a, b):
        return a ** b


print(myPower(5, 188))
```

```
2.5650µ 2.5650000452515087e-06
254894705781192364324620863642833888945767288308190057718217365417860642420893057508943799552292297
505573515081778168678283691406 25
```

In [371]:

```
PRECISION = 4
```

In [372]:

```
a = decimal.Decimal(math.e)
print(a)
print(type(a))
print(float(a))
```

```
2.71828182845904509079559829842764884233474731445312 5
<class 'decimal.Decimal'>
2.718281828459045
```

In [373]:

```
a = decimal.Decimal("2.4040403138938243888847472828")
print(a)
print(type(a))
print(float(a))
```

```
2.4040403138938243888847472828
<class 'decimal.Decimal'>
2.4040403138938244
```

In [374]:

```
a = decimal.Decimal(2.23848929485929292275758693)
print(a)
print(type(a))
print(float(a))
```

```
2.2384892948592929329265643900725990533828735351562 5
<class 'decimal.Decimal'>
2.238489294859293
```

In [375]:

```
l = [decimal.Decimal(random.random()) for _ in range(10)]
a = sum(l) / len(l)
```

```
print(a)
print(type(a))
```

```
0.635507006839796340091908177 7
<class 'decimal.Decimal'>
```

In [376]:

```
def normalize(f):

        def callF(*args):
                result = f(*args)
                # print(type(result))
                result = float(result)
                return float(f"{result:.{PRECISION}f}")

        return callF
```

In [377]:

```
@normalize
def perc_diff(x, y):
        try:
                return (y - x) / x * 100
        except ZeroDivisionError:
                return 0


@normalize
def sum_squares(*args):
        return sum(x ** 2 for x in args)


@normalize
def avg(*args):
        if len(args) == 0:
                return 0

        numbers = [decimal.Decimal(x) for x in args]
        sum_ = sum(numbers)
        return sum_ / len(numbers)
```

In [378]:

```
perc_diff(5088, 2872)
```

Out[378]: -43.5535

In [379]:

```
avg(2.4, 4.1, 5.932929, 8.2838485)
```

Out[379]: 5.1792

In [380]:

```
def add(x, y):
        time.sleep(2)
        return x + y

```

```
print(timeit.timeit("add(234032083258038504384380 5805,
320342408320482804320584305)", globals = globals(), number = 10))
```

```
20.000953659999993
```

In [381]:

```
@functools.lru_cache
def add(x, y):
        time.sleep(2)
        return x + y


print(timeit.timeit("add(234032083258038504384380 5805,
320342408320482804320584305)", globals = globals(), number = 10))
```

```
2.0001189919999547
```

In [382]:

```
def add(x, y):
        time.sleep(2)
        return x + y


print(timeit.repeat("add(234032083258038504384380 5805,
320342408320482804320584305)", globals = globals(), number = 10))
```

```
[20.000912009000047, 20.000890174000006, 20.000836774999982, 20.00079347799999, 20.000815604000024]
```

In [383]:

```
@functools.lru_cache
def add(x, y):
        time.sleep(2)
        return x + y


print(timeit.repeat("add(234032083258038504384380 5805,
320342408320482804320584305)", globals = globals(), number = 10))
```

```
[2.000094929999989, 1.943999961895315e-06, 1.5530000041508174e-06, 1.4529999816659256e-06,
1.4830000054644188e-06]
```

In [384]:

```
def graham(x, y, z):
        return x ** y ** z


print(toEnggNotation(timeit.timeit("graham(4, 5, 6)", globals = globals(),
number = 10)))
```

```
686.5070µ       0.0006865070000117157
```

In [385]:

```
@functools.lru_cache
def graham(x, y, z):
        return x ** y ** z


print(toEnggNotation(timeit.timeit("graham(2, 3, 4)", globals = globals(),
number = 10)))
```

```
3.9570µ 3.9569999898958486e-06
```

In [386]:
```
print(toEnggNotation(timeit.timeit("2 ** 3 ** 4", number = 100)))
```

```
18.8660µ        1.8866000004891248e-05
```

In [387]:
```
def normalize(precision):

        def callF(f):

                def callFInner(*args):
                        result = f(*args)
                        result = f"{result:.{precision}f}"
                        return result

                return callFInner

        return callF
```

In [388]:
```
@normalize(2)
def perc_diff(x, y):
        try:
                return (y - x) / x * 100
        except ZeroDivisionError:
                return 0


@normalize(4)
def sum_squares(*args):
        return sum(x ** 2 for x in args)


@normalize(8)
def avg(*args):
        if len(args) == 0:
                return 0

        numbers = [decimal.Decimal(x) for x in args]
        sum_ = sum(numbers)
        return sum_ / len(numbers)
```

In [389]:
```
print(perc_diff(3, 7))
print(sum_squares(3, 4, 5))
print(avg(1.25, 1.75, 1.5, 1.0, 2.0, 2.5, 2.25, 2.75))
```

```
133.33
50.0000
1.87500000
```

In [390]:
```
def normalize(f):

        def callF(*args):
```

```
                result = f(*args)
                return result

        return callF
```

In [391]:

```
try:

        @normalize(2)
        def perc_diff(x, y):
                try:
                        return (y - x) / x * 100
                except ZeroDivisionError:
                        return 0

        @normalize(4)
        def sum_squares(*args):
                return sum(x ** 2 for x in args)

        @normalize(8)
        def avg(*args):
                if len(args) == 0:
                        return 0

                numbers = [decimal.Decimal(x) for x in args]
                sum_ = sum(numbers)
                return sum_ / len(numbers)

except TypeError as ex:
        print(type(ex).         + ": " + ex.args[0] + ".")
```

```
TypeError: 'int' object is not callable.
```

In [392]:

```
@functools.lru_cache
def getFibonacci(i):
        if i == 0:
                return 1
        if i == 1:
                return 1
        if i < 0:
                return 0
        else:
                return getFibonacci(i - 1) + getFibonacci(i - 2)


print(getFibonacci(60))

with open("../var/out000.txt", "w") as fl:
        fl.write("i,fib,fact,gcd\n")
        for i in range(101):
                fl.write(f"{i},{math.factorial(i)},{getFibonacci(i)},
{math.gcd(getFibonacci(i),math.factorial(i))}\n")

```

```
with open("../var/out000.txt") as fl:
        for line in fl:
                print(line.strip())
```

```
2504730781961
i,fib,fact,gcd
0,1,1,1
1,1,1,1
2,2,2,2
3,6,3,3
4,24,5,1
5,120,8,8
6,720,13,1
7,5040,21,21
8,40320,34,2
9,362880,55,5
10,3628800,89,1
11,39916800,144,144
12,479001600,233,1
13,6227020800,377,13
14,87178291200,610,10
15,1307674368000,987,21
16,20922789888000,1597,1
17,355687428096000,2584,136
18,6402373705728000,4181,1
19,121645100408832000,6765,165
20,2432902008176640000,10946,26
21,51090942171709440000,17711,1
22,1124000727777607680000,28657,1
23,25852016738884976640000,46368,46368
24,620448401733239439360000,75025,25
25,15511210043330985984000000,121393,1
26,403291461126605635584000000,196418,34
27,10888869450418352160768000000,317811,39
28,304888344611713860501504000000,514229,1
29,8841761993739701954543616000000,832040,440
30,265252859812191058636308480000000,1346269,1
31,8222838654177922817725562880000000,2178309,21
32,263130836933693530167218012160000000,3524578,2
33,8683317618811886495518194401280000000,5702887,1
34,295232799039604140847618609643520000000,9227465,65
35,10333147966386144929666651337523200000000,14930352,139536
36,3719933267899012174679994481508352000000000,24157817,1
37,137637530912263450463159795815809024000000000,39088169,37
38,5230226174666011117600072241000742912000000000,63245986,2
39,203978820811974433586402817399028973568000000000,102334155,1155
40,8159152832478977343456112695961158942720000000000,165580141,1
41,33452526613163807108170062053440751665152000000000,267914296,3016
42,1405006117752879898543142606244511569936384000000000,433494437,1
43,604152630633738356373551320685139975072645120000000000,701408733,129
44,26582715747884487680436258110146158903196385280000000000,1134903170,170
45,1196222208654801945619631614956577150643837337600000000000,1836311903,1
46,55026221598120889498503054288002548929616517529600000000000,2971215073,1
47,2586232415111681806429643551536119799691976323891200000000000,4807526976,4358592
48,124139155925360726708622890473733750385214863546777600000000000,7778742049,13
49,6082818640342675608722521633212953768875528313792102400000000000,12586269025,275
50,304140932017133780436126081660647688443776415689605120000000000000,20365011074,2
51,1551118753287382280224243016469303211063259720016986112000000000000,32951280099,3
52,80658175170943878571660636856403766975289505440883277824000000000000,53316291173,1
53,427488328406002556429801375338939964969034378836681372467200000000000000,86267571272,136952
54,2308436973392413804720927426830275810832785645718079411322880000000000000,139583862445,5
55,12696403353658275925965100847566516959580321051449436762275840000000000000000,225851433717,55419
56,7109985878048634518540456474637249497364979788811684586874470400000000000000,365435296162,74
57,40526919504877216755680601905432322134980384796226602145184481280000000000000,591286729879,1
58,2350561331282878571829474910515074683828862318181142924420699914240000000000000,956722026041,1
```

59,13868311854568983573793901972038940634590287677268743254082129494016000000000000000,1548008755920,10066320
60,832098711274139014427634118322336438075417260636124595244927769640960000000000000000,2504730781961,1
61,5075802138772247988008568121766252272260045289880360030994059394809856000000000000000,4052739537881,1
62,3146997326038793752565312235495076408801228079725823192163168247821107200000000000000,6557470319842,442
63,19826083154044400641161467083618981375447736902272686281062795996127297536000000000000000,10610209857723,987
64,12688693218588416410343338933516148080286551617454519219880189437521470423040000000000000000,17167680177565,5
65,824765059208247066672317030678549625218625855134543749292212313438895577497600000000000000000,27777890035288,8
66,5443449390774430640037292402478427526442930643887988745328601268696710811484160000000000000000,44945570212853,1
67,364711109181886852882498590966054644271676353140495245937016285002679624369438720000000000000000,727234602481 41,201
68,248003554243683059960099041856917158104739920135536767237171073801822144571218329600000000000000000,117669030460994,2
69,171122452428141311372468338881272839092270544893520369393648040923257279754140647424000000000000000000,190392490709135,20735
70,119785716699698917960727837216890987364589381425464258575553628646280095827898453196800000000000000000,308061521170129,1
71,850478588567862317521167644239926010288584608120796235886430763388588680378079017697280000000000000000,498454011879264,44930592
72,61234458376886086861524070385274672740778091784697328983823014963978384987221689274204160000000000000000,806515533049393,1
73,44701154615126843408912571381250511100768007002829050158190800923704221040671833170169036800000000000000000,1304969544928657,73
74,33078854415193864122595302822125378214568325182093497117061192683541123570097156545925087232000000000000000,2111485077978050,3050
75,24809140811395398091946477116594033660926243886570122837795894512655842677572867409443815424000000000000000000,3416454622906707,111
76,18854947016660502549879322608611465582303945353793293356724879829618440434955379231177299722240000000000000000,5527939700884757,13
77,14518309202828586963407078408630828498374037922420835884678157468806199134915642008006520786124800000000000000000,8944394323791464,8
78,113242811782062978314575211587320462287317495794882519900489628256688353252342007662450862131773440000000000000000000,14472334024676221,1
79,894618213078297528685144171539831652069808216779571907213868063227837990693501860533361810841010176000000000000000000,23416728348467685,2225685
80,715694570462638022948115337231865321655846573423657525771094450582270392554801488426689448672808140800000000000000000000,37889062373143906,1802
81,579712602074736798587973423157810910541235724473162595874586504971639017969389205625618453424974594048000000000000000000000,61305790721611591,1
82,475364333701284174842138206989404946643813294067993328617160934076743994734899148613007131808479167119360000000000000000000000,99194853094755497,1
83,394552396972065865118974711801206105714365034076434462752243575283697515629966293348795919401037708709068800000000000000000000000,160500643816367088,4505904
84,331424013456535326699938757913013128800066628624204948711884603238305913129171686412988572296871675315617792000000000000000000000,259695496911122585,5
85,2817104114380550276949479442260611594800566343305742064051019127525600261597959334510402864523409240182751232000000000000000000000,420196140727489673,1
86,2422709538367273238176552320344125971528487055242938175083876449672016224974245027678946463490131946557166059520000000000000000000000,679891637638612258,2
87,2107757298379527717213600518699389595229783738061356212322972511214654115727593174080683423236414793504734471782400000000000000000000000,1100087778366101931,903
88,1854826422573984391147968456455462843802209689493993466844215809868895621840281993191001412448045018284166335168512000000000000000000000,1779979416004714189,1
89,16507955160908461081216919262453619309839666236496541854913520707833171034378509739399912570787 60

```
0662729080382999756800000000000000000000,2880067194370816120,268748920
90,1485715964481761497309522733620825737885569961284688766942216863704985393094065876545992131370884
0596456172344699781120000000000000000000000,4660046610375530309,169
91,1352001527678402962551665687594951421475868664769066777917417345971536707715599947656852839547504
4942775116833676800819200000000000000000000000,7540113804746346429,3
92,1243841405464130725547532432587355307757799171587541435684023958293813771098351951844304612383704
1347353107486982656753664000000000000000000000,1220016041512187673 8,2
93,1156772507081641574759205162306240436214753229576413535186142281213246807121467315215203289516844
8453038389962893870780907520000000000000000000000,19740274219868223167,1
94,1087366156656743080273652852567866010041868035801828723074973744340451998694179276302291092145834
1545856086565120238534053068800000000000000000000000,31940434634990099905,185
95,1032997848823905926259970209939472709539774634011737286921225057123429398759470312487176537538542
4468563282236864226607350415360000000000000000000000000,51680708854858323072,8717184
96,9916779348709496892095714015418938011581836486512677954443760548384922228090914999876894760370007
4898207509473896575430563987456000000000000000000000000,83621143489848422977,1
97,9619275968248211985332842594956369871234381391917297615810447731933374561248187549880587917558907
2651261284189679678167647067832320000000000000000000000000,135301852344706746049,36569
98,9426890448883247745626185743057242473809693764078951663494238777294707070023223798882976159207729
1198236058505886084604294126475673600000000000000000000000000,21892299583455516902 6,3026
99,9332621544394415268169923885626670049071596826438162146859296389521759999322991560894146397615651
8286253697920827223758251185210916864000000000000000000000000,3542248481792619150 75,33825
100,9332621544394415268169923885626670049071596826438162146859296389521759999322991560894146397615651
8286253697920827223758251185210916864000000000000000000000000,5731478440138170841 01,1
```

In [393]:

```
fibonacciNumbers = {}
factorialNumbers = {}
gcd = {}

with open("../var/out000.txt") as fl:
        next(fl)
        for ln in fl:
                l = ln.strip().split(",")
                factorialNumbers[int(l[0])] = int(l[1])
                fibonacciNumbers[int(l[0])] = int(l[2])
                gcd[(int(l[1]), int(l[2]))] = int(l[3])


# @functools.lru_cache
def getFibonacci(i):
        print(f"getFibonacci({i}):")
        if i < 0:
                return 0
        if i in fibonacciNumbers:
                print("Found in cache!")
                return fibonacciNumbers[i]
        print("Not found in cache...")
        fibonacciNumbers[i] = getFibonacci(i - 1) + getFibonacci(i - 2)
        return fibonacciNumbers[i]


print(getFibonacci(110))
print(getFibonacci(11))


def f(i):
```

```
        def g():
                return math.factorial(i)

        return g


def getFactorial(i):
        print(f"getFactorial({i}):")
        if i in factorialNumbers:
                print("Found in cache!")
                return factorialNumbers[i]
        print("Not found in cache...")
        return f(i)()
        # return factorialNumbers.get(i, f(i)())        # Doesn't work because
the closure is always called.  Have to use condition.


print(getFactorial(110))
print(getFactorial(11))


def f1(i, j):

        def g1():
                return math.gcd(i, j)

        return g1


def getGCD(i, j):
        print(f"getGCD({i, j}):")
        if (i, j) in gcd:
                print("Found in cache!")
                return gcd[(i, j)]
        print("Not found in cache...")
        return math.gcd(i, j)


print(getGCD(5040, 21))
print(getGCD(5040, 22))
```

```
8
 6
9
 7
0
 7
1
```

```
getFibonacci(110):
Not found in cache...
getFibonacci(109):
Not found in cache...
getFibonacci(108):
Not found in cache...
getFibonacci(107):
Not found in cache...
getFibonacci(106):
Not found in cache...
getFibonacci(105):
Not found in cache...
getFibonacci(104):
Not found in cache...
getFibonacci(103):
Not found in cache...
getFibonacci(102):
Not found in cache...
getFibonacci(101):
Not found in cache...
getFibonacci(100):
Found in cache!
getFibonacci(99):
Found in cache!
getFibonacci(100):
Found in cache!
getFibonacci(101):
Found in cache!
getFibonacci(102):
Found in cache!
getFibonacci(103):
Found in cache!
getFibonacci(104):
Found in cache!
getFibonacci(105):
Found in cache!
getFibonacci(106):
Found in cache!
getFibonacci(107):
Found in cache!
getFibonacci(108):
Found in cache!
70492524767089125814114
getFibonacci(11):
Found in cache!
144
getFactorial(110):
Not found in cache...
15882455415227429404253703127090772871724410234473563207581748318444567162948183030959960131517678520479243672638179990208521148623422266876757623911219200000000000000000000000000
getFactorial(11):
Found in cache!
39916800
getGCD((5040, 21)):
Found in cache!
21
getGCD((5040, 22)):
Not found in cache...
2
```

In [394]:
```
def importMPL():
        import matplotlib as mpl

        return mpl


mpl = importMPL()
print(dir(mpl))
```

```
['ExecutableNotFoundError', 'MatplotlibDeprecationWarning', 'MutableMapping', 'Parameter', 'Path',
'RcParams', '_ExecInfo', '_VersionInfo', '__all__', '__bibtex__', '__builtins__', '__cached__',
'__doc__', '__file__', '__getattr__', '__loader__', '__name__', '__package__', '__path__',
'__spec__', '_add_data_doc', '_api', '_c_internal_utils', '_check_versions', '_cm', '_cm_bivar',
'_cm_listed', '_cm_multivar', '_color_data', '_deprecated_ignore_map', '_deprecated_map',
'_deprecated_remain_as_none', '_docstring', '_ensure_handler', '_enums', '_fontconfig_pattern',
'_get_config_or_cache_dir', '_get_executable_info', '_get_ssl_context', '_get_version',
'_get_xdg_cache_dir', '_get_xdg_config_dir', '_image', '_init_tests', '_label_from_arg', '_log',
'_logged_cached', '_open_file_or_url', '_parse_to_version_info', '_path', '_preprocess_data',
'_rc_params_in_file', '_replacer', '_val_or_rc', '_version', 'artist', 'atexit', 'backends',
'bezier', 'bivar_colormaps', 'cbook', 'cm', 'color_sequences', 'colorizer', 'colormaps', 'colors',
'contextlib', 'cycler', 'defaultParams', 'ft2font', 'functools', 'get_backend', 'get_cachedir',
'get_configdir', 'get_data_path', 'importlib', 'inspect', 'interactive', 'is_interactive', 'locale',
'logging', 'matplotlib_fname', 'multivar_colormaps', 'namedtuple', 'os', 'parse_version', 'path',
'pprint', 'rc', 'rcParams', 'rcParamsDefault', 'rcParamsOrig', 'rc_context', 'rc_file',
'rc_file_defaults', 'rc_params', 'rc_params_from_file', 'rcdefaults', 'rcsetup', 're',
'sanitize_sequence', 'scale', 'set_loglevel', 'shutil', 'subprocess', 'sys', 'tempfile', 'ticker',
'transforms', 'use', 'validate_backend']
```

In [395]:
```
epochTime = time.gmtime(0)
print(epochTime.tm_year, epochTime.tm_mday, epochTime.tm_hour,
epochTime.tm_min, epochTime.tm_zone)
```

```
1970 1 0 0 GMT
```

In [396]:
```
time.perf_counter()
```

Out[396]: 440.340111055

In [397]:
```
start = time.perf_counter()
time.sleep(3)
end = time.perf_counter()
print(end - start)
```

```
3.0002507969999783
```

In [398]:
```
print(time.strftime("%I:%M %p %z", time.gmtime(time.time())))
```

```
12:00 PM +0000
```

In [ ]:
```
print(datetime.datetime.now(datetime.UTC))
print(datetime.datetime.now())
```

```
2026-09-18 12:00:08.720941+00:00
2026-09-18 17:30:08.721215
```

In [400]:
```
print(time.strftime("%Y-%m-%dT%H:%M:%S%z", time.gmtime(time.time())))
```

```
2026-09-18T12:00:08+0000
```

In [401]:
```
my_time_struct = time.strptime("2020-09-17T16:36:16Z", "%Y-%m-%dT%H:%M:%S%z")
print(my_time_struct.tm_year, my_time_struct.tm_mon, my_time_struct.tm_mday,
my_time_struct.tm_hour, my_time_struct.tm_min, my_time_struct.tm_sec,
my_time_struct.tm_gmtoff)
```

```
2020 9 17 16 36 16 0
```

In [402]:
```
my_time_struct = time.strptime("2020-09-17T16:36:16+0530", "%Y-%m-
%dT%H:%M:%S%z")
print(my_time_struct.tm_year, my_time_struct.tm_mon, my_time_struct.tm_mday,
my_time_struct.tm_hour, my_time_struct.tm_min, my_time_struct.tm_sec,
my_time_struct.tm_gmtoff)
```

```
2020 9 17 16 36 16 19800
```

In [403]:
```
print("hello")
```

```
hello
```

In [404]:
```
s = "Monday, April 18, 2020"
myTimeStruct = time.strptime(s, "%A, %B %d, %Y")
print(myTimeStruct)
```

```
time.struct_time(tm_year=2020, tm_mon=4, tm_mday=18, tm_hour=0, tm_min=0, tm_sec=0, tm_wday=0,
tm_yday=109, tm_isdst=-1)
```

In [420]:
```
myDate = datetime.date(2020, 8, 20)
print(myDate)
print(repr(myDate))
print(myDate.day, myDate.month)
```

```
2020-08-20
datetime.date(2020, 8, 20)
20 8
```

In [419]:
```
print(myDate.isoformat())
```

```
2020-08-20
```

In [421]:
```
myDate = datetime.date.today()
print(myDate)
print(repr(myDate))
print(myDate.day, myDate.month)
```

```
2026-09-18
datetime.date(2026, 9, 18)
18 9
```

In [ ]:
```
print(myDate.isoformat())
```

```
2026-09-18
18 9
```

In [407]:
```
myDate = datetime.date.fromtimestamp(time.time())
print(myDate.year)
```

```
2026
```

In [408]:
```
myDate = datetime.date.fromtimestamp(0)
print(myDate.year)
```

```
1970
```

In [ ]:
```
myDate = datetime.date.fromisoformat("2020-08-24")
print(myDate)
print(repr(myDate))
```

```
2020-08-24
datetime.date(2020, 8, 24)
```

In [410]:
```
myTime = datetime.time(15, 23, 45)
print(myTime.isoformat())
```

```
15:23:45
```

In [411]:
```
print(datetime.time(15, 30, 45, 600))
```

```
15:30:45.000600
```

In [424]:
```
try:
        myTime = datetime.time.now()
        print(myTime)
except AttributeError as ex:
        print(type(ex).        , ex.args)
```

```
AttributeError ("type object 'datetime.time' has no attribute 'now'",)
```

In [412]:
```
print(datetime.time.fromisoformat("12:34:45.123"))
```

```
12:34:45.123000
```

In [413]:
```
myTime = datetime.time.fromisoformat("12:34:45.1234")
print(myTime)
print(myTime.hour)
print(myTime.microsecond)
try:
        print(myTime[:3])
except TypeError as ex:
        print(type(ex).        , ex.args)
```

```
12:34:45.123400
12
123400
TypeError ("'datetime.time' object is not subscriptable",)
```

In [423]:
```
try:
        myTime = datetime.time.fromtimestamp(0)
        print(myTime)
```

```
except AttributeError as ex:
        print(type(ex).        , ex.args)
```

```
AttributeError ("type object 'datetime.time' has no attribute 'fromtimestamp'",)
```

In [414]:
```
myDateTime = datetime.datetime(2020, 12, 9, 12, 43, 32, 38829)
print(myDateTime.isoformat())
```

```
2020-12-09T12:43:32.038829
```

In [426]:
```
myDateTime = datetime.datetime.now()
print(myDateTime)
print(repr(myDateTime))
print(myDateTime.hour)
print(myDateTime.isoformat())
```

```
2026-09-18 18:03:08.224995
datetime.datetime(2026, 9, 18, 18, 3, 8, 224995)
18
2026-09-18T18:03:08.224995
```

In [429]:
```
myDateTime = datetime.datetime.fromtimestamp(0)
print(myDateTime)
print(myDateTime.hour)
print(myDateTime.isoformat())
```

```
1970-01-01 05:30:00
5
1970-01-01T05:30:00
```

In [436]:
```
myDateTime = datetime.datetime.fromtimestamp(time.time())
print(myDateTime)
print(myDateTime.hour)
print(myDateTime.isoformat())
```

```
2026-09-18 18:11:43.908956
18
2026-09-18T18:11:43.908956
```

In [431]:
```
myDateTime = datetime.datetime.fromisoformat("2025-02-02T15:23:44+0545")
print(myDateTime)
print(myDateTime.hour)
print(myDateTime.isoformat())
```

```
2025-02-02 15:23:44+05:45
15
2025-02-02T15:23:44+05:45
```

In [432]:
```
myDateTime = datetime.datetime.fromisoformat("2025-02-02T15:23:44")
print(myDateTime)
print(myDateTime.hour)
print(myDateTime.isoformat())
```

```
2025-02-02 15:23:44
15
2025-02-02T15:23:44
```

In [433]:
```
myDateTime = datetime.datetime.fromisoformat("2025-02-02 15:23:44")
print(myDateTime)
print(myDateTime.hour)
print(myDateTime.isoformat())
```

```
2025-02-02 15:23:44
15
2025-02-02T15:23:44
```

In [434]:
```
myDateTime = datetime.datetime.fromisoformat("2025-02-02 15:23")
print(myDateTime)
print(myDateTime.hour)
print(myDateTime.isoformat())
```

```
2025-02-02 15:23:00
15
2025-02-02T15:23:00
```

In [435]:
```
myDateTime = datetime.datetime.fromisoformat("2025-02-02")
print(myDateTime)
print(myDateTime.hour)
print(myDateTime.isoformat())
```

```
2025-02-02 00:00:00
0
2025-02-02T00:00:00
```

In [438]:
```
myDateTime = datetime.datetime.utcnow()
print(myDateTime)
print(myDateTime.hour)
print(myDateTime.tzinfo)
print(myDateTime.isoformat())
```

```
2026-09-18 12:54:43.306385
12
None
2026-09-18T12:54:43.306385
/tmp/ipykernel_18021/573860576.py:1: DeprecationWarning: datetime.datetime.utcnow() is deprecated and scheduled for removal in a future version. Use timezone-aware objects to represent datetimes in UTC: datetime.datetime.now(datetime.UTC).
  myDateTime = datetime.datetime.utcnow()
```

In [440]:
```
myDateTime = datetime.datetime.now(datetime.UTC)
print(myDateTime)
print(myDateTime.hour)
print(myDateTime.tzinfo)
print(myDateTime.isoformat())
```

```
2026-09-18 12:55:53.446540+00:00
12
UTC
2026-09-18T12:55:53.446540+00:00
```

In [442]:
```
try:
        myDateTime = datetime.datetime.now(datetime.IST)
        print(myDateTime)
```

```
        print(myDateTime.hour)
        print(myDateTime.tzinfo)
        print(myDateTime.isoformat())
except AttributeError as ex:
        print(type(ex).        , ex.args)
```

```
AttributeError ("module 'datetime' has no attribute 'IST'",)
```

In [415]:

```
print(cmath.sqrt(-4))
```

```
2j
```

In [451]:

```
dt1 = datetime.datetime.now()
print(dt1)
dt2 = datetime.datetime.today()
print(dt2)
td = dt1 - dt2
print(toEnggNotation(abs(td.total_seconds())))
```

```
2026-09-18 18:53:23.132215
2026-09-18 18:53:23.132384
169.0000µ       0.000169
```

In [459]:

```
dt1 = datetime.datetime.now()
print(dt1)
startOfDay = datetime.datetime.fromisoformat(datetime.date.today().isoformat())
print(startOfDay)
td = dt1 - startOfDay
print(td.total_seconds())
```

```
2026-09-18 18:57:44.890444
2026-09-18 00:00:00
68264.890444
```

In [461]:

```
td = datetime.timedelta(weeks = 52, seconds = 86400)
print(datetime.datetime.now() + td)
```

```
2027-09-18 19:01:37.580957
```

In [467]:

```
print(time.strftime("%FT%T", time.strptime("1/6/2024 2:30 PM", "%d/%m/%Y %I:%M %p")))
```

```
2024-06-01T14:30:00
```

In [472]:

```
td = datetime.timedelta(days = -1, seconds = 5400)
print(td + datetime.datetime.now())
print(td.total_seconds() // 60 / 60)
```

```
2026-09-18 18:41:55.925042
-22.5
```

In [480]:

```
myTZ = datetime.timezone(datetime.timedelta(hours = 5.5), name = "IST")
print(myTZ)
UTCTZ = datetime.timezone.utc
print(UTCTZ)
```

```
IST
UTC
```

In [489]:
```
print(datetime.datetime.now().replace(tzinfo =
datetime.timezone(datetime.timedelta(hours = 5.5), name =
"IST")).astimezone(datetime.timezone(datetime.timedelta(hours =
9))).isoformat())
```

```
2026-09-19T21:28:11.050467+09:00
```

In [507]:
```
currentTime = datetime.datetime.now()
tzIST = datetime.timezone(datetime.timedelta(hours = 5.5))
currentISTTime = currentTime.replace(tzinfo = tzIST)
tzJST = datetime.timezone(datetime.timedelta(hours = 9))
currentJSTTime = currentISTTime.astimezone(tzJST)
print(currentJSTTime)
currentJSTTime = currentJSTTime.replace(tzinfo = None)
tzDifference = currentJSTTime - datetime.datetime.now()
print(tzDifference)
print(type(tzDifference).         )
print(round(tzDifference.total_seconds() // 60 / 60, 1))
```

```
2026-09-19 21:49:02.698258+09:00
3:29:59.999810
timedelta
3.5
```

In [514]:
```
epochTime = 12345678.9
dt = datetime.datetime.fromtimestamp(epochTime).replace(day = 1, hour = 0,
minute = 0, second = 0, microsecond = 0)
print(repr(dt))
```

```
datetime.datetime(1970, 5, 1, 0, 0)
```

In [ ]:
```
epochTime = 12345678.9
dt = datetime.datetime.fromtimestamp(epochTime)
print(repr(datetime.datetime(year = dt.year, month = dt.month, day = 1)))
```

```
datetime.datetime(2026, 9, 1, 0, 0)
```

In [ ]:
```
epochTime = time.time()

dt = datetime.datetime.fromtimestamp(epochTime)
print(repr(datetime.datetime(year = dt.year, month = dt.month, day = 1)))
```

```
datetime.datetime(2026, 9, 1, 0, 0)
```

In [520]:
```
start = "2001-01-01T13:50:23"
end = "2001-06-12T14:23:50"
diff = int(round((datetime.datetime.fromisoformat(end) -
datetime.datetime.fromisoformat(start)).total_seconds() // 60 / 60, 0))
print(diff)
```

```
3889
```

In [544]:
```
start = "2001-01-01T00:00:00"
end = "2001-01-01T01:00:00"
diff = int(round((datetime.datetime.fromisoformat(end) -
datetime.datetime.fromisoformat(start)).total_seconds() // 60 / 60, 0))
print(diff)
```

```
1
```

In [545]:
```
start = "2001-01-01T00:00:00"
end = "2001-01-02T01:50:00"
diff = int(round((datetime.datetime.fromisoformat(end) -
datetime.datetime.fromisoformat(start)).total_seconds() // 60 / 60, 0))
print(diff)
```

```
26
```

In [534]:
```
def formatDateTime(dateTimeObj, formatStr = "%FT%T"):
        return dateTimeObj.strftime(formatStr)



dateTimeObj = datetime.datetime.strptime("02/04/2025 03:30:30 PM", "%m/%d/%Y
%I:%M:%S %p")
print(formatDateTime(dateTimeObj))
print(formatDateTime(dateTimeObj, "%Y-%m-%d"))
```

```
2025-02-04T15:30:30
2025-02-04
```

In [546]:
```
def formatDateTime(dateTimeObj, formatStr = "%FT%T"):
        return dateTimeObj.strftime(formatStr)



dateTimeObj = datetime.datetime(2020, 2, 1, 13, 34, 5)
print(formatDateTime(dateTimeObj))
print(formatDateTime(dateTimeObj, "%Y-%m-%d"))
```

```
2020-02-01T13:34:05
2020-02-01
```

In [50]:
```
with open("../var/actors.csv") as fh:
        itr = csv.reader(fh)
        for row in itr:
                print(*row)
```

```
First Name Last Name DOB Sketches
John Cleese 10/27/39 The Cheese Shop, Ministry of Silly Walks, It's the Arts
Eric Idle 3/29/43 The Cheese Shop, Nudge Nudge, "Spam"
Peter O'Toole 8/2/32 Lawrence of Arabia
```

In [ ]:
```
with open("../var/actors.pdv") as fh:
        itr = csv.reader(fh, delimiter = "|", quotechar = "'", escapechar =
"\\", skipinitialspace = True)
        for row in itr:
                print(row)
```

```
['FIRST_NAME', 'LAST_NAME', 'DOB', 'SKETCHES']
['John', 'Cleese', '10/27/39', "The Cheese Shop, Ministry of Silly Walks, It's the Arts"]
['Eric', 'Idle', '3/29/43', 'The Cheese Shop, Nudge Nudge, "Spam"']
['Peter', "O'Toole", '8/2/32', 'Lawrence of Arabia']
```

In [14]:
```
csv.register_dialect("pdv", delimiter = "|", quotechar = "'", skipinitialspace
= True, escapechar = "\\")
csv.list_dialects()
```

Out[14]:
```
['excel', 'excel-tab', 'unix', 'mine', 'pdv']
```

In [563]:
```
with open("../var/actors.pdv") as fh:
        itr = csv.reader(fh, dialect = "pdv")
        for row in itr:
                print(row)
```

```
['FIRST_NAME', 'LAST_NAME', 'DOB', 'SKETCHES']
['John', 'Cleese', '10/27/39', "The Cheese Shop, Ministry of Silly Walks, It's the Arts"]
['Eric', 'Idle', '3/29/43', 'The Cheese Shop, Nudge Nudge, "Spam"']
['Peter', "O'Toole", '8/2/32', 'Lawrence of Arabia']
```

In [13]:
```
csv.register_dialect("mine", delimiter = "|", skipinitialspace = True,
escapechar = "\\", doublequote = False)
csv.list_dialects()
```

Out[13]:
```
['excel', 'excel-tab', 'unix', 'mine']
```

In [56]:
```
# csv.register_dialect("mine", delimiter = "|", skipinitialspace = True,
quoting = csv.QUOTE_NONE, escapechar = "\\", doublequote = False,
lineterminator = "\n")
csv.register_dialect("mine", delimiter = "|", skipinitialspace = True, quoting
= csv.QUOTE_NONE, lineterminator = "\n", escapechar = "\\", doublequote =
False, quotechar = None)
csv.list_dialects()
```

Out[56]:
```
['excel', 'excel-tab', 'unix', 'mine', 'pdv']
```

In [59]:

```
print("Current literal contents...")
with open("../var/actors.pdv") as fh1:
        for ln in fh1:
                print(ln, end = "")

print("\n\nAfter converting to my format...")
with open("../var/actors.pdv") as fh1:
        with open("../var/actors.tsv", "w") as fh2:
                itr1 = csv.reader(fh1, dialect = "pdv")
                itr2 = csv.writer(fh2, dialect = "mine")
                for row in itr1:
                        itr2.writerow(row)

with open("../var/actors.tsv") as fh:
        for row in fh:
                print(row, end = "")
```

```
Current literal contents...
FIRST_NAME| LAST_NAME| DOB| SKETCHES
John|Cleese| 10/27/39| 'The Cheese Shop, Ministry of Silly Walks, It\'s the Arts'
Eric| Idle| 3/29/43| 'The Cheese Shop, Nudge Nudge, "Spam"'
Peter| 'O\'Toole'| 8/2/32| Lawrence of Arabia

After converting to my format...
FIRST_NAME|LAST_NAME|DOB|SKETCHES
John|Cleese|10/27/39|The Cheese Shop, Ministry of Silly Walks, It's the Arts
Eric|Idle|3/29/43|The Cheese Shop, Nudge Nudge, "Spam"
Peter|O'Toole|8/2/32|Lawrence of Arabia
```

In [566]:

```
with open("../var/file1.csv") as fh:
        for ln in fh:
                print(ln, end = "")
```

```
col1,col2,col3
10,20,30
30,40,50
60,70,80
```

In [571]:

```
with open("../var/file1.csv") as fh:
        itr = csv.reader(fh)
        smallest, largest = math.inf, -math.inf
        for row in itr:
                for fld in row:
                        try:
                                n = int(fld)
                        except ValueError:
                                pass
                        if n < smallest:
                                smallest = n
                        if n > largest:
                                largest = n

print(smallest, largest)
```

```
10 80
```

In [572]:
```
with open("../var/file2.csv") as fh:
        for ln in fh:
                print(ln, end = "")
```

```
1, 3.14, 500
20, 1, -2
-1.1, -2.2, -3.3
```

In [588]:
```
print(type(2.0))
print(type(2.0) == float)
print(isinstance(2.0, float))
```

```
<class 'float'>
True
True
```

In [609]:
```
a = "500.0"
b = "500"
if "." in a:
        print("a is fractional")
if "." in b:
        print("b is fractional")
```

```
a is fractional
```

In [611]:
```
with open("../var/file2.csv") as fh:
        itr = csv.reader(fh, skipinitialspace = True)
        smallest, largest = math.inf, -math.inf
        for row in itr:
                for fld in row:
                        n = float(fld) if "." in fld else int(fld)
                        if n < smallest:
                                smallest = n
                        if n > largest:
                                largest = n

print(smallest, largest)
```

```
-3.3 500
```

In [614]:
```
with open("../var/file3.tsv") as fh:
        for ln in fh:
                print(ln, end = "")
```

```
col1    col2    col3
10      20      30
40      50      60
100     200     300
```

In [623]:
```
with open("../var/file3.tsv") as fh:
        itr = csv.reader(fh, skipinitialspace = True, delimiter = "\t")
        smallest, largest = math.inf, -math.inf
        next(fh)
```

```
        for row in itr:
                for fld in row:
                        n = int(fld)
                        if n < smallest:
                                smallest = n
                        if n > largest:
                                largest = n

print(smallest, largest)
```

```
10 300
```

In [627]:

```
with open("../var/file4.csv") as fh:
        for ln in fh:
                print(ln, end = "")
```

```
-col1-|-col2-|-col3-
10|20|30
-3.14-|-25-|-100-
---3.14-|20|-30-
```

In [633]:

```
a = "--3.3-"
print(a.removesuffix("-").removeprefix("-"))
```

```
-3.3
```

In [ ]:

```
print(a.replace("-", ""))
```

```
3.3
```

In [642]:

```
with open("../var/file4.csv") as fh:
        itr = csv.reader(fh, delimiter = "|")
        smallest, largest = math.inf, -math.inf
        next(fh)
        for row in itr:
                for fld in row:
                        fld = fld.replace("-", "")
                        n = float(fld) if "." in fld else int(fld)
                        if n < smallest:
                                smallest = n
                        if n > largest:
                                largest = n

print(smallest, largest)
```

```
3.14 100
```

In [648]:

```
a = "b"
print(ord(a) - ord("a"))
```

```
1
```

In [655]:

```
data = [{'a': '1_a', 'b': '1_b', 'c': '1_c'}, {'c': '2_c', 'd': '2_d'}, {'a':
'3_a', 'c': '3_c', 'e': '3_e'}]
with open("../var/out2.csv", "w") as fh:
```

```
        itr = csv.writer(fh)
        itr.writerow("abcde")
        for d in data:
                row = ["", "", "", "", ""]
                for k, v in d.items():
                        row[ord(k) - ord("a")] = v
                itr.writerow(row)

with open("../var/out2.csv") as fh:
        for ln in fh:
                print(ln, end = "")
```

```
a,b,c,d,e
1_a,1_b,1_c,,
,,2_c,2_d,
3_a,,3_c,,3_e
```

In [8]:

```
for _ in range(10):
        print(random.random(), end = " ")

randList = [random.random() for _ in range(5000000)]
print("Min:", toEnggNotation(min(randList)))
print("Max:", toEnggNotation(max(randList)))
minima = min(randList)
print(f"{minima:.200f}".rstrip("0"))
dec = decimal.Decimal(minima)
print(dec)
print(dec.normalize())
recurring = 1 / 3
print(decimal.Decimal(recurring))
print(f"{recurring:.200f}")
```

```
0.3760162724736019 0.35412034599545605 0.25655884480246993 0.008207749667868414 0.3570890359413942
0.48723070742025765 0.219258165097041 0.3653899598022412 0.08372909345952739 0.36041332341489574
Min: 23.9507n   2.3950684879814332e-08
Max: 999.9999m  0.9999998702222609
0.00000002395068487981433236200246028602123260498046875
2.3950684879814332362002460286021232604980468 75E-8
2.395068487981433236200246029E-8
0.33333333333333331482961625624739099293947219848632812
0.33333333333333331482961625624739099293947219848632812500000000000000000000000000000000000000000000000000
00000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000
00
```

In [9]:
```
1  print(randList[:5])
```

```
[0.08172163591676762, 0.6379410884560406, 0.36610121100531756, 0.47189283950477456,
0.7181724643263364]
```

In [ ]:
```
1  recurring = 1 / 10
2  print(decimal.Decimal(recurring))
3  print(f"{recurring:.200f}")
```

```
0.10000000000000000555111512312578270211815834045410156 25
0.10000000000000000555111512312578270211815834045410156250000000000000000000000000000000000000000000000000
00000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000
00
```

In [89]:
```
1  recurring = 2000 / 10000
2  print(f"{recurring:.200f}")
```

```
0.20000000000000001110223024625156540423631668090820312500000000000000000000000000000000000000000000000000
00000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000
00
```

In [14]:
```
1  print(f"{random.random():f}")
2  print(random.random())
3  try:
4          print(f"{random.random():b}")
5  except ValueError as ex:
6          print(type(ex).         + ": " + ex.args[0])
7  print(f"{123:b}")
8  print(sys.float_info)
9  try:
10         print(f"{0.1:b}")
11 except ValueError as ex:
12         print(type(ex).         + ": " + ex.args[0])
13 print(int("0b1001", 2))
14 print(int("1001", 2))
15 try:
16         print(int(1001, 2))
17 except TypeError as ex:
18         print(type(ex).         + ": " + ex.args[0])
```

```
0.590234
0.5573487087000488
ValueError: Unknown format code 'b' for object of type 'float'
1111011
sys.float_info(max=1.7976931348623157e+308, max_exp=1024, max_10_exp=308, min=2.2250738585072014e-
308, min_exp=-1021, min_10_exp=-307, dig=15, mant_dig=53, epsilon=2.220446049250313e-16, radix=2,
rounds=1)
ValueError: Unknown format code 'b' for object of type 'float'
9
9
TypeError: int() can't convert non-string with explicit base
```

In [15]:

```
print((10.75).hex())
print((12.34).hex())
print((10e-34).hex())
```

```
0x1.5800000000000p+3
0x1.8ae147ae147aep+3
0x1.4c4e977ba1f5cp-110
```

In [25]:

```
print(sys.float_info.max)
print(sys.float_info.min)
print(f"{sys.float_info.min:.1023f}")
```

```
1.7976931348623157e+308
2.2250738585072014e-308
0.00000000000
```

```
0.0000000000
```

```
except TypeError as ex:
        print(type(ex).         + ": " + ex.args[0])

try:
        print({4, 3, 2, 1, 0}[2])
except TypeError as ex:
        print(type(ex).         + ": " + ex.args[0])
```

```
TypeError: 'set' object is not subscriptable
TypeError: 'set' object is not subscriptable
TypeError: 'set' object is not subscriptable
<>:12: SyntaxWarning: 'set' object is not subscriptable; perhaps you missed a comma?
<>:12: SyntaxWarning: 'set' object is not subscriptable; perhaps you missed a comma?
/tmp/ipykernel_31787/1848623714.py:12: SyntaxWarning: 'set' object is not subscriptable; perhaps you missed a comma?
  print({4, 3, 2, 1, 0}[2])
```

In [35]:

```
try:
        print(random.choice({4, 3, 2, 1}))
except TypeError as ex:
        print(type(ex).         + ": " + ex.args[0])
```

```
TypeError: 'set' object is not subscriptable
```

In [ ]:

```
face_values = ['9', '10', 'J', 'Q', 'K', 'A']


def make_sample_space(face_values):
        return [(e1, e2) for e1 in face_values for e2 in face_values]


print(make_sample_space(face_values))
```

```
[('9', '9'), ('9', '10'), ('9', 'J'), ('9', 'Q'), ('9', 'K'), ('9', 'A'), ('10', '9'), ('10', '10'),
('10', 'J'), ('10', 'Q'), ('10', 'K'), ('10', 'A'), ('J', '9'), ('J', '10'), ('J', 'J'), ('J', 'Q'),
('J', 'K'), ('J', 'A'), ('Q', '9'), ('Q', '10'), ('Q', 'J'), ('Q', 'Q'), ('Q', 'K'), ('Q', 'A'),
('K', '9'), ('K', '10'), ('K', 'J'), ('K', 'Q'), ('K', 'K'), ('K', 'A'), ('A', '9'), ('A', '10'),
('A', 'J'), ('A', 'Q'), ('A', 'K'), ('A', 'A')]
```

In [41]:

```
sample_space = make_sample_space(face_values)


def simulate_throws_from_sample_space(sample_space, n):
        return [random.choice(sample_space) for _ in range(n)]


print(simulate_throws_from_sample_space(sample_space, 10))
```

```
[('9', 'K'), ('Q', '9'), ('Q', 'Q'), ('J', 'A'), ('A', 'A'), ('A', 'J'), ('A', 'K'), ('K', 'A'),
('10', 'Q'), ('A', 'K')]
```

In [43]:

```
def simulate_throws(face_values, n):
        return [tuple(random.choices(face_values, k = 2)) for _ in range(n)]

```

```
print(simulate_throws(face_values, 10))
```

```
[('Q', 'A'), ('K', 'Q'), ('10', '10'), ('10', '9'), ('Q', 'Q'), ('A', 'K'), ('A', 'K'), ('A', 'Q'),
('10', 'J'), ('10', 'Q')]
```

In [ ]:

```
def countOccurrences(simulationFunction, nThrows):
        input = sample_space if simulationFunction ==
simulate_throws_from_sample_space else face_values
        throws = simulationFunction(input, nThrows)
        occurrences = dict.fromkeys(face_values, 0)
        for k, v in throws:
                occurrences[k] = occurrences.get(k, 0) + 1
                occurrences[v] = occurrences.get(v, 0) + 1
        return occurrences


print(countOccurrences(simulate_throws, 10))
print(countOccurrences(simulate_throws_from_sample_space, 10))
```

```
[('K', 'A'), ('J', 'J'), ('Q', '10'), ('Q', '9'), ('K', '10'), ('K', '10'), ('K', 'Q'), ('A', '9'),
('K', '9'), ('10', '10')]
{'9': 3, '10': 5, 'J': 2, 'Q': 3, 'K': 5, 'A': 2}
[('J', 'A'), ('9', '10'), ('10', '9'), ('Q', 'J'), ('9', 'A'), ('10', 'K'), ('K', 'J'), ('A', 'K'),
('10', '9'), ('A', '9')]
{'9': 5, '10': 4, 'J': 3, 'Q': 1, 'K': 3, 'A': 4}
```

In [3]:

```
def randomFloat(low, hig):
        return low + (random.random() * (hig - low))


print(max([randomFloat(2, 4) for _ in range(100_000_000)]))
```

```
3.9999999851096346
```

In [7]:

```
def randomFloat(low, hig):
        return low + (random.random() * (hig - low))


print(max([randomFloat(2, 2.0001 + sys.float_info.min) for _ in
range(100_000_000)]))
```

```
2.000099999999641
```

In [14]:
```
print(sum([0.1] * 10))
print(f"{sum([0.1] * 10):.1074f}".rstrip("0").rstrip("."))
```

```
1.0
1
```

In [25]:
```
print(math.nextafter(1, 2, steps = 1))
print(math.nextafter(1, 2, steps = 2))
print(math.nextafter(1, 1 + sys.float_info.min, steps = 1))
print(math.nextafter(2, 3, steps = 1))
print(math.nextafter(100, 101, steps = 1))
print(math.nextafter(100_000, 101_000, steps = 1))
print(math.nextafter(0.1, 0.2, steps = 1))
print(math.nextafter(0.01, 0.02, steps = 1))
print(math.nextafter(0.001, 0.002, steps = 1))
```

```
1.0000000000000002
1.0000000000000004
1.0
2.0000000000000004
100.00000000000001
100000.00000000001
0.10000000000000002
0.010000000000000002
0.0010000000000000002
```

In [27]:
```
print(math.comb(3, 2))
print(math.perm(3, 2))
```

```
3
6
```

In [33]:
```
data = [(0, 1), (1, 2, 3), (1, 3, 5, 7), (1, 1, 2, 3, 5, 8, 13)]
print([math.hypot(*vec) for vec in data])
```

```
[1.0, 3.7416573867739413, 9.16515138991168, 16.522711641858304]
```

In [48]:
```
data = [61, 35, 99, 100, 75, 94, 88, 14, 21, 39, 53, 25, 87, 84, 81, 55, 86,
18, 69, 44, 16, 33, 66, 52, 70, 52, 95, 45, 94, 35, 68, 70, 52, 53, 30, 87, 79,
51, 92, 72, 55, 40, 15, 74, 86, 87, 91, 70, 45, 37]
print("sorted data:", sorted(data))
print("count:", len(data))
print("unique count:", len(set(data)))
print("min:", min(data))
print("max:", max(data))
print(f"mean: {statistics.fmean(data):.3f}")
print(f"std dev: {statistics.stdev(data):.3f}")
print("modes:", statistics.multimode(data))
print(f"25th percentile: {statistics.quantiles(data, n = 4)[0]:.3f}")
print(f"median: {statistics.median(data):.3f}")
print(f"75th percentile: {statistics.quantiles(data, n = 4)[2]:.3f}")


```

```
9
 1
0
 1
1
 1
2
```

```
sorted data: [14, 15, 16, 18, 21, 25, 30, 33, 35, 35, 37, 39, 40, 44, 45, 45, 51, 52, 52, 52, 53,
53, 55, 55, 61, 66, 68, 69, 70, 70, 70, 72, 74, 75, 79, 81, 84, 86, 86, 87, 87, 87, 88, 91, 92, 94,
94, 95, 99, 100]
count: 50
unique count: 38
min: 14
max: 100
mean: 60.800
std dev: 25.283
modes: [87, 52, 70]
25th percentile: 39.750
median: 63.500
75th percentile: 86.000
```

In [53]:

```
print(statistics.quantiles(range(0, 202, 2), n = 4))
print(statistics.quantiles(range(0, 202, 2), n = 4, method = "inclusive"))
print(statistics.quantiles(range(0, 202, 2), n = 4, method = "exclusive"))
```

```
[49.0, 100.0, 151.0]
[50.0, 100.0, 150.0]
[49.0, 100.0, 151.0]
```

In [55]:

```
print(statistics.quantiles(range(0, 1005, 5), n = 4))
print(statistics.quantiles(range(0, 1005, 5), n = 4, method = "inclusive"))
print(statistics.quantiles(range(0, 1005, 5), n = 4, method = "exclusive"))
```

```
[247.5, 500.0, 752.5]
[250.0, 500.0, 750.0]
[247.5, 500.0, 752.5]
```

In [82]:

```
vec1 = (1, 1)
print(math.hypot(*vec1))
print(math.hypot(1))
print(math.hypot(2))
print(math.hypot(3))
print(math.hypot(3, 3))
print(math.dist((0, 0), (3, 3)))
print(math.hypot(3, 3) == math.dist((0, 0), (3, 3)))
print(math.hypot(3, 3) == math.dist((1, 1), (3, 3)))
print(math.lcm(5, 16))
```

```
1.4142135623730951
1.0
2.0
3.0
4.242640687119285
4.242640687119285
True
False
80
```

In [90]:

```python
normalDist1 = statistics.NormalDist(mu = 0, sigma = 1)
print(normalDist1)
normalDist2 = statistics.NormalDist(mu = 0.5, sigma = 1)
print(normalDist1.overlap(normalDist2))
normalDist3 = statistics.NormalDist(mu = 1.0, sigma = 1)
print(normalDist1.overlap(normalDist3))
normalDist4 = statistics.NormalDist(mu = 2.0, sigma = 1)
print(normalDist1.overlap(normalDist4))
normalDist5 = statistics.NormalDist(mu = 3.0, sigma = 1)
print(normalDist1.overlap(normalDist5))
try:
        print(len(normalDist1))
except TypeError as ex:
        print(type(ex).         + ": " + ex.args[0])
```

```
NormalDist(mu=0.0, sigma=1.0)
0.8025873486341526
0.6170750774519739
0.31731050786291415
0.13361440253771617
TypeError: object of type 'NormalDist' has no len()
```

In [100]:

```python
print(math.inf)
print(type(math.inf))
print(math.inf - sys.float_info.min)
print(f"{(math.inf - sys.float_info.min):.1074f}")
print(f"{(math.inf - sys.float_info.max):.1074f}")
print(f"{(math.inf - sys.float_info.max * sys.float_info.max):.1074f}")
print(math.nan)
print(type(math.nan))
```

```
inf
<class 'float'>
inf
inf
inf
nan
nan
<class 'float'>
```

In [ ]:

```python
print(decimal.Decimal("0.1") + decimal.Decimal("0.1") + decimal.Decimal("0.1")
== decimal.Decimal("0.3"))
print(decimal.Decimal(0.1) + decimal.Decimal(0.1) + decimal.Decimal(0.1) ==
decimal.Decimal(0.3))
```

```
True
False
```

In [104]:
```
print(decimal.Decimal(1) / decimal.Decimal(3))
```

```
0.33333333333333333333333333333
```

In [105]:
```
print(decimal.getcontext())
```

```
Context(prec=28, rounding=ROUND_HALF_EVEN, Emin=-999999, Emax=999999, capitals=1, clamp=0, flags=
[Inexact, FloatOperation, Rounded], traps=[InvalidOperation, DivisionByZero, Overflow])
```

In [ ]:
```
import decimal as dc


print(dc.getcontext())
dc = 1
try:
        print(dc.getcontext())
except AttributeError as ex:
        print(type(ex).         + ": " + ex.args[0])
```

```
Context(prec=28, rounding=ROUND_HALF_EVEN, Emin=-999999, Emax=999999, capitals=1, clamp=0, flags=
[Inexact, FloatOperation, Rounded], traps=[InvalidOperation, DivisionByZero, Overflow])
AttributeError: 'int' object has no attribute 'getcontext'
```

In [111]:
```
print(decimal.Decimal(0.1))
print(len(str(decimal.Decimal(0.1))))
```

```
0.1000000000000000055511151231257827021181583404541015625
57
```

In [112]:
```
ctx = decimal.getcontext()
print(ctx.prec)
ctx.prec = 14
print(ctx.prec)
```

```
28
14
```

In [113]:
```
print(ctx.prec)
```

```
14
```

In [119]:
```
with decimal.localcontext() as ctx:
        print(decimal.getcontext())
        ctx.prec = 6
        print(decimal.Decimal(0.1))
        print(+decimal.Decimal(0.1))

print(decimal.getcontext())
print(+decimal.Decimal(0.1))
```

```
Context(prec=14, rounding=ROUND_HALF_EVEN, Emin=-999999, Emax=999999, capitals=1, clamp=0, flags=
[Inexact, FloatOperation, Rounded], traps=[InvalidOperation, DivisionByZero, Overflow])
0.10000000000000000555111512312578270211815834045410156 25
0.100000
Context(prec=14, rounding=ROUND_HALF_EVEN, Emin=-999999, Emax=999999, capitals=1, clamp=0, flags=
[Inexact, FloatOperation, Rounded], traps=[InvalidOperation, DivisionByZero, Overflow])
0.10000000000000
```

In [122]:
```
decimal.getcontext().prec = 28
print(+decimal.Decimal(0.1))
```

```
0.1000000000000000055511151231
```

In [126]:
```
print(float("2.4"))
print(float("-2.4"))
```

```
2.4
-2.4
```

In [144]:
```
print(decimal.getcontext())
```

```
Context(prec=28, rounding=ROUND_HALF_EVEN, Emin=-999999, Emax=999999, capitals=1, clamp=0, flags=
[Inexact, FloatOperation, Rounded], traps=[InvalidOperation, DivisionByZero, Overflow])
```

In [154]:
```
print(sum([decimal.Decimal("0.1234567890123456789"),
decimal.Decimal("0.1234567890123456789")]))
print(0.1234567890123456789 + 0.1234567890123456789)
```

```
0.2469135780246913578
0.24691357802469135
```

In [ ]:
```
print(sum([decimal.Decimal('-11.022038'), decimal.Decimal('-56.825416')]))
print(sum([decimal.Decimal('-50.402044'), decimal.Decimal('-29.852735'),
decimal.Decimal('-82.337178'), decimal.Decimal('-4.214264')]))
print(2.999999 + 3.999999)  # Adding two numbers with six fractional digits
give a number with six fractional digits only, not more.
```

```
-67.847454
-166.806221
6.999998
```

In [6]:
```
start = time.perf_counter()
with open("../var/transactions.csv") as fh:
        itr = csv.reader(fh)
        floats = []
        floatSum = 0
        next(fh)
        for row in itr:
                # floats.append(float(row[2]))
                floatSum += float(row[2])
        # floatSum = math.fsum(floats)
end = time.perf_counter()
print(end - start)
print(floatSum)

start = time.perf_counter()
```

```
with open("../var/transactions.csv") as fh:
        itr = csv.reader(fh)
        decimals = []
        # decimalSum = decimal.Decimal("0")
        next(fh)
        for row in itr:
                decimals.append(decimal.Decimal(row[2]))
                # if len(decimals) < 10:
                #       print(decimals, sum(decimals))
                # decimalSum += decimal.Decimal(row[2])
        decimalSum = sum(decimals)
end = time.perf_counter()
print(end - start)
print(decimalSum)

# print(decimalSum - decimal.Decimal(str(floatSum)))
print(decimal.Decimal(str(floatSum)) - decimalSum)
```

```
1.093041632000677
116387.51306500046
1.8709300100017572
116387.513065
4.6E-10
```

In [ ]:

```
from decimal import Decimal


f_name = "../var/transactions.csv"


def sum_amount(f_name, *, as_decimal = False):
        total = 0

        with open(f_name) as f:
                reader = csv.reader(f)
                next(f)  # skip header row
                for row in reader:
                        amount_str = row[-1]
                        if as_decimal:
                                total += Decimal(amount_str)
                        else:
                                total += float(amount_str)
        return total


total_float = sum_amount(f_name)
print(type(total_float), total_float)

total_decimal = sum_amount(f_name, as_decimal = True)
print(type(total_decimal), total_decimal)
```

```
<class 'float'> 116387.51306500046
<class 'decimal.Decimal'> 116387.513065
```

In [17]:

```
with decimal.localcontext() as ctx:
        ctx.rounding = decimal.ROUND_HALF_UP
        with open("../var/transactions.csv") as fh:
                itr = csv.reader(fh)
                next(itr)
                feeHalfUp = 0
                for row in itr:
                        amount = row[-1]
                        feeHalfUp += round((decimal.Decimal("0.123") *
abs(decimal.Decimal(amount))) / 100, 8)
                print(feeHalfUp)
```

```
125501.66978197
```

In [16]:

```
with decimal.localcontext() as ctx:
        ctx.rounding = decimal.ROUND_HALF_EVEN
        with open("../var/transactions.csv") as fh:
                itr = csv.reader(fh)
                next(itr)
                feeHalfEven = 0
                for row in itr:
                        amount = row[-1]
                        feeHalfEven += round((decimal.Decimal("0.123") *
abs(decimal.Decimal(amount))) / 100, 8)
                print(feeHalfEven)
```

```
125501.66977180
```

In [19]:

```
print(feeHalfUp - feeHalfEven)
```

```
0.00001017
```

In [ ]:

```
print(int.        )
```

```
int
```

In [5]:
```
print(__name__)
```

```
__main__
```

In [16]:
```
try:
        print(name())
except NameError as ex:
        print(type(ex).__name__ + ": " + str(ex))
```

```
NameError: name 'name' is not defined
```

In [18]:
```
print(type(type(10)))
```

```
<class 'type'>
```

In [46]:
```
class Circle:

        def __init__(self, radius):
                self.radius = radius

        def area(self):
                return math.pi * self.radius ** 2


circle = Circle(1)
print(circle.__dict__)
print(vars(circle))
print(sorted(dir(circle)))
print(sorted(circle.__dir__()))
print(sorted(dir(circle)) == sorted(circle.__dir__()))
```

```
{'radius': 1}
{'radius': 1}
['__class__', '__delattr__', '__dict__', '__dir__', '__doc__', '__eq__', '__firstlineno__',
'__format__', '__ge__', '__getattribute__', '__getstate__', '__gt__', '__hash__', '__init__',
'__init_subclass__', '__le__', '__lt__', '__module__', '__ne__', '__new__', '__reduce__',
'__reduce_ex__', '__repr__', '__setattr__', '__sizeof__', '__static_attributes__', '__str__',
'__subclasshook__', '__weakref__', 'area', 'radius']
['__class__', '__delattr__', '__dict__', '__dir__', '__doc__', '__eq__', '__firstlineno__',
'__format__', '__ge__', '__getattribute__', '__getstate__', '__gt__', '__hash__', '__init__',
'__init_subclass__', '__le__', '__lt__', '__module__', '__ne__', '__new__', '__reduce__',
'__reduce_ex__', '__repr__', '__setattr__', '__sizeof__', '__static_attributes__', '__str__',
'__subclasshook__', '__weakref__', 'area', 'radius']
True
```

In [47]:
```
print(circle.area())
```

```
3.141592653589793
```

In [ ]:
```
circle1 = Circle("radius")
try:
        circle1.area()
except TypeError as ex:
        print(type(ex).__name__ + ": " + str(ex))
```

```
TypeError: unsupported operand type(s) for ** or pow(): 'str' and 'int'
```

In [33]:

```
class Circle:

        def         (self, radius):
                if isinstance(radius, int) and radius > 0:
                        self.radius = radius
                else:
                        raise ValueError("Radius is not a positive integer")

        def area(self):
                return math.pi * self.radius ** 2


circle2 = Circle(1)
print(circle2.area())
```

```
3.141592653589793
```

In [36]:

```
try:
        circle3 = Circle("radius")
        print(circle3.area())
except ValueError as ex:
        print(type(ex).         + ": " + str(ex))
```

```
ValueError: Radius is not a positive integer
```

In [37]:

```
try:
        circle3 = Circle("100")
        print(circle3.area())
except ValueError as ex:
        print(type(ex).         + ": " + str(ex))
```

```
ValueError: Radius is not a positive integer
```

In [38]:

```
try:
        circle3 = Circle(-1)
        print(circle3.area())
except ValueError as ex:
        print(type(ex).         + ": " + str(ex))
```

```
ValueError: Radius is not a positive integer
```

In [ ]:

```
print(circle.area())
```

```
3.141592653589793
```

In [49]:

```
try:
        circle.radius = "radius"
        print(circle.area())
except TypeError as ex:
        print(type(ex).         + ": " + str(ex))
```

```
TypeError: unsupported operand type(s) for ** or pow(): 'str' and 'int'
```

In [ ]:

```
class Circle:

        def         (self, radius):
                if isinstance(radius, int) and radius > 0:
                        self._radius = radius
                else:
                        raise ValueError("Radius is not a positive integer")

        def radius(self):
                return self._radius

        def area(self):
                return math.pi * self._radius ** 2  # Can be self.radius()
also


circle4 = Circle(10)
print(circle4.radius())
print(circle4.area())
```

```
10
314.1592653589793
```

In [56]:

```
try:
        circle4._radius = "radius"
        print(circle4.area())
except TypeError as ex:
        print(type(ex).         + ": " + str(ex))
```

```
TypeError: unsupported operand type(s) for ** or pow(): 'str' and 'int'
```

In [57]:

```
print(circle4.radius)
```

```
<bound method Circle.radius of <__main__.Circle object at 0x7503522c12b0>>
```

In [61]:

```
class Circle:

        def         (self, radius):
                if isinstance(radius, int) and radius > 0:
                        self._radius = radius
                else:
                        raise ValueError("Radius is not a positive integer.")

        @property
        def radius(self):
                return self._radius

        def area(self):
                return math.pi * self.radius ** 2


circle5 = Circle(100)
print(circle5.area())
```

```
31415.926535897932
```

In [63]:

```
circle5._radius = 10
print(circle5.area())
```

```
314.1592653589793
```

In [ ]:

```
circle5._radius = -1  # No sanitizing input
print(circle5.area())
```

```
3.141592653589793
```

In [65]:

```
try:
        circle5._radius = "radius"
        print(circle5.area())
except TypeError as ex:
        print(type(ex).         + ": " + str(ex))
```

```
TypeError: unsupported operand type(s) for ** or pow(): 'str' and 'int'
```

In [67]:

```
try:
        circle5.radius = 10
        print(circle5.area())
except AttributeError as ex:
        print(type(ex).         + ": " + str(ex))
```

```
AttributeError: property 'radius' of 'Circle' object has no setter
```

In [70]:

```
class Circle:

        def         (self, radius):
                if isinstance(radius, int) and radius > 0:
                        self._radius = radius
                else:
```

```
                        raise ValueError("Radius is not a positive integer.")

        @property
        def radius(self):
                return self._radius

        @radius.setter
        def radius(self, newRadius):
                if isinstance(newRadius, int) and newRadius > 0:
                        self._radius = newRadius
                else:
                        raise ValueError("New radius is not a positive integer.")

        def area(self):
                return math.pi * self.radius ** 2


circle5 = Circle(100)
print(circle5.area())
```

```
31415.926535897932
```

In [71]:

```
circle5.radius = 1
print(circle5.area())
```

```
3.141592653589793
```

In [73]:
```
try:
        circle5.radius = -1
except ValueError as ex:
        print(type(ex).         + ": " + str(ex))
```

```
ValueError: New radius is not a positive integer.
```

In [74]:
```
print(circle5.radius)
```

```
1
```

In [76]:
```
try:
        circle5.radius = -1
except ValueError as ex:
        print(type(ex).         + ": " + str(ex))
```

```
ValueError: New radius is not a positive integer.
```

In [77]:
```
print(circle5.radius)
```

```
1
```

In [80]:
```
try:
        circle5.radius = -1
except ValueError as ex:
        print(type(ex).         + ": " + str(ex))
```

```
ValueError: New radius is not a positive integer.
```

In [78]:
```
class Circle:

        def         (self, radius):
                self.radius = radius

        @property
        def radius(self):
                return self._radius

        @radius.setter
        def radius(self, newRadius):
                if isinstance(newRadius, int) and newRadius > 0:
                        self._radius = newRadius
                else:
                        raise ValueError("New radius is not a positive
integer.")

        def area(self):
                return math.pi * self.radius ** 2


circle5 = Circle(100)
print(circle5.area())
```

```
```

```
31415.926535897932
```

In [79]:
```
print(circle5.radius)
```

```
100
```

In [81]:
```
try:
        circle5.radius = -1
except ValueError as ex:
        print(type(ex).         + ": " + str(ex))
```

```
ValueError: New radius is not a positive integer.
```

In [7]:
```
class Pencil:

        def         (self, brand, model, color):
                self.brand = brand
                self.model = model
                self.color = color

        def        (self):
                return f"This is a {self.color} {self.model} pencil from the
{self.brand} brand"

        def         (self):
                return f"This is a repr representation of a {self.color}
{self.model} pencil from the {self.brand} brand"


kuru_toga = Pencil("Uni", "Kuru Toga", "white")
print(kuru_toga)
```

```
0
 1
1
 1
2
 1
3
 1
4
 1
5
 1
6
```

```
This is a white Kuru Toga pencil from the Uni brand
```

In [11]:
```
1  kuru_toga
```

Out[11]:
```
This is a repr representation of a white Kuru Toga pencil from the Uni brand
```

In [12]:
```
1  print(dir(20))
```

```
['__abs__', '__add__', '__and__', '__bool__', '__ceil__', '__class__', '__delattr__', '__dir__',
'__divmod__', '__doc__', '__eq__', '__float__', '__floor__', '__floordiv__', '__format__', '__ge__',
'__getattribute__', '__getnewargs__', '__getstate__', '__gt__', '__hash__', '__index__', '__init__',
'__init_subclass__', '__int__', '__invert__', '__le__', '__lshift__', '__lt__', '__mod__',
'__mul__', '__ne__', '__neg__', '__new__', '__or__', '__pos__', '__pow__', '__radd__', '__rand__',
'__rdivmod__', '__reduce__', '__reduce_ex__', '__repr__', '__rfloordiv__', '__rlshift__',
'__rmod__', '__rmul__', '__ror__', '__round__', '__rpow__', '__rrshift__', '__rshift__', '__rsub__',
'__rtruediv__', '__rxor__', '__setattr__', '__sizeof__', '__str__', '__sub__', '__subclasshook__',
'__truediv__', '__trunc__', '__xor__', 'as_integer_ratio', 'bit_count', 'bit_length', 'conjugate',
'denominator', 'from_bytes', 'imag', 'is_integer', 'numerator', 'real', 'to_bytes']
```

In [10]:
```
    class Pencil:
1
        def         (self, brand, model, color):
2               self.brand = brand
                self.model = model
3               self.color = color

4       def        (self):
                return f"This is a {self.color} {self.model} pencil from the
5   {self.brand} brand"

6       def         (self):
                return f"This is a repr representation of a {self.color}
7   {self.model} pencil from the {self.brand} brand"

8
    kuru_toga = Pencil("Uni", "Kuru Toga", "white")
9   print(repr(kuru_toga))
 1
0
 1
```

```
This is a repr representation of a white Kuru Toga pencil from the Uni brand
```

In [14]:

```
class Pencil:

        def         (self, brand, model, color):
                self.brand = brand
                self.model = model
                self.color = color

        def         (self):
                return f"This is a repr representation of a {self.color}
{self.model} pencil from the {self.brand} brand"


kuru_toga = Pencil("Uni", "Kuru Toga", "white")
print(kuru_toga)
```

```
This is a repr representation of a white Kuru Toga pencil from the Uni brand
```

In [15]:

```
print(str(kuru_toga))
```

```
This is a repr representation of a white Kuru Toga pencil from the Uni brand
```

In [9]:

```
class Pencil:

        def         (self, brand, model, color):
                self.brand = brand
                self.model = model
                self.color = color
```

```
kuru_toga = Pencil("Uni", "Kuru Toga", "white")
print(kuru_toga)
```

```
<__main__.Pencil object at 0x773af8670050>
```

In [26]:

```
class Pencil:

        def          (self, brand, model, color):
                self._brand = brand
                self._model = model
                self._color = color

        def        (self):
                return f"This is a {self.color} {self.model} pencil from the
{self.brand} brand"

        @property
        def brand(self):
                return self._brand

        @property
        def model(self):
                return self._model

        @property
        def color(self):
                return self._color

        @brand.setter
        def brand(self, brandValue):
                self._brand = brandValue

        @model.setter
        def model(self, modelValue):
                self._model = modelValue

        @color.setter
        def color(self, colorValue):
                self._color = colorValue


kuru_toga = Pencil("Uni", "Kuru Toga", "white")
print(kuru_toga)
kuru_toga.brand = "Uni-Ball"
kuru_toga.color = "Yellow"
print(kuru_toga)
```

```
This is a white Kuru Toga pencil from the Uni brand
This is a Yellow Kuru Toga pencil from the Uni-Ball brand
```

In [29]:

```
class Pencil:

        def         (self, brand, model, color):
                self._brand = brand
                self._model = model
                self._color = color
                return "Yellow"

        def        (self):
                return f"This is a {self.color} {self.model} pencil from the
{self.brand} brand"

        @property
        def brand(self):
```

```
                return self._brand

        @property
        def model(self):
                return self._model

        @property
        def color(self):
                return self._color

        @brand.setter
        def brand(self, brandValue):
                self._brand = brandValue

        @model.setter
        def model(self, modelValue):
                self._model = modelValue

        @color.setter
        def color(self, colorValue):
                self._color = colorValue


try:
        kuru_toga = Pencil("Uni", "Kuru Toga", "white")
except TypeError as ex:
        print(type(ex).      )
print(kuru_toga)
kuru_toga.brand = "Uni-Ball"
kuru_toga.color = "Yellow"
print(kuru_toga)
```

```
TypeError
This is a Yellow Kuru Toga pencil from the Uni-Ball brand
This is a Yellow Kuru Toga pencil from the Uni-Ball brand
```

In [33]:

```python
class Pencil:

        def          (self, brand, model, color):
                self._brand = brand
                self._model = model
                self._color = color
                print("Yellow from init!")

        def        (self):
                return f"This is a {self.color} {self.model} pencil from the {self.brand} brand"

        @property
        def brand(self):
                return self._brand

        @property
        def model(self):
                return self._model

        @property
        def color(self):
                return self._color

        @brand.setter
        def brand(self, brandValue):
```

```
                self._brand = brandValue

        @model.setter
        def model(self, modelValue):
                self._model = modelValue

        @color.setter
        def color(self, colorValue):
                self._color = colorValue


kuru_toga = Pencil("Uni", "Kuru Toga", "white")
print(kuru_toga)
kuru_toga.brand = "Uni-Ball"
kuru_toga.color = "Yellow"
print(kuru_toga)
```

```
Yellow from init!
This is a white Kuru Toga pencil from the Uni brand
This is a Yellow Kuru Toga pencil from the Uni-Ball brand
```

In [75]:

```
class OverdraftNotAllowedError(Exception):
        print("EXCEPTION!!!")
```

```
EXCEPTION!!!
```

In [86]:

```
class Account:

        def         (self, firstName, lastName, accountNumber, balance,
isOverdraftAllowed):
                self._firstName = firstName
                self._lastName = lastName
                self._accountNumber = accountNumber
                self._isOverdraftAllowed = isOverdraftAllowed
                self._ledger = []
                self._balance = balance

        @property
        def balance(self):
                return self._balance

        def getLedger(self):
                return self._ledger

        def addToLedger(self, transactionDate, transactionAmount):
                self._ledger.append([transactionDate, transactionAmount])

        def updateBalance(self, transactionAmount):
                self._balance += transactionAmount

        @property
        def isOverdraftAllowed(self):
                return self._isOverdraftAllowed

        @isOverdraftAllowed.setter
        def isOverdraftAllowed(self, allowed):
                self._isOverdraftAllowed = allowed

        def deposit(self, transactionAmount):
                if transactionAmount < 0:
                        raise ValueError("Amount is not positive")
                self._balance += transactionAmount

self.addToLedger(datetime.datetime.now().astimezone(datetime.timezone.utc).iso
format(), transactionAmount)
```

```

        def withdraw(self, transactionAmount):
                if self._balance >= transactionAmount or
self.isOverdraftAllowed:
                        self._balance -= transactionAmount
                elif not self.isOverdraftAllowed:
                        raise OverdraftNotAllowedError("Not allowed")
                elif transactionAmount > 0:
                        raise ValueError("Amount is not positive")


self.addToLedger(datetime.datetime.now().astimezone(datetime.timezone.utc).iso
format(), -transactionAmount)

        def        (self):
                return f"{self._firstName} {self._lastName}
({self._accountNumber}): {self._balance}"
```

```
6
4
7
4
8
4
9
```

In [87]:

```
myAccount = Account("Punit", "Arya", 1001, 100.00, False)
print(myAccount)
myAccount.deposit(20)
print(myAccount)
myAccount.withdraw(80)
print(myAccount.getLedger())
print(myAccount.balance)
try:
        myAccount.withdraw(80)
except OverdraftNotAllowedError as ex:
        print(type(ex).         + ": " + str(ex))
print(myAccount.balance)
```

```
Punit Arya (1001): 100.0
Punit Arya (1001): 120.0
[['2026-10-08T20:09:28.607203+00:00', 20], ['2026-10-08T20:09:28.607271+00:00', -80]]
40.0
OverdraftNotAllowedError: Not allowed
40.0
```

In [88]:

```
myAccount = Account("Punit", "Arya", 1001, 100.00, True)
print(myAccount)
myAccount.deposit(20)
print(myAccount)
myAccount.withdraw(80)
print(myAccount.getLedger())
print(myAccount.balance)
myAccount.withdraw(80)
print(myAccount.balance)
```

```
Punit Arya (1001): 100.0
Punit Arya (1001): 120.0
[['2026-10-08T20:09:32.093242+00:00', 20], ['2026-10-08T20:09:32.093304+00:00', -80]]
40.0
-40.0
```

In [91]:

```
herAccount = Account("Lalvi", "Arya", 1001, 100.00, False)
print(herAccount)
herAccount.deposit(20)
print(herAccount)
herAccount.withdraw(80)
print(herAccount.getLedger())
print(herAccount.balance)
try:
        herAccount.withdraw(80)
except OverdraftNotAllowedError as ex:
```

```
        print(type(ex).          + ": " + str(ex))
print(herAccount.balance)
print(herAccount.isOverdraftAllowed)
herAccount.isOverdraftAllowed = True
print(herAccount.isOverdraftAllowed)
herAccount.withdraw(80)
print(herAccount.balance)
```

```
Lalvi Arya (1001): 100.0
Lalvi Arya (1001): 120.0
[['2026-10-08T20:10:09.969771+00:00', 20], ['2026-10-08T20:10:09.969836+00:00', -80]]
40.0
OverdraftNotAllowedError: Not allowed
40.0
False
True
-40.0
```

In [92]:

```
class Account:

        def         (self, firstName, lastName, accountNumber, balance,
isOverdraftAllowed):
                self._firstName = firstName
                self._lastName = lastName
                self._accountNumber = accountNumber
                self._isOverdraftAllowed = isOverdraftAllowed
                self._ledger = []
                self._balance = balance

        @property
        def balance(self):
                return self._balance

        def getLedger(self):
                return self._ledger

        def addToLedger(self, transactionDate, transactionAmount):
                self._ledger.append([transactionDate, transactionAmount])

        def updateBalance(self, transactionAmount):
                self._balance += transactionAmount

        @property
        def isOverdraftAllowed(self):
                return self._isOverdraftAllowed

        @isOverdraftAllowed.setter
        def isOverdraftAllowed(self, allowed):
                self._isOverdraftAllowed = allowed

        def deposit(self, transactionAmount):
                if transactionAmount < 0:
                        raise ValueError("Amount is not positive")
                self._balance += transactionAmount
```

```

self.addToLedger(datetime.datetime.now().astimezone(datetime.timezone.utc).iso
format(), transactionAmount)

        def withdraw(self, transactionAmount):
                if self._balance >= transactionAmount or
self.isOverdraftAllowed:
                        self._balance -= transactionAmount
                elif not self.isOverdraftAllowed:
                        raise OverdraftNotAllowedError("Not allowed")
                elif transactionAmount > 0:
                        raise ValueError("Amount is not positive")


self.addToLedger(datetime.datetime.now().astimezone(datetime.timezone.utc).iso
format(), -transactionAmount)

        def        (self):
                return f"{self._firstName} {self._lastName}
({self._accountNumber}): {self._balance}"

        def       (self, account2):
                return self._accountNumber == account2._accountNumber
```

```
4
5
4
6
4
7
4
8
4
9
5
0
5
1
5
2
```

In [93]:

```
myAccount = Account("Punit", "Arya", 1000, 100.00, False)
herAccount = Account("Lalvi", "Arya", 1001, 100.00, False)
print(myAccount == herAccount)
```

```
False
```

In [94]:

```
myAccount = Account("Punit", "Arya", 1001, 100.00, False)
herAccount = Account("Lalvi", "Arya", 1001, 100.00, False)
print(myAccount == herAccount)
```

```
True
```

In [118]:

```
random.seed(-1)
d = statistics.NormalDist(mu = 3, sigma = 5)
samples = d.samples(50)
print(round(statistics.median(samples), 2))
```

```
2.34
```

In [125]:

```
random.seed(-1)

dist = statistics.NormalDist(mu = 3, sigma = 5)
# sample = [dist for _ in range(50)]
# print(round(statistics.median(sample), 2))
# print(statistics.median(sample))

sample = [random.gauss(3, 5) for _ in range(50)]
print(round(statistics.median(sample), 2))
```

```
3.83
```

In [110]:

```
print(sum(samples) / 50)
print(min(samples))
```

```
2.424038880302O5
-11.309120249622138
```

In [128]:

```
print(pytz.all_timezones)
print(len(pytz.all_timezones))
```

```
['Africa/Abidjan', 'Africa/Accra', 'Africa/Addis_Ababa', 'Africa/Algiers', 'Africa/Asmara',
'Africa/Asmera', 'Africa/Bamako', 'Africa/Bangui', 'Africa/Banjul', 'Africa/Bissau',
'Africa/Blantyre', 'Africa/Brazzaville', 'Africa/Bujumbura', 'Africa/Cairo', 'Africa/Casablanca',
'Africa/Ceuta', 'Africa/Conakry', 'Africa/Dakar', 'Africa/Dar_es_Salaam', 'Africa/Djibouti',
'Africa/Douala', 'Africa/El_Aaiun', 'Africa/Freetown', 'Africa/Gaborone', 'Africa/Harare',
'Africa/Johannesburg', 'Africa/Juba', 'Africa/Kampala', 'Africa/Khartoum', 'Africa/Kigali',
'Africa/Kinshasa', 'Africa/Lagos', 'Africa/Libreville', 'Africa/Lome', 'Africa/Luanda',
'Africa/Lubumbashi', 'Africa/Lusaka', 'Africa/Malabo', 'Africa/Maputo', 'Africa/Maseru',
'Africa/Mbabane', 'Africa/Mogadishu', 'Africa/Monrovia', 'Africa/Nairobi', 'Africa/Ndjamena',
'Africa/Niamey', 'Africa/Nouakchott', 'Africa/Ouagadougou', 'Africa/Porto-Novo', 'Africa/Sao_Tome',
'Africa/Timbuktu', 'Africa/Tripoli', 'Africa/Tunis', 'Africa/Windhoek', 'America/Adak',
'America/Anchorage', 'America/Anguilla', 'America/Antigua', 'America/Araguaina',
'America/Argentina/Buenos_Aires', 'America/Argentina/Catamarca', 'America/Argentina/ComodRivadavia',
'America/Argentina/Cordoba', 'America/Argentina/Jujuy', 'America/Argentina/La_Rioja',
'America/Argentina/Mendoza', 'America/Argentina/Rio_Gallegos', 'America/Argentina/Salta',
'America/Argentina/San_Juan', 'America/Argentina/San_Luis', 'America/Argentina/Tucuman',
'America/Argentina/Ushuaia', 'America/Aruba', 'America/Asuncion', 'America/Atikokan',
'America/Atka', 'America/Bahia', 'America/Bahia_Banderas', 'America/Barbados', 'America/Belem',
'America/Belize', 'America/Blanc-Sablon', 'America/Boa_Vista', 'America/Bogota', 'America/Boise',
'America/Buenos_Aires', 'America/Cambridge_Bay', 'America/Campo_Grande', 'America/Cancun',
'America/Caracas', 'America/Catamarca', 'America/Cayenne', 'America/Cayman', 'America/Chicago',
'America/Chihuahua', 'America/Ciudad_Juarez', 'America/Coral_Harbour', 'America/Cordoba',
'America/Costa_Rica', 'America/Coyhaique', 'America/Creston', 'America/Cuiaba', 'America/Curacao',
'America/Danmarkshavn', 'America/Dawson', 'America/Dawson_Creek', 'America/Denver',
'America/Detroit', 'America/Dominica', 'America/Edmonton', 'America/Eirunepe',
'America/El_Salvador', 'America/Ensenada', 'America/Fort_Nelson', 'America/Fort_Wayne',
'America/Fortaleza', 'America/Glace_Bay', 'America/Godthab', 'America/Goose_Bay',
'America/Grand_Turk', 'America/Grenada', 'America/Guadeloupe', 'America/Guatemala',
'America/Guayaquil', 'America/Guyana', 'America/Halifax', 'America/Havana', 'America/Hermosillo',
'America/Indiana/Indianapolis', 'America/Indiana/Knox', 'America/Indiana/Marengo',
'America/Indiana/Petersburg', 'America/Indiana/Tell_City', 'America/Indiana/Vevay',
'America/Indiana/Vincennes', 'America/Indiana/Winamac', 'America/Indianapolis', 'America/Inuvik',
'America/Iqaluit', 'America/Jamaica', 'America/Jujuy', 'America/Juneau',
'America/Kentucky/Louisville', 'America/Kentucky/Monticello', 'America/Knox_IN',
'America/Kralendijk', 'America/La_Paz', 'America/Lima', 'America/Los_Angeles', 'America/Louisville',
'America/Lower_Princes', 'America/Maceio', 'America/Managua', 'America/Manaus', 'America/Marigot',
'America/Martinique', 'America/Matamoros', 'America/Mazatlan', 'America/Mendoza',
'America/Menominee', 'America/Merida', 'America/Metlakatla', 'America/Mexico_City',
'America/Miquelon', 'America/Moncton', 'America/Monterrey', 'America/Montevideo',
'America/Montreal', 'America/Montserrat', 'America/Nassau', 'America/New_York', 'America/Nipigon',
'America/Nome', 'America/Noronha', 'America/North_Dakota/Beulah', 'America/North_Dakota/Center',
'America/North_Dakota/New_Salem', 'America/Nuuk', 'America/Ojinaga', 'America/Panama',
'America/Pangnirtung', 'America/Paramaribo', 'America/Phoenix', 'America/Port-au-Prince',
'America/Port_of_Spain', 'America/Porto_Acre', 'America/Porto_Velho', 'America/Puerto_Rico',
'America/Punta_Arenas', 'America/Rainy_River', 'America/Rankin_Inlet', 'America/Recife',
'America/Regina', 'America/Resolute', 'America/Rio_Branco', 'America/Rosario',
'America/Santa_Isabel', 'America/Santarem', 'America/Santiago', 'America/Santo_Domingo',
'America/Sao_Paulo', 'America/Scoresbysund', 'America/Shiprock', 'America/Sitka',
'America/St_Barthelemy', 'America/St_Johns', 'America/St_Kitts', 'America/St_Lucia',
'America/St_Thomas', 'America/St_Vincent', 'America/Swift_Current', 'America/Tegucigalpa',
'America/Thule', 'America/Thunder_Bay', 'America/Tijuana', 'America/Toronto', 'America/Tortola',
'America/Vancouver', 'America/Virgin', 'America/Whitehorse', 'America/Winnipeg', 'America/Yakutat',
'America/Yellowknife', 'Antarctica/Casey', 'Antarctica/Davis', 'Antarctica/DumontDUrville',
'Antarctica/Macquarie', 'Antarctica/Mawson', 'Antarctica/McMurdo', 'Antarctica/Palmer',
'Antarctica/Rothera', 'Antarctica/South_Pole', 'Antarctica/Syowa', 'Antarctica/Troll',
'Antarctica/Vostok', 'Arctic/Longyearbyen', 'Asia/Aden', 'Asia/Almaty', 'Asia/Amman', 'Asia/Anadyr',
'Asia/Aqtau', 'Asia/Aqtobe', 'Asia/Ashgabat', 'Asia/Ashkhabad', 'Asia/Atyrau', 'Asia/Baghdad',
'Asia/Bahrain', 'Asia/Baku', 'Asia/Bangkok', 'Asia/Barnaul', 'Asia/Beirut', 'Asia/Bishkek',
'Asia/Brunei', 'Asia/Calcutta', 'Asia/Chita', 'Asia/Choibalsan', 'Asia/Chongqing', 'Asia/Chungking',
'Asia/Colombo', 'Asia/Dacca', 'Asia/Damascus', 'Asia/Dhaka', 'Asia/Dili', 'Asia/Dubai',
'Asia/Dushanbe', 'Asia/Famagusta', 'Asia/Gaza', 'Asia/Harbin', 'Asia/Hebron', 'Asia/Ho_Chi_Minh',
```

```
'Asia/Hong_Kong', 'Asia/Hovd', 'Asia/Irkutsk', 'Asia/Istanbul', 'Asia/Jakarta', 'Asia/Jayapura',
'Asia/Jerusalem', 'Asia/Kabul', 'Asia/Kamchatka', 'Asia/Karachi', 'Asia/Kashgar', 'Asia/Kathmandu',
'Asia/Katmandu', 'Asia/Khandyga', 'Asia/Kolkata', 'Asia/Krasnoyarsk', 'Asia/Kuala_Lumpur',
'Asia/Kuching', 'Asia/Kuwait', 'Asia/Macao', 'Asia/Macau', 'Asia/Magadan', 'Asia/Makassar',
'Asia/Manila', 'Asia/Muscat', 'Asia/Nicosia', 'Asia/Novokuznetsk', 'Asia/Novosibirsk', 'Asia/Omsk',
'Asia/Oral', 'Asia/Phnom_Penh', 'Asia/Pontianak', 'Asia/Pyongyang', 'Asia/Qatar', 'Asia/Qostanay',
'Asia/Qyzylorda', 'Asia/Rangoon', 'Asia/Riyadh', 'Asia/Saigon', 'Asia/Sakhalin', 'Asia/Samarkand',
'Asia/Seoul', 'Asia/Shanghai', 'Asia/Singapore', 'Asia/Srednekolymsk', 'Asia/Taipei',
'Asia/Tashkent', 'Asia/Tbilisi', 'Asia/Tehran', 'Asia/Tel_Aviv', 'Asia/Thimbu', 'Asia/Thimphu',
'Asia/Tokyo', 'Asia/Tomsk', 'Asia/Ujung_Pandang', 'Asia/Ulaanbaatar', 'Asia/Ulan_Bator',
'Asia/Urumqi', 'Asia/Ust-Nera', 'Asia/Vientiane', 'Asia/Vladivostok', 'Asia/Yakutsk', 'Asia/Yangon',
'Asia/Yekaterinburg', 'Asia/Yerevan', 'Atlantic/Azores', 'Atlantic/Bermuda', 'Atlantic/Canary',
'Atlantic/Cape_Verde', 'Atlantic/Faeroe', 'Atlantic/Faroe', 'Atlantic/Jan_Mayen',
'Atlantic/Madeira', 'Atlantic/Reykjavik', 'Atlantic/South_Georgia', 'Atlantic/St_Helena',
'Atlantic/Stanley', 'Australia/ACT', 'Australia/Adelaide', 'Australia/Brisbane',
'Australia/Broken_Hill', 'Australia/Canberra', 'Australia/Currie', 'Australia/Darwin',
'Australia/Eucla', 'Australia/Hobart', 'Australia/LHI', 'Australia/Lindeman', 'Australia/Lord_Howe',
'Australia/Melbourne', 'Australia/NSW', 'Australia/North', 'Australia/Perth',
'Australia/Queensland', 'Australia/South', 'Australia/Sydney', 'Australia/Tasmania',
'Australia/Victoria', 'Australia/West', 'Australia/Yancowinna', 'Brazil/Acre', 'Brazil/DeNoronha',
'Brazil/East', 'Brazil/West', 'CET', 'CST6CDT', 'Canada/Atlantic', 'Canada/Central',
'Canada/Eastern', 'Canada/Mountain', 'Canada/Newfoundland', 'Canada/Pacific', 'Canada/Saskatchewan',
'Canada/Yukon', 'Chile/Continental', 'Chile/EasterIsland', 'Cuba', 'EET', 'EST', 'EST5EDT', 'Egypt',
'Eire', 'Etc/GMT', 'Etc/GMT+0', 'Etc/GMT+1', 'Etc/GMT+10', 'Etc/GMT+11', 'Etc/GMT+12', 'Etc/GMT+2',
'Etc/GMT+3', 'Etc/GMT+4', 'Etc/GMT+5', 'Etc/GMT+6', 'Etc/GMT+7', 'Etc/GMT+8', 'Etc/GMT+9', 'Etc/GMT-
0', 'Etc/GMT-1', 'Etc/GMT-10', 'Etc/GMT-11', 'Etc/GMT-12', 'Etc/GMT-13', 'Etc/GMT-14', 'Etc/GMT-2',
'Etc/GMT-3', 'Etc/GMT-4', 'Etc/GMT-5', 'Etc/GMT-6', 'Etc/GMT-7', 'Etc/GMT-8', 'Etc/GMT-9',
'Etc/GMT0', 'Etc/Greenwich', 'Etc/UCT', 'Etc/UTC', 'Etc/Universal', 'Etc/Zulu', 'Europe/Amsterdam',
'Europe/Andorra', 'Europe/Astrakhan', 'Europe/Athens', 'Europe/Belfast', 'Europe/Belgrade',
'Europe/Berlin', 'Europe/Bratislava', 'Europe/Brussels', 'Europe/Bucharest', 'Europe/Budapest',
'Europe/Busingen', 'Europe/Chisinau', 'Europe/Copenhagen', 'Europe/Dublin', 'Europe/Gibraltar',
'Europe/Guernsey', 'Europe/Helsinki', 'Europe/Isle_of_Man', 'Europe/Istanbul', 'Europe/Jersey',
'Europe/Kaliningrad', 'Europe/Kiev', 'Europe/Kirov', 'Europe/Kyiv', 'Europe/Lisbon',
'Europe/Ljubljana', 'Europe/London', 'Europe/Luxembourg', 'Europe/Madrid', 'Europe/Malta',
'Europe/Mariehamn', 'Europe/Minsk', 'Europe/Monaco', 'Europe/Moscow', 'Europe/Nicosia',
'Europe/Oslo', 'Europe/Paris', 'Europe/Podgorica', 'Europe/Prague', 'Europe/Riga', 'Europe/Rome',
'Europe/Samara', 'Europe/San_Marino', 'Europe/Sarajevo', 'Europe/Saratov', 'Europe/Simferopol',
'Europe/Skopje', 'Europe/Sofia', 'Europe/Stockholm', 'Europe/Tallinn', 'Europe/Tirane',
'Europe/Tiraspol', 'Europe/Ulyanovsk', 'Europe/Uzhgorod', 'Europe/Vaduz', 'Europe/Vatican',
'Europe/Vienna', 'Europe/Vilnius', 'Europe/Volgograd', 'Europe/Warsaw', 'Europe/Zagreb',
'Europe/Zaporozhye', 'Europe/Zurich', 'GB', 'GB-Eire', 'GMT', 'GMT+0', 'GMT-0', 'GMT0', 'Greenwich',
'HST', 'Hongkong', 'Iceland', 'Indian/Antananarivo', 'Indian/Chagos', 'Indian/Christmas',
'Indian/Cocos', 'Indian/Comoro', 'Indian/Kerguelen', 'Indian/Mahe', 'Indian/Maldives',
'Indian/Mauritius', 'Indian/Mayotte', 'Indian/Reunion', 'Iran', 'Israel', 'Jamaica', 'Japan',
'Kwajalein', 'Libya', 'MET', 'MST', 'MST7MDT', 'Mexico/BajaNorte', 'Mexico/BajaSur',
'Mexico/General', 'NZ', 'NZ-CHAT', 'Navajo', 'PRC', 'PST8PDT', 'Pacific/Apia', 'Pacific/Auckland',
'Pacific/Bougainville', 'Pacific/Chatham', 'Pacific/Chuuk', 'Pacific/Easter', 'Pacific/Efate',
'Pacific/Enderbury', 'Pacific/Fakaofo', 'Pacific/Fiji', 'Pacific/Funafuti', 'Pacific/Galapagos',
'Pacific/Gambier', 'Pacific/Guadalcanal', 'Pacific/Guam', 'Pacific/Honolulu', 'Pacific/Johnston',
'Pacific/Kanton', 'Pacific/Kiritimati', 'Pacific/Kosrae', 'Pacific/Kwajalein', 'Pacific/Majuro',
'Pacific/Marquesas', 'Pacific/Midway', 'Pacific/Nauru', 'Pacific/Niue', 'Pacific/Norfolk',
'Pacific/Noumea', 'Pacific/Pago_Pago', 'Pacific/Palau', 'Pacific/Pitcairn', 'Pacific/Pohnpei',
'Pacific/Ponape', 'Pacific/Port_Moresby', 'Pacific/Rarotonga', 'Pacific/Saipan', 'Pacific/Samoa',
'Pacific/Tahiti', 'Pacific/Tarawa', 'Pacific/Tongatapu', 'Pacific/Truk', 'Pacific/Wake',
'Pacific/Wallis', 'Pacific/Yap', 'Poland', 'Portugal', 'ROC', 'ROK', 'Singapore', 'Turkey', 'UCT',
'US/Alaska', 'US/Aleutian', 'US/Arizona', 'US/Central', 'US/East-Indiana', 'US/Eastern',
'US/Hawaii', 'US/Indiana-Starke', 'US/Michigan', 'US/Mountain', 'US/Pacific', 'US/Samoa', 'UTC',
'Universal', 'W-SU', 'WET', 'Zulu']
597
```

In [130]:
```
ist = pytz.timezone("Asia/Kolkata")
print(ist.zone)
```

```
Asia/Kolkata
```

In [134]:
```
tzAware = ist.localize(datetime.datetime.now())
print(tzAware)
```

```
2026-10-09 20:40:41.203232+05:30
```

In [141]:
```
print(datetime.UTC)
print(datetime.datetime.now(tz = datetime.UTC))
```

```
UTC
2026-10-09 15:28:32.229566+00:00
```

In [149]:
```
utcTime = datetime.datetime.now(tz = datetime.UTC)
print(type(utcTime))
timeInIST = ist.fromutc(utcTime.replace(tzinfo = None))
print(timeInIST)
```

```
<class 'datetime.datetime'>
2026-10-09 21:02:06.077073+05:30
```

In [ ]:
```
print(type(pytz.utc))
print(type(pytz.UTC))
print(pytz.utc)
print(isinstance(pytz.utc, datetime.timezone))
print(isinstance(pytz.UTC, datetime.timezone))
print(pytz.UTC == pytz.utc)
```

```
<class 'pytz.UTC'>
<class 'pytz.UTC'>
UTC
False
False
True
```

In [ ]:
```
print(isinstance(pytz.UTC, datetime.tzinfo))
print(type(datetime.timezone))
```

```
True
<class 'type'>
```

In [164]:
```
print(dateutil.parser.parse("2020/3/4"))
print(type(dateutil.parser.parse("2020/3/4")))
print(dateutil.parser.parse("3/4"))
print(dateutil.parser.parse("3/4", dayfirst = True))
```

```
2020-03-04 00:00:00
<class 'datetime.datetime'>
2026-03-04 00:00:00
2026-04-03 00:00:00
```

In [170]:
```
try:
        print(dateutil.parser.parse("Today is May the 3rd, 2020"))
```

```
except dateutil.parser.ParserError as ex:
        print(ex)
```

```
Unknown string format: Today is May the 3rd, 2020
```

In [ ]:

```
try:
        print(dateutil.parser.parse("Today is May the 3rd, 2020",
fuzzy_with_tokens = True))
except dateutil.parser.ParserError as ex:
        print(ex)
```

```
2020-05-03 00:00:00
```

In [173]:

```
try:
        print(dateutil.parser.parse("Today is May the 3rd, 2020",
fuzzy_with_tokens = True)[0])
except dateutil.parser.ParserError as ex:
        print(ex)
```

```
2020-05-03 00:00:00
```

In [ ]:

```
try:
        print(dateutil.parser.parse("May the fourth be with you",
fuzzy_with_tokens = True)[0])
except dateutil.parser.ParserError as ex:
        print(ex)
```

```
2026-05-10 00:00:00
```

In [176]:

```
try:
        print(dateutil.parser.parse("May the Fourth be with you",
fuzzy_with_tokens = True)[0])
except dateutil.parser.ParserError as ex:
        print(ex)
```

```
2026-05-10 00:00:00
```

In [177]:

```
try:
        print(dateutil.parser.parse("May the 4th be with you",
fuzzy_with_tokens = True)[0])
except dateutil.parser.ParserError as ex:
        print(ex)
```

```
2026-05-04 00:00:00
```

In [237]:

```
theGOAT = {"name": "Isaac Newton", "age": 28, "interests": ["Mechanics",
"Calculus", "Alchemy"], "books": {"Principia": 1687, "Optiks": 1704,},
"children": None, "spouses": [],}
print(type(theGOAT))
print(type(json.dumps(theGOAT)))
s = json.dumps(theGOAT)
print(type(s))
print(s)
```

```
<class 'dict'>
<class 'str'>
<class 'str'>
{"name": "Isaac Newton", "age": 28, "interests": ["Mechanics", "Calculus", "Alchemy"], "books":
{"Principia": 1687, "Optiks": 1704}, "children": null, "spouses": []}
```

In [250]:

```
theGOAT = {"name": "Isaac Newton", "age": 28, "interests": ["Mechanics",
"Calculus", "Alchemy"], "books": {"Principia": 1687, "Optiks": 1704,},
"children": None, "spouses": [],}
print(type(theGOAT))
print(type(json.dumps(theGOAT)))
s = json.dumps(theGOAT, indent = 2)
print(s)
```

```
<class 'dict'>
<class 'str'>
{
  "name": "Isaac Newton",
  "age": 28,
  "interests": [
    "Mechanics",
    "Calculus",
    "Alchemy"
  ],
  "books": {
    "Principia": 1687,
    "Optiks": 1704
  },
  "children": null,
  "spouses": []
}
```

In [238]:

```
s = json.dumps(theGOAT, indent = 2)
print(type(s))
print(s)
```

```
<class 'str'>
{
  "name": "Isaac Newton",
  "age": 28,
  "interests": [
    "Mechanics",
    "Calculus",
    "Alchemy"
  ],
  "books": {
    "Principia": 1687,
    "Optiks": 1704
  },
  "children": null,
  "spouses": []
}
```

In [235]:

```
print(type(json.loads(s)))
print(json.loads(s))
d = json.loads(s)
print(d)
```

```
<class 'dict'>
{'name': 'Isaac Newton', 'age': 28, 'interests': ['Mechanics', 'Calculus', 'Alchemy'], 'books':
{'Principia': 1687, 'Optiks': 1704}, 'children': None, 'spouses': []}
{'name': 'Isaac Newton', 'age': 28, 'interests': ['Mechanics', 'Calculus', 'Alchemy'], 'books':
{'Principia': 1687, 'Optiks': 1704}, 'children': None, 'spouses': []}
```

In [246]:

```
print(json.dumps(dict(i = 9, j = 10)))
print(json.dumps(9))
print(json.dumps("9"))
print(json.dumps("a"))
print(json.dumps(None))
print(json.dumps([-1, 2, 3.0, "4"]))
```

```
{"i": 9, "j": 10}
9
"9"
"a"
null
[-1, 2, 3.0, "4"]
```

In [247]:

```
print(type(json.dumps(dict(i = 9, j = 10))))
print(type(json.dumps(9)))
print(type(json.dumps("9")))
print(type(json.dumps("a")))
print(type(json.dumps(None)))
print(type(json.dumps([-1, 2, 3.0, "4"])))
```

```
<class 'str'>
<class 'str'>
<class 'str'>
<class 'str'>
<class 'str'>
<class 'str'>
```

In [255]:

```
try:
        theGOAT = {"name": "Isaac Newton", "dob": datetime.datetime(1643, 1,
4), "age": 28, "interests": ["Mechanics", "Calculus", "Alchemy"], "books":
{"Principia": 1687, "Optiks": 1704,}, "children": None, "spouses": [],}
        print(type(theGOAT))
        print(type(json.dumps(theGOAT)))
        s = json.dumps(theGOAT, indent = 2)
        print(s)
except TypeError as ex:
        print(ex)
```

```
<class 'dict'>
Object of type datetime is not JSON serializable
```

In [14]:

```
def mySerializer(o):
        if isinstance(o, datetime.datetime):
                return o.isoformat()
        elif isinstance(o, decimal.Decimal):
                return float(o)
        else:
```

```
            return str(o)
        raise TypeError(str(type(o)))
```

In [15]:

```
theGOAT = {"name": "Isaac Newton", "dob": datetime.datetime(1643, 1, 4), "age": 28, "interests": ["Mechanics", "Calculus", "Alchemy"], "books": {"Principia": 1687, "Optiks": 1704,}, "children": None, "spouses": [], "height": decimal.Decimal(1.68), "calculusNotation": {".", "'", "/"},}
print(json.dumps(theGOAT, indent = 2, default = mySerializer))
```

```
{
  "name": "Isaac Newton",
  "dob": "1643-01-04T00:00:00",
  "age": 28,
  "interests": [
    "Mechanics",
    "Calculus",
    "Alchemy"
  ],
  "books": {
    "Principia": 1687,
    "Optiks": 1704
  },
  "children": null,
  "spouses": [],
  "height": 1.68,
  "calculusNotation": "{'/', \"'\", '.'}"
}
```

In [59]:

```
t = "Feb 8, 2021 5:30pm (Denver Time)"
denver_time_zone = pytz.timezone("America/Denver")
print(denver_time_zone.zone)
denver_time = dateutil.parser.parse(t, fuzzy_with_tokens = True)
print(denver_time)
# denver_time = denver_time[0].replace(tzinfo = denver_time_zone)
denver_time = denver_time_zone.localize(denver_time[0])
print(denver_time.astimezone(pytz.utc).replace(tzinfo = None))
```

```
America/Denver
(datetime.datetime(2021, 2, 8, 17, 30), (' ', ' ', ' (Denver Time)'))
2021-02-09 00:30:00
```

In [41]:

```
url = 'https://en.wikipedia.org/wiki/John_von_Neumann'
headers = {"User-Agent": "fetch-jon-von-neumann-bot/0.0
(https://en.wikipedia.org/wiki/User:Punit.Arya/; arya.punit@gmail.com) python-requests/0.1"}
response = requests.get(url, headers = headers)
```

In [45]:

```
print(response.status_code)
# print(response.text)
start = response.text.find("<title>")
end = response.text.find("</title>")
print(response.text[start + 7:end])
```

```
200
John von Neumann - Wikipedia
```

In [48]:
```
url = 'https://httpbin.org/json'
response = requests.get(url)
print(response.headers["content-type"])
responseAsObj = json.loads(response.text)
print(responseAsObj)
print(type(responseAsObj))
```

```
application/json
{'slideshow': {'author': 'Yours Truly', 'date': 'date of publication', 'slides': [{'title': 'Wake up
to WonderWidgets!', 'type': 'all'}, {'items': ['Why <em>WonderWidgets</em> are great', 'Who
<em>buys</em> WonderWidgets'], 'title': 'Overview', 'type': 'all'}], 'title': 'Sample Slide Show'}}
<class 'dict'>
```

In [60]:
```
url = 'https://httpbin.org/json'
response = requests.get(url)
print(response.headers["content-type"])
responseAsObj = response.json()
print(responseAsObj)
print(type(responseAsObj))
```

```
application/json
{'slideshow': {'author': 'Yours Truly', 'date': 'date of publication', 'slides': [{'title': 'Wake up
to WonderWidgets!', 'type': 'all'}, {'items': ['Why <em>WonderWidgets</em> are great', 'Who
<em>buys</em> WonderWidgets'], 'title': 'Overview', 'type': 'all'}], 'title': 'Sample Slide Show'}}
<class 'dict'>
```

In [3]:
```
url = 'https://httpbin.org/anything'
params = {"a": 1, "b": 2}
data = {'x': 100, 'y': 200, 'z': ['a', 'b', 'c']}
response = requests.post(url, params = params, json = data)
```

In [16]:
```
print(response.status_code)
responseObj = json.loads(response.text)
pprint.pprint(responseObj, indent = 1)
```

```
200
{'args': {'a': '1', 'b': '2'},
 'data': '{"x": 100, "y": 200, "z": ["a", "b", "c"]}',
 'files': {},
 'form': {},
 'headers': {'Accept': '*/*',
             'Accept-Encoding': 'gzip, deflate, zstd',
             'Content-Length': '42',
             'Content-Type': 'application/json',
             'Host': 'httpbin.org',
             'User-Agent': 'python-requests/2.34.2',
             'X-Amzn-Trace-Id': 'Root=1-6ace94bf-30631fe1420e5598511212e0'},
 'json': {'x': 100, 'y': 200, 'z': ['a', 'b', 'c']},
 'method': 'POST',
 'origin': '223.233.77.74',
 'url': 'https://httpbin.org/anything?a=1&b=2'}
```

In [18]:
```
print(pyaml.dumps(responseObj))
```

```
args:
  a: '1'
  b: '2'
data: '{"x": 100, "y": 200, "z": ["a", "b", "c"]}'
files: {}
form: {}
headers:
  Accept: '*/*'
  Accept-Encoding: gzip, deflate, zstd
  Content-Length: '42'
  Content-Type: application/json
  Host: httpbin.org
  User-Agent: python-requests/2.34.2
  X-Amzn-Trace-Id: Root=1-6ace94bf-30631fe1420e5598511212e0
json:
  x: 100
  y: 200
  z:
    - a
    - b
    - c
method: POST
origin: 223.233.77.74
url: https://httpbin.org/anything?a=1&b=2
```

In [3]:
```
print(statistics.quantiles([5, 7, 8, 12, 13, 14, 15, 16, 20, 22, 25, 100], n = 4))
```

```
[9.0, 14.5, 21.5]
```

In [9]:
```
print(statistics.fmean([5, 7, 8, 12, 13, 14]))
```

```
9.833333333333334
```

In [10]:
```
print(statistics.median([5, 7, 8, 12, 13, 14]))
```

```
10.0
```

In [11]:
```
print(statistics.mode([5, 7, 8, 12, 13, 14]))
```

```
5
```

In [7]:
```
print(statistics.quantiles([5, 7, 8, 12, 13, 14, 15, 16, 20, 22, 25, 100], n = 4))
```

```
[9.0, 14.5, 21.5]
```

In [5]:
```
print(statistics.quantiles([30, 45, 58, 63, 88, 90], n = 4))
```

```
[41.25, 60.5, 88.5]
```

In [42]:
```
sales = list(range(60, 110, 5))
print(sales)
print(statistics.fmean(sales))
print(statistics.mode(sales))
print(statistics.stdev(sales))
print(statistics.variance(sales))
```

```
coefficientOfVariation = (statistics.stdev(sales) / statistics.fmean(sales)) *
100
print(coefficientOfVariation)
```

```
[60, 65, 70, 75, 80, 85, 90, 95, 100, 105]
82.5
60
15.138251770487459
229.16666666666666
18.349396085439345
```

In [43]:

```
revenue = list(range(30, 80, 5))
print(revenue)
print(statistics.fmean(revenue))
print(statistics.mode(revenue))
print(statistics.stdev(revenue))
print(statistics.variance(revenue))
coefficientOfVariation = (statistics.stdev(revenue) /
statistics.fmean(revenue)) * 100
print(coefficientOfVariation)
```

```
[30, 35, 40, 45, 50, 55, 60, 65, 70, 75]
52.5
30
15.138251770487459
229.16666666666666
28.83476527711897
```

In [ ]:

```
d = statistics.NormalDist(mu = 52.5, sigma = statistics.stdev(list(range(30,
80, 5))))
print(d.median)
print(d.mode)
print(d.quantiles(n = 4))
print(sorted(d.samples(10)))
```

```
52.5
52.5
[42.28940434491852, 52.5, 62.71059565508148]
[37.98143523156844, 47.03955941836336, 51.35166532572754, 54.54527891451425, 58.36289152129854,
59.62879545610937, 64.8169744388781, 65.6928540617609, 71.16988910453125, 74.39469062609953]
```

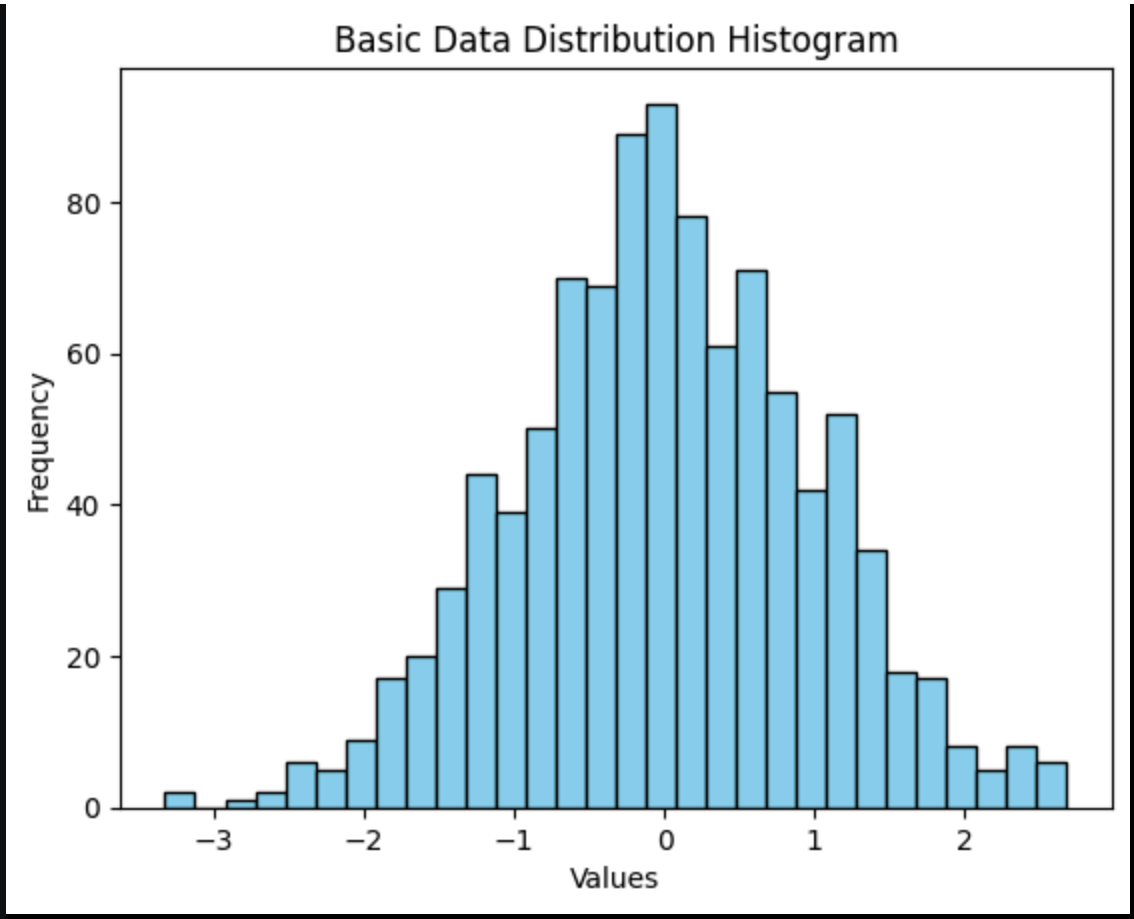
In [39]:

```
# 1. Generate or provide your series of numbers
data = np.random.normal(loc = 0, scale = 1, size = 1000)

# 2. Create the histogram
# 'edgecolor' separates the individual bars visually
plt.hist(data, bins = 30, color = "skyblue", edgecolor = "black")

# 3. Add titles and labels
plt.title("Basic Data Distribution Histogram")
plt.xlabel("Values")
plt.ylabel("Frequency")

# 4. Display the plot
plt.show()
```



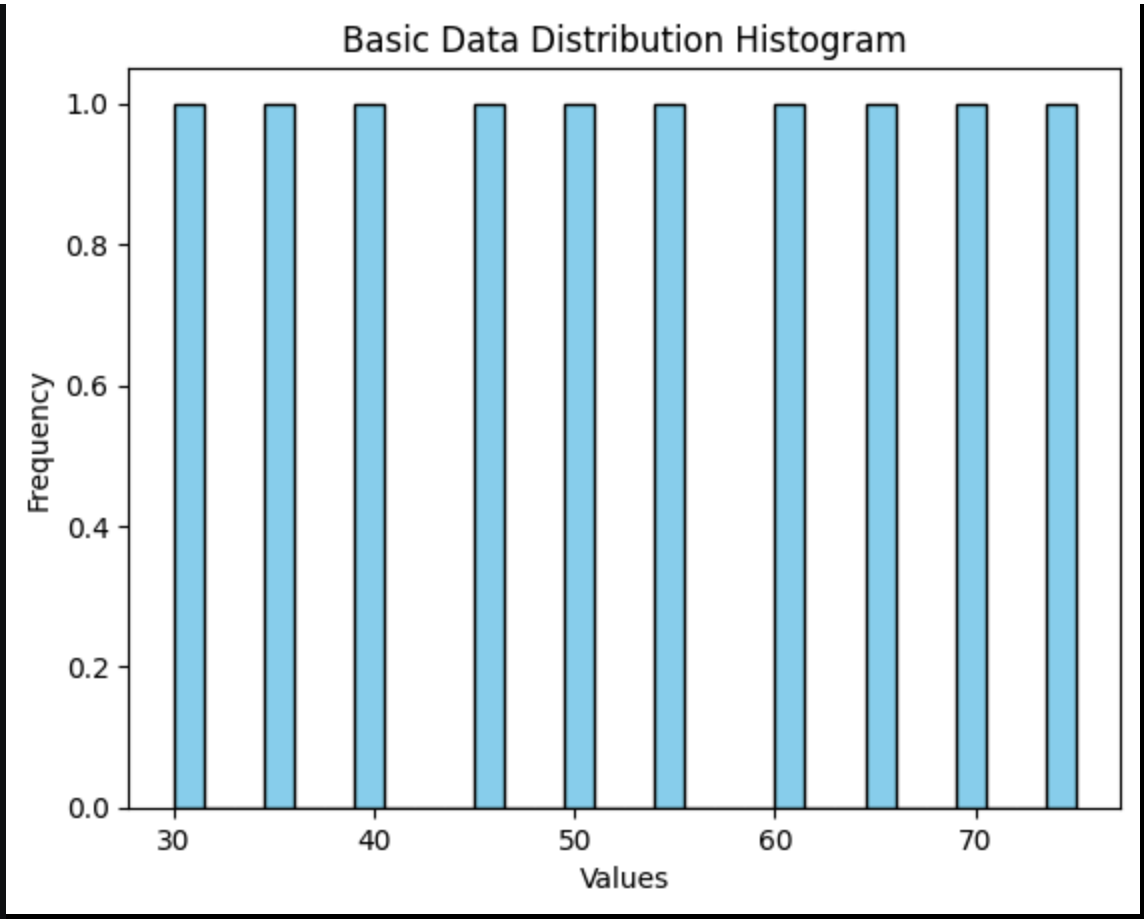
In [41]:

```python
# 1. Generate or provide your series of numbers
# data = np.random.normal(loc = 0, scale = 1, size = 1000)
data = list(range(30, 80, 5))

# 2. Create the histogram
# 'edgecolor' separates the individual bars visually
plt.hist(data, bins = 30, color = "skyblue", edgecolor = "black")

# 3. Add titles and labels
plt.title("Basic Data Distribution Histogram")
plt.xlabel("Values")
plt.ylabel("Frequency")

# 4. Display the plot
plt.show()
```



In [46]:
```
d = list(range(30, 80, 5))
print(statistics.mean(d))
print(statistics.median(d))
print(statistics.mode(d))
print((3 * (statistics.mean(d) - statistics.median(d))) / statistics.stdev(d))
```

```
52.5
52.5
30
0.0
```

In [ ]:
```
d = statistics.NormalDist(mu = 52.5, sigma = statistics.stdev(list(range(30,
80, 5))))
print(d.mean)
print(d.median)
print(d.mode)
print((3 * (d.mean - d.median)) / d.stdev)
```

```
52.5
52.5
52.5
0.0
```

In [60]:
```
print([random.randrange(2, 10, 2) for _ in range(20)])
```

```
[4, 2, 4, 4, 4, 4, 6, 8, 6, 4, 4, 6, 8, 6, 4, 6, 8, 6, 4, 8]
```

In [3]:
```
print([random.randint(2, 10) for _ in range(20)])
```

```
[7, 7, 5, 2, 5, 9, 7, 3, 8, 6, 9, 8, 4, 4, 6, 10, 5, 4, 8, 5]
```